

IPFire-Wall: Consolidated Project Report

Generated on: Fri Feb 27 03:32:37 PM CET 2026

Chapter 1: Refactoring & Optimization

This chapter documents the structural and performance improvements implemented during the refactoring phase of the IPFire-Wall project.

1.1. Modular Source Reorganization

The module was transitioned from a flat, monolithic directory structure to a specialized modular hierarchy to improve maintainability and codebase clarity.

Directory Mapping

Subdirectory	Responsibility
common/	Shared data structures, constants, and global utility functions.
filter/	The core filtering engine, rule traversal logic, and stateful hooks.
filter/state/	State machine implementation and state table management.
nat/	Translation logic for DNAT, SNAT, and Masquerade.
netlink/	Communication layer between the kernel and userspace commands.
proc/	Implementation of the /proc/ipfire interface.
includes/	Centralized repository for all internal headers.

1.2. Per-CPU Statistics Optimization

To ensure maximum performance on multi-core systems, the statistics tracking was moved from global atomics (which cause cache-line bouncing) to purely per-CPU counters.

- Infrastructure:** Counters are instances of `struct ipfi_counters` allocated via `alloc_percpu`.
- Latency Reduction:** Updates in the network hot path (like `INPUT` or `FORWARD`) use the `IPFI_STAT_INC` macro, which translates to a single instruction per-core increment.
- On-Demand Aggregation:** Global totals are only computed when a user runs `ipfire -s`, at which point the kernel iterates over all online CPUs and sums the values.

1.3. Netlink Protocol Simplification

The Netlink message structure was streamlined by removing legacy per-packet sequence IDs (`packet_id` and `logu_id`).

- Rationale:** Sequential IDs created a bottleneck for parallel packet processing.
- Reporting Transition:** Reliability is now monitored by the kernel itself. If a Netlink send fails (e.g., due to a full buffer), the kernel increments a `total_lost` counter. The userspace app now requests this counter on-demand rather than inferring loss from gaps in sequence IDs.

1.4. Build System Modernization

A shadow-tree build system was implemented using a `build/` directory.

- **Feature:** Source files are symlinked into `build/` before compilation.
- **Benefit:** All intermediate artifacts (`.o` , `.mod` , etc.) are hidden from the main source tree, keeping the environment clean for development.

1.5. Unified Table Lifecycle & Slab Allocation

The memory management for filtering tables was standardized to use a unified lifecycle and dedicated kernel slab caches (`kmem_cache`).

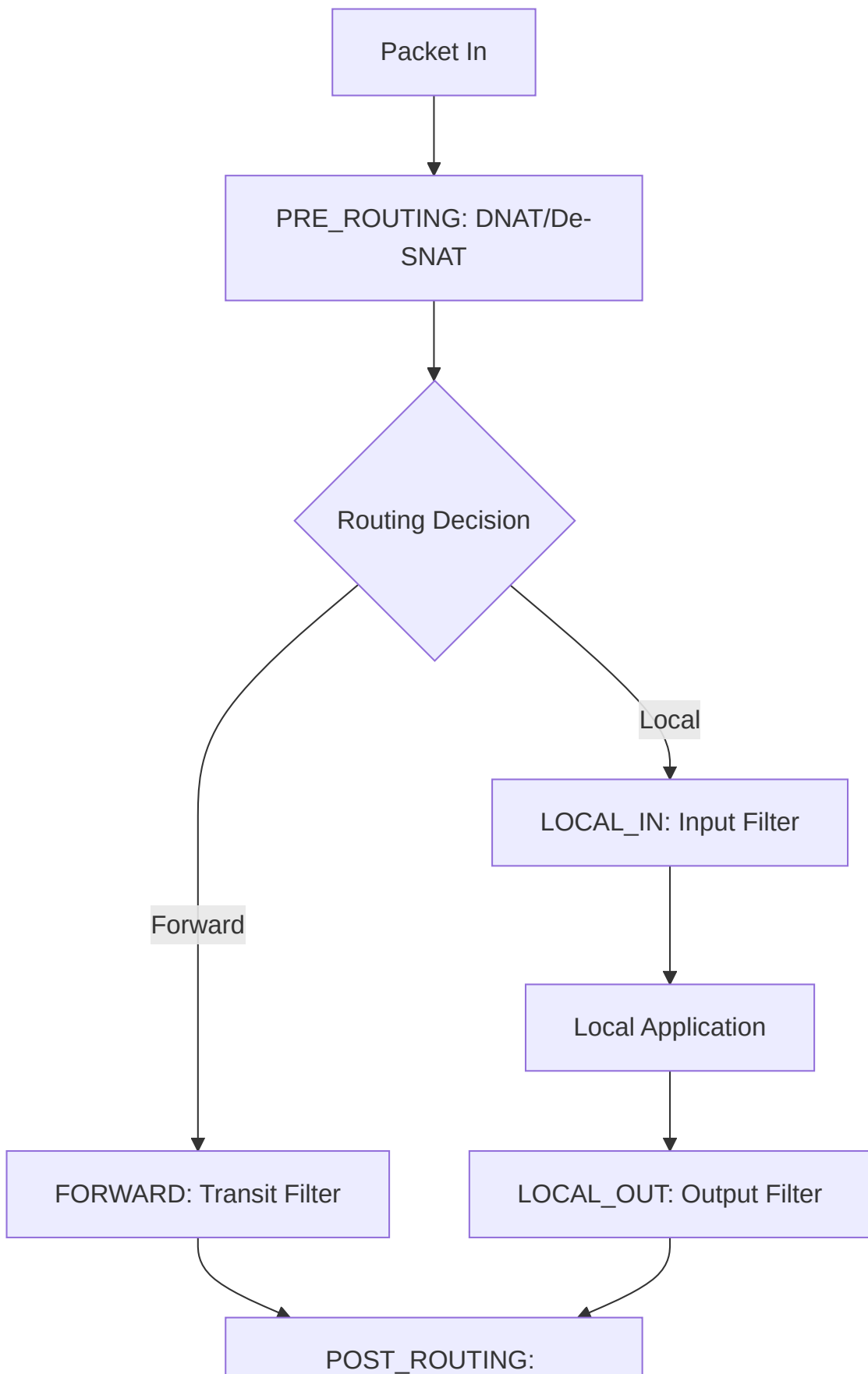
- **Unified Lifecycle:** All table entries (State, NAT, and LogInfo) now embed `struct ipfi_entry_head` , sharing consistent RCU-based synchronization and reference counting.
 - **Slab Migration:**
 - The static `loginfo_pool` was removed in favor of the `ipfi_loginfo` slab.
 - State and NAT tables were migrated from dynamic `kmalloc` to their respective `ipfi_state` and `ipfi_nat` slabs.
 - **Benefits:**
 - **Efficiency:** Slab allocation provides better cache locality and zero internal fragmentation for fixed-size entries.
 - **Consistency:** A single code path (`table_lifecycle.c`) manages the complex interactions between RCU, workqueues, and timers for all module entries.
 - **Transparency:** Memory usage is now visible per-module via `/proc/slabinfo` .
-

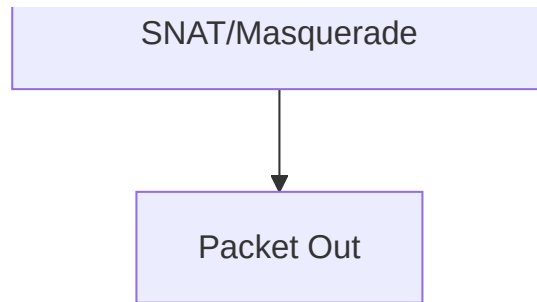
Chapter 2: Kernel Module Architecture

This chapter explains the internal workings of the IPFire-Wall kernel module, its integration with Netfilter, and how it evaluates security policies.

2.1. Netfilter Hook Integration

IPFire-Wall registers itself at several points in the Linux network stack (Hooks).





2.2. Rule Hierarchy & Evaluation Logic

When a packet hits a filtering hook (`LOCAL_IN` , `LOCAL_OUT` , or `FORWARD`), it is evaluated against several lists of rules in a prioritized sequence.

The Priority Chain:

1. **Blacklist (Dropped Rules):** The engine first scans the `dropped` list. If a match is found, the packet is discarded immediately. This ensures that blocked entities cannot bypass later permission checks.
2. **Administrator Rules (Root):** The `allowed` list is scanned for rules inserted by the system administrator.
3. **User Rules:** If enabled, rules defined by non-root users are checked next.

Evaluation Mechanism:

- **First-Match Wins:** The evaluation stops as soon as a rule matches.
- **Default Policy:** If no rule matches any list, the module applies the `default_policy` (configured as either `ACCEPT` or `DROP`).

2.3. Filtering Granularity

Rules can match packets based on a wide array of criteria:

- **L3 (IP):** Source/Destination IP addresses and IP Options.
- **L4 (Transport):** Protocol (TCP, UDP, ICMP, IGMP) and Port numbers.
- **Direction:** Inbound, Outbound, or Forwarded.
- **Interface:** Incoming or Outgoing network device indices.
- **Payload:** FTP command inspection (for dynamic NAT).

Chapter 3: Stateful Connection Management

IPFire-Wall employs a stateful inspection engine that tracks the status of network connections to improve both security and performance.

3.1. The Stateful Fast-Path

One of the core design goals is to avoid re-evaluating the entire rule list for every packet in a high-volume stream.

- **Slow Path:** The **first packet** (e.g., a TCP SYN) triggers a full scan of the rule lists. If accepted by a stateful rule, an entry is created in the `state_table` .

- **Fast Path:** Lookups for **subsequent packets** are performed against the state table using a hash-based mechanism (`jhash_3words`). This lookup is **O(1)**, meaning it stays fast regardless of how many rules are active.

3.2. State Transition Tracking

The module implements a dedicated state machine for different protocols.

TCP Connection Lifecycle:

- `SYN_SENT` : Initial request seen.
- `SYN_RECV` : Reply from server seen (SYN/ACK).
- `ESTABLISHED` : Handshake complete (ACK seen).
- `FIN_WAIT` / `CLOSE_WAIT` : Connection termination in progress.

UDP "Pseudo-States":

Since UDP is connectionless, the engine creates virtual states (`UDP_NEW` -> `UDP_ESTABLISHED`) to allow return traffic (e.g., DNS responses) through the firewall for a defined period.

3.3. Table Management

- **Lookups:** Perform bidirectional hashing. Both sides of a connection (`A:port1 <-> B:port2` and `B:port2 <-> A:port1`) produce the same hash key, allowing consistent tracking of bidirectional flows.
- **Lifetimes:** Every state entry has an associated kernel timer. If no traffic is seen for a specific duration (e.g., 3600s for ESTABLISHED TCP, or ~30s for UDP), the entry is automatically purged to free resources.
- **Allocation:** State entries are allocated from a dedicated `kmem_cache` slab (`ipfi_state`), ensuring low-latency access and optimized memory layout.
- **Namespaces:** In the current implementation, state tables are shared across all network namespaces (even with `per_net=1`).
- **Capacity:** The firewall enforces a `max_state_entries` limit to prevent resource exhaustion attacks.

3.4. FTP Support

The engine includes a specialized parser for the FTP protocol. It monitors the "control" channel for `PASV` commands and dynamically injects "Data Channel" states, allowing passive FTP to function through the NAT without requiring manual rule openings for high-numbered ports.

Chapter 4: Userspace Application (ipfire)

The `ipfire` utility is the primary administrative tool for interacting with the kernel-space firewall module.

4.1. Core Usage & CLI Flags

The application communicates with the kernel via Netlink sockets using the `IPFI_CONTROL` and `IPFI_DATA` protocols.

General Options

Flag	Description
-h, --help	Show the help message and exit.
-quiet	Do not print packets to the console.
-daemon	Run the application as a background daemon.
-quiet_daemon	Run as a background daemon and redirect standard output/error to /dev/null.
-lang <file>	Specify a language file (detected automatically by default).

Initialization & Cleanup (Root Only)

Flag	Description
-load, -rc	Load all rules from configuration files and run in the background.
-flush	Flush all rules from the kernel and exit immediately.
-noflush	Prevent the automatic flushing of rules when the userspace process exits.
-rmod	Unload the ipfire kernel module upon exit.
-clearlog	Truncate the log file at startup.

Configuration Paths

Flag	Description
-allowed <file>	Override the path for the permission rules file.
-blacklist <file>	Override the path for the denial rules file.
-blacksites <file>	Override the path for the blocked sites list.
-translation <file>	Override the path for the NAT rules file.
-logfile <file>	Override the path for the system log file (Root only).

Advanced Features & Tuning

Flag	Description
-kloglevel <0-7>	Set the kernel-space log level (0=Emergency, 7=Debug).
-loguser <0-7>	Set the userspace interface log level.
-log <0-7>	Set the logging level for the file-based logger.
-dns <seconds>	Enable the DNS resolver with a specific refresh interval.
-nodns	Disable the background DNS resolver.
-services	Enable resolution of port numbers to service names (via /etc/services).

-noservices	Disable port-to-service name resolution.
-allstate	Enable stateful tracking for ALL traffic, regardless of rule flags.
-mailer <V> <U>	Send email summaries every <V> units of <U> (sec, min, hour, days).
-user	Allow normal unprivileged users to define their own rules and interact with the firewall (Root only).
-nouser	Restrict all firewall operations to the Root user.
-no-nat	Deactivate NAT on the fly, overriding the config file (Root only).
-no-masquerade	Deactivate Masquerade on the fly, overriding the config file (Root only).

4.2. Configuration Files

The application behavior can be customized via config files. For the root user, these are located in `/etc/ipfire/`. For normal users, they are in `~/.IPFIRE/`.

- `allowed` : List of rules to be automatically loaded on startup.
- `options` : Global configuration file (compatible with IqFIREwall) using a simple `KEY=VALUE` format.
- Shared assets (default rules, documentation, help) are stored in `/usr/share/ipfire/`.

4.3. Interpreting Statistics

The `-s` (Statistics) output is divided into three sections:

1. **Userspace Stats**: Counts of packets actually received and displayed by the tool. Useful for auditing.
2. **Kernel Stats**: High-level counters for `INPUT`, `OUTPUT`, `FORWARD`, and `POST-ROUTING`.
3. **Transmission Health**: Specifically reports `total_lost` (packets the kernel tried to log but couldn't due to buffer pressure).

4.4. Logging and Real-time Monitoring

When running, `ipfire` can act as a listener, printing headers for every packet matched by a rule with the `NOTIFY` flag. These logs include:

- **Verdict Markers:**
 - `<span style="color:violet">[?X]</span>` : No matching rule was found (packet fallback to default policy).
 - `<span style="color:green">[OK N]</span>` : Packet accepted by permission rule number `N`.
 - `<span style="color:red">[X M]</span>` : Packet dropped by denial rule number `M`.
- Timestamp and user ID.
- Hook location and verdict (`ACCEPT` / `DROP`).
- Detailed IP/TCP/UDP header information.

4.5. Logging Deduplication & Technical Bounds

To prevent system instability and terminal flooding, the kernel enforces technical bounds on logging configuration. These values control the **deduplication window**: a packet hitting a rule for the first time is logged, and subsequent identical packets are suppressed until the lifetime expires.

Parameter	Default (SMB)	Enforced Range	Description
LOGINFO_LIFETIME	30s	5s – 600s	The TTL for a deduplication entry.
MAX_LOGINFO_ENTRIES	256	64 – 65536	The maximum number of distinct flows to track for deduplication.

If a user attempts to set values outside these bounds via the Netlink interface, the kernel will automatically adjust them to the nearest limit and print a warning in `dmesg`.

4.6. Installation & Systemd Integration

Building the Project

IPFire-Wall relies on two distinct compilation processes: one for the kernel module and one for the userspace application.

1. **Kernel Module (`ipfi.ko`)**: Navigate to the `kernel/` directory and use the provided `Makefile`. You will need the Linux headers for your currently running kernel installed.

```
cd kernel/
make
```

This produces the `ipfi.ko` module, which must be loaded by the `sysadmin` or automatically via the system service before starting the userspace application.

2. **Userspace Application (`ipfire`)**: The userspace tools and configuration libraries are built using CMake from the `ipfi/` subdirectory.

```
cd ipfi/
cmake -B build -DCMAKE_INSTALL_PREFIX=/usr
sudo cmake --build build --target install
```

Note: The `CMAKE_INSTALL_PREFIX` dictates where the binaries (`/bin/`), configuration files (`/etc/ipfire/`), and shared language assets (`/share/ipfire/`) will be installed. If omitted, it defaults to `/usr/local`.

Systemd Service Setup

When the userspace project is installed via `make install`, a Systemd unit file (`ipfire.service`) is automatically placed into `${CMAKE_INSTALL_PREFIX}/etc/systemd/system/`.

This service automates the lifecycle of both the kernel module and the userspace daemon:

- **Start (`systemctl start ipfire`)**: The service executes `/usr/bin/ipfire -rc -user`. This tells the userspace application to load all Root configuration rules (`-rc`) into the kernel immediately securely setting them at the highest priority. It then detaches into the background and simultaneously allows standard users (`-user`) to begin defining and managing their personal packet filtering rules.

- **Stop (`systemctl stop ipfire`):** Systemd automatically sends a `SIGTERM` signal to all processes in the service's cgroup. The `ipfire` application intercepts this signal, frees its memory, cleanly unloads from the kernel socket, and prints a comforting exit message: *"The application has been closed due to a service stop (SIGTERM)."* Afterward, the service runs `modprobe -r ipfi` to safely remove the kernel module. If a user is currently running a foreground `ipfire` instance (outside Systemd), the module removal will fail gracefully, ensuring active sessions are not interrupted.
 - **Reload (`systemctl reload ipfire`):** Systemd sends a `SIGHUP` signal to the `ipfire` main process. The application catches the `SIGHUP` and prints: *"The application has been closed due to a service reload (SIGHUP)."* The service then removes the kernel module, clearing all state, and re-executes the startup sequence (`ipfire -rc -user`) to apply fresh configurations.
-

Chapter 5: Packet Flow Walkthroughs

This chapter provides step-by-step analysis of how specific real-world packet scenarios traverse the IPFire-Wall engine.

5.1. Scenario 1: DNAT to Forwarded Internal Server

An external client connects to the firewall's public IP on port 80, which is redirected to an internal web server.

1. PRE_ROUTING Hook:

- The packet `Src:Client`, `Dst:Firewall_Public` is intercepted.
- `dnat_translation()` matches a rule and changes the destination to `Firewall_Internal_Server`.
- The kernel is notified of the `daddr` change and clears the old route.

2. Routing Decision:

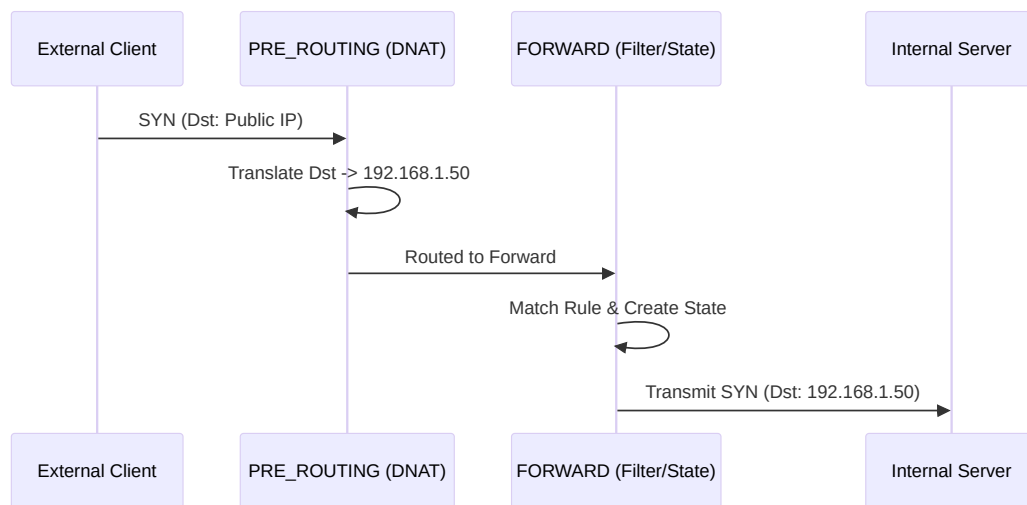
The kernel finds the route for the new internal destination; it is not local, so the packet moves to the FORWARD hook.

3. FORWARD Hook:

- `ipfire_filter()` is called.
- It identifies this as a new connection.
- It matches a stateful rule allowing traffic to the server.
- `keep_state()` creates an entry in the state table.

4. POST_ROUTING Hook:

The packet leaves the firewall towards the internal server.



5.2. Scenario 2: Masqueraded Outbound Connection

An internal client connects to the internet; the firewall hides the internal IP.

1. **LOCAL_OUT / FORWARD:** The packet is accepted by the filtering engine.
2. **POST_ROUTING:**
 - `masquerade_translation()` is called.
 - The engine dynamically finds the IP address of the outgoing WAN interface (`get_ifaddr`).
 - The packet's source address is replaced.
 - An entry is added to the `snat_table` to handle return traffic.

5.3. Scenario 3: Stateful Return Traffic

The reply from a previously accepted connection arrives.

1. **PRE_ROUTING:** If the original connection was NATed, the reverse translation is applied here.
2. **LOCAL_IN / FORWARD:**
 - `check_state()` is called.
 - A **reverse match** is found in the state table.
 - **Verdict: ACCEPT.** Rule evaluation is skipped entirely.
 - The state machine transitions (e.g., to `SYN_RECV` or `ESTABLISHED`).
3. **Delivery:** The packet is delivered to the initial requestor.

IPFire Packet Flow Walkthrough (Technical Master Edition)

This document provides exhaustive technical walkthroughs of packet flows through the IPFire kernel module. It goes beyond high-level logic to explore the internal function call stacks, specific source code segments, and low-level state transitions.

1. The Filtering Lifecycle: Hook to Verdict

Every packet handled by IPFire follows a predictable lifecycle within the kernel.

The Standard Call Stack

When a packet is intercepted by a Netfilter hook, the following internal chain is executed:

1. `ipfire.c:process()` : The primary dispatcher for all hooks.
 2. `ipfi_response()` : Orchestrates the filtering decision.
 3. `check_state()` : Performs the O(1) hash lookup for established flows.
 4. `ipfire_filter()` : (If state miss) Performs the O(n) rule comparison.
 5. `keep_state()` : (If match) Initializes a new state tracking entry.
-

2. Scenario 1: Stateful HTTP Connection (OUTPUT)

2.1. Initial SYN (The "Slow Path")

When an application initiates a connection, the first packet triggers the "Slow Path" logic.

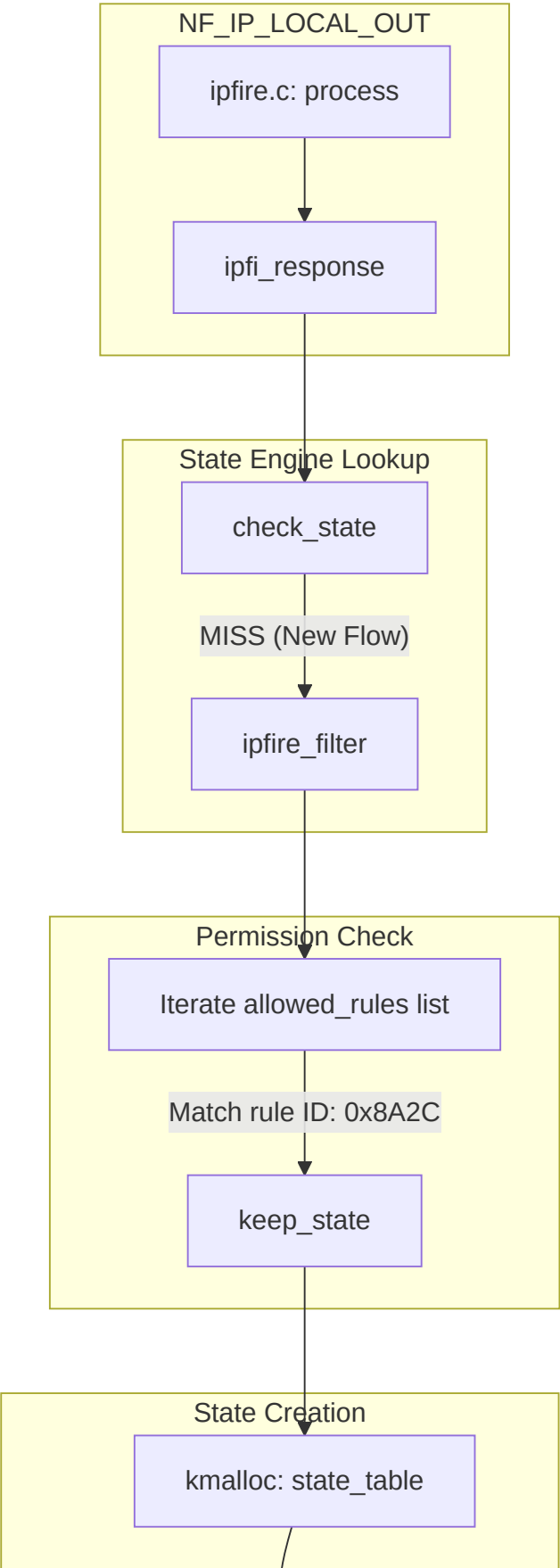
Analysis of Rule Matching Logic

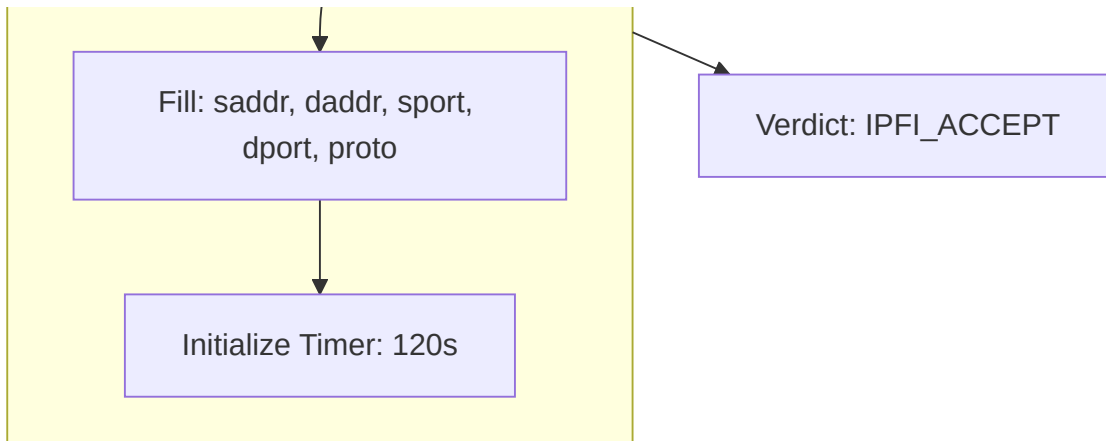
In `ipfire_filter()`, the engine iterates through the `allowed` list. For an HTTP rule, the matching sequence looks like this:

```
// Internal Matching Core (conceptual simplified C)
if (rule->direction == flow->direction &&
    (rule->protocol == 0 || rule->protocol == ip_hdr(skb)->protocol)) {

    // Transport layer check
    if (transport_protos_match(skb, rule)) {
        // Source/Dest IP check
        if (ip_match(skb, rule)) {
            return MATCH_FOUND;
        }
    }
}
```

Detailed Outgoing SYN Workflow





2.2. Returning SYN-ACK (The Transition)

The reply packet is the first "Reverse" match. This packet is critical because it transitions the state from `SYN_SENT` to `SYN_RECV`.

Internal Call: `check_state()`

The lookup in `check_state()` uses a normalized hash to find the entry regardless of direction:

```
// Hash computation normalization
u32 hash = get_state_hash(saddr, daddr, sport, dport, proto);
// The saddr/daddr are swapped in the call to get_state_hash for reverse packets
```

2.3. The Fast Path (Established Data)

Once the 3-way handshake is complete, every subsequent packet avoids the rule list entirely.

Performance Metrics

- **Slow Path (First Packet):** $O(N)$ where N is number of rules in the list.
- **Fast Path (Subsequent):** $O(1)$ Hash Table lookup + State machine update.

3. Scenario 2: Passive FTP (Dynamic NAT)

FTP is complex because it uses a control channel to negotiate a dynamic data channel.

3.1. The Payload Inspection

When `FTP_SUPPORT=YES` is enabled, the state engine invokes the FTP helper during the control connection's lifetime.

The Hook: `helpers/ftp.c`

When a packet is identified as part of an FTP control stream, the engine calls `ftp_support()`:

```
// Scanning for PASV response
if (data_start_with_227(payload)) {
    extract_ftp_info(payload, &finfo);
    if (finfo.valid) {
```

```
        add_ftp_dynamic_rule(sttable, finfo.ip, finfo.port);
    }
}
```

3.2. Data Channel "Prediction"

The `add_ftp_dynamic_rule` creates a "zombie" state entry that has no active ports yet but is pre-authorized by the control channel's policy.

4. Scenario 3: DNAT Interaction with FORWARD Hook

This is one of the most technical flows in the module because it spans across different kernel subsystem triggers.

4.1. Destination Translation (PRE_ROUTING)

The packet enters `PRE_ROUTING` and is transformed.

Feature	Logic
Transformation	<code>ip_hdr(skb)->daddr</code> is changed to the internal server IP.
Checkums	<code>ip_send_check(iph)</code> is called to recompute the L3 CRC.
Routing Trigger	The destination cache (<code>skb_dst(skb)</code>) is released to force a re-route.

```
// Forcing Re-routing in ipfi_pre_process
if (daddr_changed) {
    dst_release(skb_dst(skb));
    skb_dst_set(skb, NULL); // Kernel will re-route before next hook
}
```

4.2. State Tracking in the FORWARD Hook

Because the destination was changed to a non-local address, the kernel routes the packet to the `FORWARD` chain instead of `LOCAL_IN`.

- State Lookup:** `check_state` misses because the *modified* addresses are hashed.
- Rule Match:** The `FORWARD` rule list is scanned.
- State Creation:** A state is created based on the **post-NAT** addresses. This is critical for matching return traffic correctly.

5. Summary Table: Flow Characteristics

Phase	Hook	Path	Algorithm	Complexity
Initialization	Any	Slow	Rule List Scan	O(Rules)
Established	Any	Fast	JHash Table	O(1)
NAT (Pre)	PRE_ROUTING	Logic	Rules + Table	O(NAT_Rules)

NAT (Post)	POST_ROUTING	Logic	Dynamic Table	O(1)
------------	--------------	-------	---------------	------

TCP/UDP State Machine: Deep Dive (Technical Master Edition)

This chapter provides an exhaustive analysis of the IPFire state machine, detailing how it tracks connection lifetimes, handles exceptions, and optimizes performance for high-throughput flows.

1. TCP State Lifecycle

The state machine implements a robust subset of RFC 793 to track connection health.

1.1. State Transition Table

The following table summarizes the legal transitions within the IPFire kernel.

Current State	Event (Flags)	Next State	Logic Context
NOSTATE	SYN (Out/In)	SYN_SENT	Initial Handshake Start
SYN_SENT	SYN+ACK (Rev)	SYN_RECV	Handshake Response
SYN_RECV	ACK (Dir)	ESTABLISHED	Connection Fully Open
ESTABLISHED	FIN (Out/In)	FIN_WAIT	Teardown Start
FIN_WAIT	ACK (Rev)	CLOSE_WAIT	Two-stage Termination
ANY	RST	CLOSED	Immediate Teardown

1.2. The Three-Way Handshake Core Logic

The implementation in `ipfi_state_machine.c` ensures strict flag checking during the setup phase:

```
// handshake logic segment
if (syn && !ack && !rst && !fin) {
    if (current_state == NOSTATE) return SYN_SENT;
}

if (syn && ack && !rst && !fin && reverse) {
    if (current_state == SYN_SENT) return SYN_RECV;
}

if (!syn && ack && !rst && !fin && !reverse) {
    if (current_state == SYN_RECV) return ESTABLISHED;
}
```

2. Timer Aggregation Logic

To avoid the performance penalty of frequent timer renewals (`mod_timer` syscall overhead), IPFire implements a "1-second throttle".

2.1. Why Aggregation?

In a high-speed Gbit link, a single connection might send thousands of packets per second. Updating the kernel timer for every packet would consume excessive CPU cycles.

2.2. The Implementation

The engine only updates the timer if at least 1 second (measured in `HZ` / jiffies) has elapsed since the last update.

```
void update_timer_of_state_entry(struct state_table *sttable) {
    unsigned long now = jiffies;
    // HZ represents 1 second in kernel-time
    if (time_after(now, sttable->last_timer_update + HZ)) {
        mod_timer(&sttable->timer_statelist,
                  jiffies + get_timeout_by_state(sttable->protocol, sttable->state)
                    * HZ);
        sttable->last_timer_update = now;
    }
}
```

3. GUESS States & Normalization

GUESS states are a unique feature of IPFire that allows it to "adopt" existing connections that were established before the module was loaded.

3.1. Mid-Flow Connection Detection

If a packet arrives that is NOT a SYN but has no matching state, the machine "guesses" where it belongs.

```
// GUESS_ESTABLISHED detection
if (!rst && !fin && !syn && ack && (current_state == NOSTATE)) {
    return GUESS_ESTABLISHED;
}
```

3.2. Normalization on Storage

The engine returns a GUESS state for accounting, but **normalizes** it to a standard state before storing it in the hash table. This ensures future packets follow standard RFC transitions.

```
// Normalization logic in set_state()
if (found_state == GUESS_ESTABLISHED) {
    entry->state.state = ESTABLISHED;
} else if (found_state == GUESS_SYN_RECV) {
    entry->state.state = SYN_RECV;
}
```

4. UDP "Pseudo-States"

Since UDP is stateless, the machine uses timeout-based persistence to simulate connection tracking.

- **UDP_NEW** : Created on the first packet.
 - **UDP_ESTAB** : Promoted on the first *return* packet (where `reverse == 1`).
 - **Timeout**: Typically 30-60 seconds of inactivity.
-

5. Security & Established Flow Bypass

One of the most important design decisions is that **Established flows bypass permission rules entirely**.

Why?

1. **Consistency**: Once a connection is allowed by a rule, it should remain allowed until it closes. If a firewall rule changes mid-connection, the active stream isn't "cut" unless explicitly flushed.
 2. **Performance**: Rule lists can be long ($O(N)$). Hash table lookups are $O(1)$. By skipping the list for millions of data packets, we save massive amounts of CPU time.
-

NAT and Stateful Flow Analysis (Technical Master Edition)

This report details the packet flow and code logic for Source NAT (SNAT), Masquerade, and Destination NAT (DNAT) within the IPFire kernel module, specifically focusing on the low-level manipulation of the `sk_buff` structure and transport headers.

1. The Core Engine: `manip_skb`

All NAT operations in IPFire eventually funnel into the `manip_skb()` function. This function is responsible for the delicate task of modifying packet headers while maintaining protocol integrity (checksums).

1.1. Step-by-Step Logic

1. **L3 Address Swap**: The IP header's source or destination address is replaced.
2. **L3 Checksum Update**: Since the IP header Changed, the CRC must be updated. IPFire uses incremental updates for performance.
3. **L4 Port Swap**: For TCP/UDP, the source/destination ports are replaced.
4. **L4 Checksum Update**: Transport layer checksums (TCP/UDP) include a "pseudo-header" that contains the IP addresses. Therefore, changing an IP address REQUIRES recomputing the L4 checksum.

1.2. Incremental Checksum Logic

Instead of recalculating the entire packet checksum from scratch (which is expensive), IPFire uses the `csum_replace4` utility from the kernel:

```
// Example: Updating IP Header Checksum after address change
csum_replace4(&iph->check, oldaddr, newaddr);
```

```
// Example: Updating TCP Checksum for Pseudo-Header change
inet_proto_csum_replace4(&tcph->check, skb, oldaddr, newaddr, true);
```

2. Source NAT (SNAT) and Masquerade

2.1. SNAT Lifecycle

In `POST_ROUTING`, the engine identifies packets requiring SNAT.

1. **Rule Match:** `snat_translation()` finds a match in `translation_post`.
2. **Accounting:** `add_snatted_entry()` records the `(original_src -> new_src)` mapping. This is vital for "De-SNATting" the return traffic.
3. **Allocation:** Entries are allocated from the `ipfi_nat` slab cache for high-performance translation tracking.
4. **Transformation:** `do_source_nat()` executes the `manip_skb` logic on the source fields.

2.2. Masquerade: The Dynamic SNAT

Masquerade is identical to SNAT except it doesn't have a fixed IP. It calls `get_ifaddr()` which uses `inet_select_addr()` to find the primary IP of the outgoing network interface.

3. Destination NAT (DNAT): The Routing Challenge

DNAT is the most complex because it happens in `PRE_ROUTING`, *before* the kernel makes its final routing decision.

3.1. Re-Routing Logic

If IPFire changes the destination IP of a packet, the kernel's original routing plan (based on the old IP) is now invalid. To fix this, IPFire must manually clear the kernel's destination cache.

```
// Force re-routing in ipfi_pre_process
if (daddr != iph->daddr) {
    dst_release(skb_dst(skb)); // Free old route
    skb_dst_set(skb, NULL);    // Tell kernel to re-route
}
```

3.2. Interaction with FORWARD Hook

When a packet's destination is changed to an internal server (e.g., Load Balancing), it no longer looks like a "Local In" packet.

1. **Kernel Decision:** After `PRE_ROUTING` and the `skb_dst_set(NULL)` call, the kernel re-runs the routing table.
2. **Path Change:** The packet is now destined for an internal network, so it enters the `FORWARD` hook.
3. **State Logic:** In the `FORWARD` hook, `check_state` misses (because it was just created/NATted), and the packet hits the FORWARDing permission rules.

3.3. Output DNAT and Filtering Order

When DNAT is applied in the `OUTPUT` direction (e.g., redirecting local web traffic to a transparent proxy cache), IPFire-Wall evaluates the packet against the filtering rules **before** the destination address is translated.

- **Why?:** This allows the administrator to still write `DROP` rules based on the original intended destination. If filtering happened *after* translation, the packet's destination would be the proxy, making it impossible to specifically drop traffic bound for a restricted external site without also blocking the proxy itself.

4. Checksum Corner Cases

4.1. UDP Zero Checksum

RFC 768 allows UDP checksums to be `0` (disabled). However, when we NAT a packet, most kernels require that if you change the address, you **MUST** provide a valid checksum or a specific "mangled zero" value if the original was zero.

```
// Handle UDP zero checksum
if (!pudphead->check) {
    pudphead->check = CSUM_MANGLED_0;
}
```

5. Summary: NAT Hook Matrix

NAT Type	Direction	Hook	Primary Objective
SNAT	Outgoing	POST_ROUTING	Hide internal IP
Masq	Outgoing	POST_ROUTING	Dynamic SNAT for WAN
DNAT	Incoming	PRE_ROUTING	Port Forwarding / LB
Re-NAT	Incoming	PRE_ROUTING	Restoring original IP

Developer API Guide: Internal Kernel Interfaces

This guide provides technical documentation for developers who wish to extend or integrate with the IPFire kernel module.

1. Centralized Data Structures (`common/`)

Shared structures are defined in `common/ipfi_structures.h` to ensure consistency between kernel and userspace.

1.1. `ipfire_info_t`

The primary metadata carrier for packets in the filtering hot path.

```
typedef struct {
    __u32 saddr, daddr;    // Source and Destination IPs
    __u16 sport, dport;    // Source and Destination Ports
    __u8 proto;            // Protocol (TCP/UDP/ICMP)
    __u8 direction;        // IPFI_INPUT, IPFI_OUTPUT, IPFI_FWD
}
```

```

    __u32 rule_id;           // Hash of the matching rule
    // ... statistics fields ...
} ipfire_info_t;

```

1.2. Unified Entry API (`ipfi_entry.h`)

All stateful entries use a standardized lifecycle API.

- `ipfi_entry_init(h, list_head, counter, list_lock)` : Initializes the entry, sets up the timer, and provides RCU-safe list insertion.
- `ipfi_entry_arm_timer(h)` : Activates the entry's timer. Should be called **after** the entry is safely on a list.
- `ipfi_entry_put(h)` : Standard reference decrement. If the count hits zero, the entry is queued for cleanup.
- `ipfi_entry_update_timer(h, proto, state)` : Refreshes the entry's lifetime based on the protocol state.
- `ipfi_entry_remove(h)` : Logically removes the entry from its table.

2. Kernel API: Core Logic Functions

2.1. `ipfi_response()`

Location: `kernel/ipfi_machine.c` **Purpose:** Main entry point for filtering decisions. **Arguments:**

- `struct sk_buff *skb` : The kernel packet buffer.
- `const ipfi_flow *flow` : Metadata about the packet's path.

2.2. `check_state()`

Location: `kernel/ipfi_state_machine.c` **Purpose:** O(1) state lookup. **Returns:** `struct response` (verdict and state metadata).

2.3. `keep_state()`

Location: `kernel/ipfi_state_machine.c` **Purpose:** Initializes a new entry in the bidirectional hash table.

3. Communication Protocol: Netlink

IPFire uses a custom Netlink protocol to communicate with the `ipfire` userspace utility.

3.1. Application-to-Kernel Interfaces

The `/proc` Interface (`/proc/IPFIRE/policy`)

For rapid default policy adjustments without restarting or using Netlink, IPFire-Wall exposes a `/proc` entry at `/proc/IPFIRE/policy`.

- **Reading:** `cat /proc/IPFIRE/policy` returns the current default verdict (`accept` or `denial`) applied to packets that do not match any explicit rule.
- **Writing:** `echo "accept" > /proc/IPFIRE/policy` or `echo "denial" > /proc/IPFIRE/policy` instantly updates the default filtering fallback policy. This is automatically cleaned up when the kernel module is safely unloaded.

Transparent Proxying via `getsockopt()`

To support transparent proxying (such as redirecting POP3 traffic to a local virus scanner like `p3scan`), IPFire-Wall registers a custom socket option. When traffic is DNAT 'ed locally, the listening proxy application needs to know the *original* destination IP address and port to complete the connection to the final server.

- **Constant:** `SO_IPFI_GETORIG_DST` (200)
- **Usage:** The userspace proxy calls `getsockopt(fd, IPPROTO_IP, SO_IPFI_GETORIG_DST, &orig_addr, &len)` .
- **Kernel Behavior:** The NAT engine looks up the associated translation entry and copies the pre-DNAT destination address back to the userspace application buffer.

Netlink Sockets

When advanced control is required, the `ipfi_netl.c` driver provides socket-based communication. It allocates multiple message families:

3.2. Message Structure

Messages are sent using the `send_data_to_user()` function in `kernel/netlink/` .

Field	Size	Description
Header	16 bytes	Netlink standard header
Proto	1 byte	Protocol ID
Direction	1 byte	In/Out/Fwd indicator
IPs	8 bytes	Source and Dest addresses
Ports	4 bytes	Source and Dest ports
Verdict	1 byte	ACCEPT/DROP

3.2. Reliability Tracking

If `netlink_unicast()` returns a negative value, the kernel increments the `total_lost` counter in the per-CPU statistics. This allows the administrator to detect if the logging daemon is falling behind.

4. Statistics Management

4.1. Reading Per-CPU Counters

To aggregate statistics for userspace, use the following pattern:

```
struct ipfi_stats total = {0};
int cpu;
for_each_possible_cpu(cpu) {
    struct ipfi_counters *c = per_cpu_ptr(kernel_stats, cpu);
    total.accepted += c->accepted;
    total.dropped += c->dropped;
}
```

```
// ... sum other fields ...  
}
```

5. Adding a New Helper

To add support for a new protocol (like FTP), developers should:

1. Implement a parser in `kernel/helpers/` .
 2. Register the helper in the `KEEP_STATE` logic within `ipfi_state_machine.c` .
 3. Ensure the helper correctly identifies the "adoption" phase versus the "data" phase.
-

Chapter 10: Lifecycle & Synchronization

This chapter describes how IPFire-Wall manages memory and concurrency for its internal tables using the unified `ipfi_entry` framework.

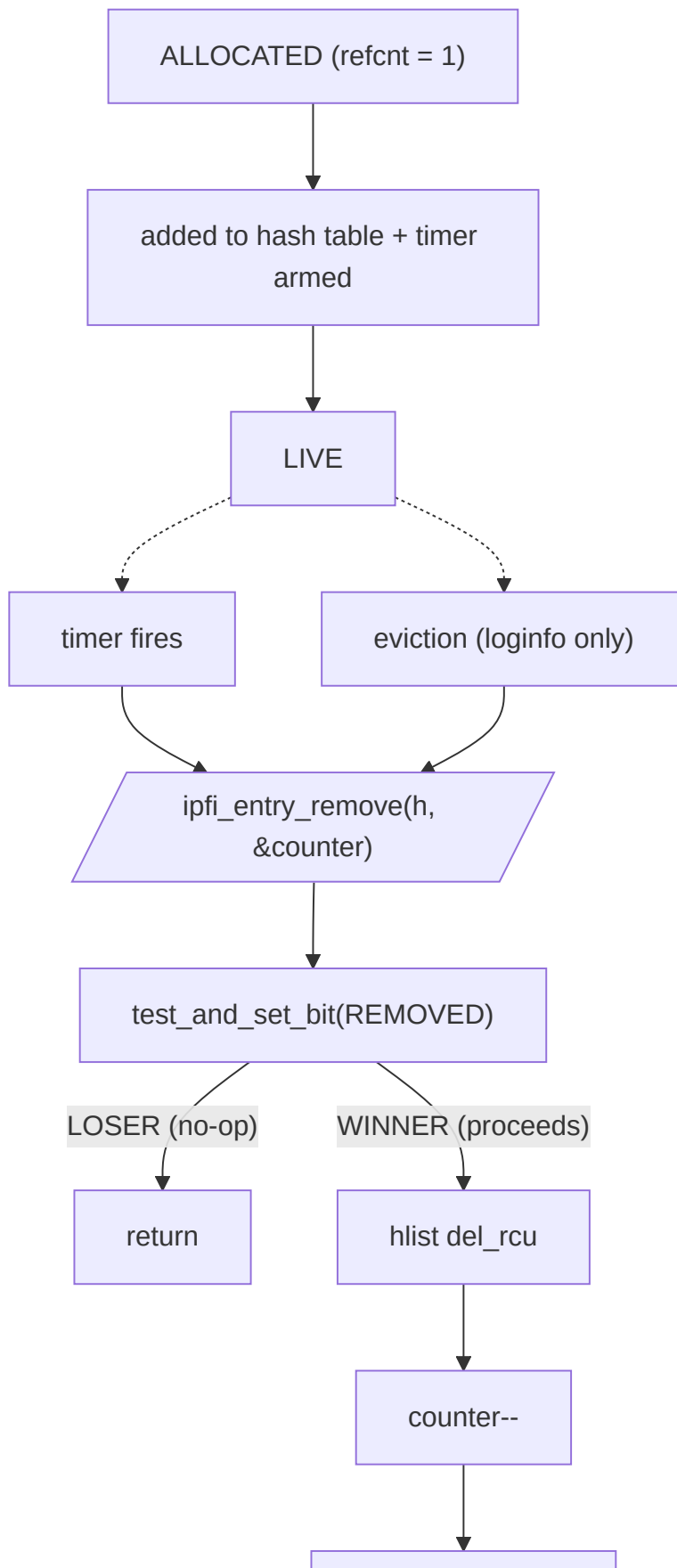
10.1. The Unified Entry Header

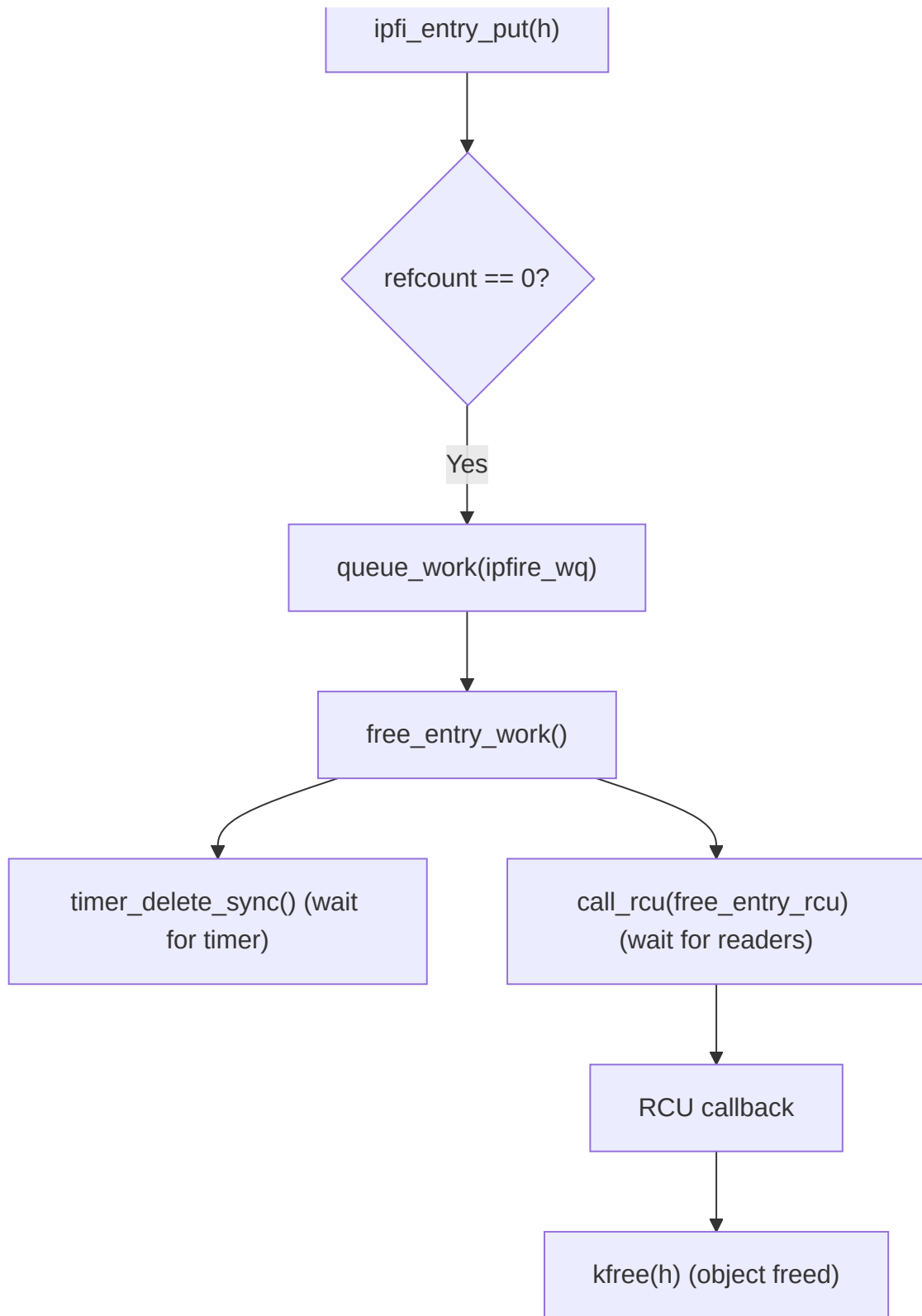
All dynamic table entries (State, NAT, LogInfo) embed `ipfi_entry_head` as their **first** member. This layout is mandated: `container_of` relies on the head being at offset 0 so that `kfree(h)` frees the whole object.

```
struct ipfi_entry_head {  
    struct timer_list    timer;           /* per-entry expiry timer */  
    struct work_struct    cleanup_work;   /* deferred-delete work item */  
    struct rcu_head      rcuh;           /* RCU callback node */  
    struct list_head      lnode;         /* list linkage (NAT; loginfo LRU) */  
    struct hlist_node     hnode;         /* primary hash linkage */  
    refcount_t            refcnt;         /* reference counter */  
    unsigned long         status;         /* IPFI_ENTRY_REMOVED bit */  
    unsigned long         last_timer_update; /* throttle timestamp */  
};
```

10.2. Entry Lifecycle — Full Chain

Every entry starts with `refcount = 1` (the **container reference**).





Key invariants

Invariant	Enforcement
-----------	-------------

REMOVED set exactly once	test_and_set_bit atomicity
ipfi_entry_put called exactly once	only the bit winner calls it
kfree happens after timer is fully dead	timer_delete_sync before call_rcu
kfree happens after all RCU readers are done	call_rcu grace period
queue_work is safe under spin_lock_bh	queue_work uses spin_lock_irqsave, never sleeps

10.3. loginfo — The Dual-Link Special Case

`ipfire_loginfo` is the **only** entry type carrying two simultaneous links:

```
active_logi_list  ← lnode → ipfire_loginfo ← hnode → loginfo_hashtable
(LRU eviction, O(1) tail)          (dedup lookup, O(1))
```

This is necessary because:

- `active_logi_list` (lnode) enables O(1) LRU eviction: oldest entry is always at the tail, removed when the table reaches `max_loginfo_entries`. The kernel enforces a capacity bound of [64, 65536] to ensure both deduplication effectiveness and memory safety.
- `loginfo_hashtable` (hnode) enables O(1) duplicate detection lookups.

`ipfi_entry_remove` handles **only hnode**. The loginfo lnode must be managed separately, under `loginfo_list_lock`, by **both** the timer callback and the eviction path.

The evict-vs-timer race and its fix

Two paths can try to remove the same lnode from `active_logi_list`:

Path A (timer callback)	Path B (loginfo_evict_oldest)
<code>spin_lock_bh(lock)</code>	<code>spin_lock_bh(lock)</code> ← serialised by lock
<code>if (!REMOVED)</code>	<code>list_del_rcu(lnode)</code>
<code>list_del_rcu(lnode)</code>	<code>ipfi_entry_remove()</code> ← sets REMOVED, puts
<code>ipfi_entry_remove()</code>	<code>timer_delete()</code>
<code>spin_unlock_bh(lock)</code>	<code>spin_unlock_bh(lock)</code>

Because both paths hold `loginfo_list_lock`, they are **serialised**. The REMOVED pre-check in path A ensures that if eviction already ran, the timer callback skips `list_del_rcu` (which would otherwise write to `LIST_POISON` addresses → silent hard panic).

`timer_delete()` in path B deflects any **pending** timer fire. If the timer is already running (blocked on the lock), the REMOVED check will cause it to skip lnode removal and be a no-op in `ipfi_entry_remove`.

10.4. Safe Module Shutdown (7-Step Sequence)

Unloading a stateful kernel module requires draining all pending timers, work items, and RCU callbacks before freeing the slab caches.

Step 1	<code>we_are_exiting = true</code>	← no new allocations
Step 2	<code>unregister_hooks()</code>	← no new packets enter
Step 3	<code>synchronize_net()</code>	← wait for in-flight packets
Step 4	<code>fini_machine / fini_log / fini_translation</code>	← flush tables: <code>ipfi_entry_put</code> chains → <code>queue_work</code>
Step 5	<code>destroy_workqueue(ipfire_wq)</code>	← drain work items: each runs <code>timer_delete_sync + call_rcu</code>
Step 6	<code>rcu_barrier()</code>	← wait for all <code>call_rcu</code> callbacks (i.e., all <code>kfree</code> calls) to complete
Step 7	<code>kmem_cache_destroy()</code>	← safe: all objects are freed

```
> [!WARNING]
> The order of Step 2 and Step 3 is critical. If `synchronize_net()` is called
*before* unregistering hooks, new packets can still enter the system immediately after
the synchronization finishes, leading to potential use-after-free conditions during
`kmem_cache_destroy()`.
```

During shutdown or when bulk-flushing elements from a state or NAT table, it is necessary to clear all active entries without disrupting concurrent RCU readers (e.g., in-flight packets traversing the filter hooks).

The module uses `ipfi\_table\_flush\_hash()` to safely drain hash buckets. For the loginfo LRU list, it uses `ipfi\_table\_flush\_all()`. Both ensure that any concurrent lockless readers traversing a node will still be safely directed to the next valid element through the RCU grace period.

The Correct Flush Pattern (Hash)

```
```c
hash_for_each_possible_rcu(ht, h, hnode, key) {
 if (!test_and_set_bit(IPFI_ENTRY_REMOVED, &h->status)) {
 hlist_del_rcu(&h->hnode);
 ipfi_entry_put(h);
 }
}
```

By using `test_and_set_bit()`, we establish an atomic "winner" for the removal of the entry. If a timer callback is simultaneously trying to expire the same entry, only one will successfully set the bit and proceed to the deletion and `ipfi_entry_put()`. The loser safely skips the element.

*[!WARNING] If steps 5 and 6 were swapped, or if step 6 were omitted, `kfree` could fire **after** `kmem_cache_destroy` — an immediate kernel panic.*

---

## 10.5. Timer Update Path — TOCTOU Mitigation

`ipfi_entry_update_timer` must not call `mod_timer` on an entry that is being freed. The function is **self-contained**: it takes its own `ipfi_entry_hold_rcu` before touching the timer, so callers do not need an extra hold. Flow:

```
ipfi_entry_update_timer(h, proto, state)
|
|— test_bit(REMOVED) → early exit if dying
|
|— if last update was > 5s ago:
 ipfi_entry_hold_rcu(h) ← atomic inc-not-zero
 test_bit(REMOVED) again ← re-check under hold
 mod_timer(...) ← safe: hold prevents free
 ipfi_entry_put(h) ← release temporary hold
```

The 5-second throttle prevents  $O(N_{\text{packets}})$  timer modifications under high load while keeping entries alive under sustained traffic.

---

# Chapter 11: Network Namespace Support

This chapter details the current implementation of Network Namespace (netns) support and its intended use cases.

## 11.1. Host-Centric Design

By default, IPFire-Wall is designed as a **Host-Based Firewall**.

- **Target:** Protect the host's primary network stack.
- **Scope:** Hooks are registered only in the initial namespace ( `init_net` ).
- **Performance:** Minimal overhead for systems using containers or namespaces that don't require firewalling.

## 11.2. The `per_net` Module Parameter

Administrators can control how the module interacts with namespaces via the `per_net` parameter:

Mode	Behavior
<code>per_net=0</code> (Default)	Hooks registered only in <code>init_net</code> . Traffic in other namespaces (including loopback inside containers) is invisible to IPFire.
<code>per_net=1</code>	Hooks are registered in <b>every</b> namespace (via <code>register_pernet_subsys</code> ).

## 11.3. Known Limitations (Current Version)

### Shared Global Tables

Even with `per_net=1`, the internal tables (State, NAT, and Logs) are **global and shared**.

- A state entry created in Namespace A can be matched by traffic in Namespace B if the IPs/Ports happen to collide.
- This is intentional for the current primary use case (local performance testing) but lacks strict isolation between tenants.

### The `MYADDR` Issue

The `MYADDR` keyword in rules (e.g., `ACCEPT src=MYADDR`) currently relies on a lookup function that is **hardcoded to search `init_net`**.

- **Symptom:** "IPFIRE: no interface matching name" messages in the kernel log.
- **Cause:** If a rule uses `MYADDR` in a non-init namespace, the lookup fails because that interface name does not exist in the initial namespace's device list.
- **Recommendation:** Avoid `MYADDR` in rules when using `per_net=1` for namespace-to-namespace traffic.

## 11.4. Primary Use Case: Local Performance Lab

The `per_net=1` mode is optimized for local benchmarking:

1. Create two namespaces connected via a `veth` pair.
2. Enable IPFire in all namespaces.



3. Flood packets between the namespaces using tools like `hping3` or `iperf` .
  4. Measure firewall overhead without involving physical network hardware or saturating the host's external NIC.
- 

## Chapter 12: Concurrency & Safety Analysis

This chapter documents the synchronization mechanisms used in IPFire-Wall to prevent race conditions, Use-After-Free (UAF), double-free, and Time-of-Check-to-Time-of-Use (TOCTOU) vulnerabilities, including the bugs discovered and fixed during high-load UDP flood testing.

---

### 12.1. The Three-Layer Safety Model

#### Layer 1 — RCU (lockless hot-path reads)

Packet processing lookups use `rcu_read_lock_bh()` to traverse lists and hash tables without taking a lock. RCU guarantees that linked objects remain pointer-valid for the duration of the read-side critical section, **even if another CPU unlinks them concurrently**.

```
rcu_read_lock_bh();
hash_for_each_possible_rcu(..., entry, hnode, key) {
 if (matches(entry)) {
 ipfi_entry_update_timer(&entry->h, ...); /* self-contained, safe */
 break;
 }
}
rcu_read_unlock_bh();
```

*[!IMPORTANT] RCU only guarantees **pointer validity** during the critical section, NOT **object liveness** beyond it. For operations that need the object to survive past the RCU lock, a **refcount hold** is required.*

#### Layer 2 — Reference counting (object liveness)

`refcount_t` ensures objects are freed only when **all** users have released them. `ipfi_entry_hold_rcu` uses `refcount_inc_not_zero` : it atomically increments the counter OR refuses if it is already at zero (dying object).

`ipfi_entry_update_timer` is self-contained: it acquires hold, updates the timer, and puts — callers do not need an external hold.

#### Layer 3 — Spinlocks (writer serialisation)

Writers (add, remove, evict) hold the table spinlock:

- prevents concurrent `hlist_del_rcu` / `list_del_rcu` on the same bucket
  - protects plain- unsigned `int` counters from data races
  - serialises the `loginfo lnode pre-check` + `list_del_rcu` sequence
- 

### 12.2. Lock Disciplines

Operation	Lock held	Locking function
Add state entry	state_list_lock	spin_lock_bh
Remove (timer) state	state_list_lock	spin_lock_bh
Add NAT entry	nat_locks[type]	spin_lock_bh
Remove (timer) NAT	nat_locks[type]	spin_lock_bh
Add loginfo entry	loginfo_list_lock	spin_lock_bh
Remove (timer) loginfo	loginfo_list_lock	spin_lock_bh
Evict loginfo (capacity)	loginfo_list_lock	spin_lock_bh
Lookup (state/NAT/log)	none (RCU)	rcu_read_lock_bh
Timer update	none (internal hold)	ipfi_entry_hold_rcu
Flush (module unload)	table spinlock	spin_lock_bh (brief)

`spin_lock_bh` disables BH (softirqs), preventing timer callbacks from running on the same CPU while a lock is held. This avoids the need for `spin_lock_irqsave` in most paths (timers run in softirq, not hard IRQ).

---

## 12.3. ipfi\_entry\_remove — Ownership Model

`ipfi_entry_remove` owns the final `ipfi_entry_put`. The **winner** of the `test_and_set_bit(REMOVED)` race performs unlink + put; the **loser** is a complete no-op.

```
void ipfi_entry_remove(struct ipfi_entry_head *h, unsigned int *counter) {
 if (test_and_set_bit(IPFI_ENTRY_REMOVED, &h->status))
 return; /* loser: no-op */

 hlist_del_rcu(&h->hnode); /* or list_del_rcu in list mode */
 (*counter)--;
 ipfi_entry_put(h); /* winner: sole caller of put */
}
```

Callers must **not** call `ipfi_entry_put` after `ipfi_entry_remove`. `queue_work` (inside `ipfi_entry_put`) is safe from BH-disabled spinlock context: it uses `spin_lock_irqsave` internally and never sleeps.

---

## 12.4. Post-Mortem: Silent Kernel Crash Under UDP Flood

### Symptom

`iperf3 -u -b 1G -P 120` crashed the machine **immediately** with no kernel oops, no kdump trace — a completely silent hard reset. Even a single stream crashed with high probability.

**Root cause: fault in softirq context before the oops path could execute.**

---

### Bug 1 — `handle_logininfo_timeout`: `list_del_rcu` on `LIST_POISON` (CRITICAL)

**File:** logging/log.c — handle\_logininfo\_timeout (hash mode)

CPU 0 (eviction, holds loginfo_list_lock)	CPU 1 (timer fires, waits for lock)
list_del_rcu(X.lnode)	[blocked on lock]
ipfi_entry_remove(X) ← sets REMOVED	
└ ipfi_entry_put → queue_work	
spin_unlock_bh()	
	spin_lock_bh() ← acquired
	list_del_rcu(X.lnode) ← X.lnode.prev
	=
LIST_POISON!	
	WRITE to 0x100 → page fault in softirq
	→ no oops printed → hard reset

**Fix:** Check `REMOVED` before `list_del_rcu` in the timer callback:

```
if (!test_bit(IPFI_ENTRY_REMOVED, &h->status))
 list_del_rcu(&h->lnode);
ipfi_entry_remove(h, &loginfo_entry_counter); /* winner puts; loser no-op */
```

## Bug 2 — `loginfo` evict oldest : no timer cancellation

**File:** logging/log.c — logininfo\_evict\_oldest

After eviction sets `REMOVED` and calls `ipfi_entry_put` (refcount  $\rightarrow$  0, work queued), the timer can still fire on another CPU before the workqueue runs `timer_delete_sync`. The timer callback then acquires the lock and reaches `list_del_rcu` with a poisoned lnode.

**Fix:** `timer_delete()` after `ipfi_entry_remove`. Non-sync is safe: if the timer is running it's blocked on the lock we hold; `REMOVED` is set so it becomes a no-op when it eventually acquires the lock.

```
ipfi_entry_remove(&oldest->h, &loginfo_entry_counter);
timer_delete(&oldest->h.timer); /* deflect pending timer fire */
```

### Bug 3 — `add_packet_to_infolist` : arm-timer after lock release (UAF)

**File:** logging/log.c — add\_packet\_to\_infolist

CPU 0	CPU 1
list_add_rcu(lnode)	
loginfo_entry_counter++	
spin_unlock_bh()	[eviction: list is at capacity]
	ipfi_entry_remove(new_entry) → put → work queued
	timer_delete_sync() → kfree(new_entry)
ipfi_entry_arm_timer()	← mod_timer on freed object! UAF!

**Fix:** Hold an extra refcount reference spanning the lock-release → arm-timer gap:

```
ipfi_entry_hold(&ipli->h); /* arm-guard: refcount → 2 */
spin_unlock_bh(&loginfo_list_lock);
ipfi_entry_arm_timer(&ipli->h); /* safe: refcount ≥ 2 */
ipfi_entry_put(&ipli->h); /* release arm-guard */
```

**Bug 4 — fini\_log (hash mode): Inode left dangling**

**File:** logging/log.c — fini\_log

ipfi\_table\_flush\_hash unlinks hnode only. In hash mode, loginfo Inodes remained in active\_logi\_list after flush → UAF on any subsequent traversal.

**Fix:** Always drain via active\_logi\_list (Inode-based flush works for both links because REMOVED prevents timer callbacks from re-entering):

```
ipfi_table_flush_all(&active_logi_list, &loginfo_list_lock,
 &loginfo_entry_counter);
```

**12.5. Final UAF / TOCTOU / Double-Free Audit**

**State table**

Path	Lock	Race risk	Status
check_state timer update	RCU	update_timer is self-contained (internal hold)	✓ safe
handle_keep_state_timeout remove	state_list_lock	single removal path, no eviction	✓ safe
lookup_state_table_n_update_timer	RCU + state_hold_rcu	redundant outer hold, harmless	✓ safe
add_state_table_to_list	state_list_lock	we_are_exiting double-check inside lock	✓ safe

**NAT table**

Path	Lock	Race risk	Status
lookup_nat_forward timer update	RCU + nat_hold_rcu	correct hold before update_timer	✓ safe
handle_nat_entry_timeout remove	nat_locks[type]	single removal path	✓ safe

**LogInfo**

Path	Lock	Race risk	Status
------	------	-----------	--------

handle_logininfo_timeout Inode	logininfo_list_lock	REMOVED pre-check added	✓ fixed
logininfo_evict_oldest timer	logininfo_list_lock	timer_delete() added	✓ fixed
add_packet_to_infolist arm	—	extra hold spanning gap	✓ fixed
fini_log Inode flush	logininfo_list_lock	Inode-based flush	✓ fixed
packet_not_seen (hash) update	RCU	update_timer self-contained	✓ safe
packet_not_seen (list) update	RCU + ipfi_entry_hold_rcu	outer hold redundant but safe	✓ safe

## Module unload

Step	Safety concern	Status
fini_machine → free_state_tables	entries in-flight in timers	REMOVED set, timer callback no-ops
destroy_workqueue	work items post call_rcu	destroy_workqueue drains all
rcu_barrier	kfree callbacks in flight	barrier waits for all
kmem_cache_destroy after rcu_barrier	use-after-destroy	ordering guarantees objects freed

## 12.6. Debugging Silent Panics

A fault in softirq context ( `list_del_rcu` → page fault at LIST\_POISON) crashes the machine before the oops message can be serialised to any output. To capture such events:

```
Serial console (capture before framebuffer flush):
GRUB_CMDLINE_LINUX="... console=ttyS0,115200 earlyprintk=serial,ttyS0,115200"

Verify kdump crash kernel is loaded:
cat /sys/kernel/kexec_crash_loaded # must be 1

Force panic on oops + immediate reboot (trips kdump):
echo 1 > /proc/sys/kernel/panic_on_oops
echo 1 > /proc/sys/kernel/panic

Compile with KASAN for in-kernel UAF detection (requires source rebuild):
CONFIG_KASAN=y
CONFIG_KASAN_INLINE=y
```

With `crashkernel=256M` already set, `kdump` should produce a `vmcore` after the next crash. Analyse with:

```
crash /usr/lib/debug/boot/vmlinux-$(uname -r) /var/crash/*/vmcore
```

## Chapter 13: Privileges and Policies

IPFire-Wall is designed to allow both system administrators and standard, unprivileged users to filter traffic and personalize their network experience asynchronously. This chapter outlines the strict separation of capabilities between the Root user and normal users.

### 13.1. The Root Administrator (uid = 0)

The system administrator is ultimately responsible for the health, routing, and fundamental security of the machine. Only the root user inherently possesses the exact rights necessary to:

#### 1. Alter Firewall Options:

- Change the kernel-space log level ( `-kloglevel` ).
- Modify the default firewall policy ( `ACCEPT` or `DROP` ).
- Change the timeout values for state connections ( `-time` ).
- Modify the maximum allowed sizing of dynamic lists, such as the Network Address Translation tables or State tables ( `-max_nat_entries` , `-max_state_entries` ).

#### 2. Perform Network Address Translation (NAT):

- Defining `SNAT` , `DNAT` , or `MASQUERADE` rules is strictly forbidden for normal users, as it influences routing directly and poses severe security implications if exposed to unprivileged users.

#### 3. Control Resolvers:

- Resolve "Blacklisted Sites" into dynamic denial rules via the daemon's background DNS resolver thread.

### The `-user` Option

The administrator explicitly grants the capability for normal users to connect their own client instances to the active firewall by starting the main daemon with the `-user` flag (e.g., `ipfire -daemon -user` ). If `-nouser` is provided, all subsequent unprivileged user requests to the kernel are rejected with `RULE_NOT_ADDED_NO_PERM` .

### 13.2. Unprivileged Users (uid > 0)

When `-user` is permitted, unprivileged users can start their own `ipfire` client.

#### Permitted Operations

- **Personalized Filtering:** Users can define their own Denial (drop) and Permission (accept) rules, which are saved in their home directory `~/ .IPFIRE/` .
- **Packet Monitoring:** Users can observe packet flows in real-time, receiving Netlink messages for packets that trigger `NOTIFY` rules.

#### Restricted Operations

- **System Alterations:** Unprivileged users cannot modify global tracking timers, max threshold limits, or initiate NAT translation rules.
- **Flushing Behavior:** When a normal user instructs the firewall to flush rules (e.g., selecting "Flush all rules" or shutting down their interface), the kernel iterates over the lists and *only* deletes rules where the `owner` matches the user's `uid`. The administrator's rules ( `uid = 0` ) remain entirely unaffected.

### 13.3. Rule Priority and Chronological Loading

While there is a clear distinction in capabilities, the kernel's packet filtering engine itself does not enforce a rigid hierarchical sorting mechanism (i.e., Root rules are not inherently prioritized simply because they belong to root).

Instead, prioritization is established through chronological list insertion:

1. Rules are evaluated sequentially using `list_for_each_entry_rcu`.
2. New rules sent via Netlink are always appended to the tail of the lists ( `list_add_tail_rcu` ).
3. Since the system `ipfire` service initiates on boot as Root loading the `/etc/ipfire/` files, the Root rules are inserted first.
4. When a user subsequently logs in and loads their rules via `~/ .IPFIRE/`, their rules are organically appended *after* the Root rules.

This ensures that the administrator's broad security sweeps (evaluated first) will catch packets before they can ever reach a user's local override rule.

---

## Chapter 14: Port Scanning Test Analysis

IPFire-Wall's stateful inspection and default DROP policies have been rigorously tested against industry-standard network scanners like **NMAP**. This chapter outlines how the kernel filtering engine responds to various stealth scanning techniques.

### 14.1. Stealth TCP Scans

Because IPFire-Wall automatically drops unapproved traffic, the firewall achieves high stealth by refusing to send `RST` (Reset) or `ICMP Unreachable` packets back to the scanner. This typically results in NMAP classifying ports as `filtered` rather than `closed`, obscuring the network topology.

#### SYN Scanning

In a standard SYN scan, NMAP sends a `SYN` packet.

- **Expected Open:** `SYN/ACK`
- **Expected Closed:** `RST`
- **IPFire-Wall Response:** If no explicit rule or state exists, IPFire-Wall drops the packet silently without sending a `RST`. This causes NMAP to eventually time out and classify the port as `filtered` rather than `closed`.

#### ACK Scanning

ACK scanning is used to determine if a firewall is stateful or simply a rudimentary SYN-blocker. It sends an `ACK` packet simulating an established connection.

- **IPFire-Wall Response:** Because IPFire-Wall tracks connections statefully, an unexpected `ACK` packet without a preceding `SYN_SENT` / `SYN_RECV` state is immediately identified as invalid by the `check_state()` engine. The packet is dropped silently, yielding a `filtered` result in NMAP.

## 14.2. Advanced Flag Manipulation Scans

Attackers often manipulate TCP flags to bypass poorly configured firewalls that only examine the `SYN` flag. IPFire-Wall's protocol tracking ensures these are caught.

### FIN Scanning

A naked `FIN` packet is sent to the target. In older RFC implementations, closed ports reply with `RST`, while open ports ignore it.

- **How IPFire-Wall Blocks It:** A user might wonder why legitimate closing connections (which use `FIN` flags) work, but bare `FIN` scans fail. Stateful connections are verified *before* the rule list is checked. A valid `FIN` belongs to an `ESTABLISHED` state. A bare `FIN` scan has no state table entry, falls through to the rule definitions (which default to `DROP`), and is silently discarded.

### NULL and XMAS Tree Scanning

- **NULL Scan:** All TCP flags (`SYN`, `ACK`, `FIN`, `RST`, `URG`, `PSH`) are turned off.
- **XMAS Scan:** The `FIN`, `PSH`, and `URG` flags are simultaneously turned on.
- **IPFire-Wall Response:** Similar to the `FIN` scan, these packets have no corresponding state in the tracking table. Because they do not match the strict state transition mapping of a legitimate flow, they are dropped. NMAP receives no response and classifies them as `open|filtered`.

## 14.3. Connectionless Scans

### UDP Scanning

UDP scanning attempts to find open UDP services by sending a 0-byte UDP packet.

- **Expected Closed:** `ICMP Port Unreachable`.
- **IPFire-Wall Response:** Without an explicit `ALLOW` rule, the UDP packet is dropped. IPFire-Wall does *not* generate an `ICMP Port Unreachable` response, causing NMAP to mark the port as `open|filtered`.

### IP Protocol Scanning

This scan sends raw IP packets without a transport layer header (TCP/UDP).

- **Expected Closed:** `ICMP Protocol Unreachable`.
- **IPFire-Wall Response:** The unsupported protocol falls through the parsing engine to the default rejection policy. It is dropped silently, resulting in an `open|filtered` scan result.

## Chapter 15: Sample Network Scenarios

This chapter aggregates practical configuration scenarios for IPFire-Wall, focusing specifically on Network Address Translation (NAT) and redirection flows. These examples illustrate the flexibility of the DNAT and SNAT engines.



## 15.1. Port Redirection (Local Host)

**Goal:** Redirect traffic from an external standard port (e.g., HTTP 80) to a different local service port (e.g., 8080).

- **Hook Used:** `PRE_ROUTING` (DNAT)
- **Mechanism:** The incoming packet's destination port is swapped before routing. The local IP stack then perceives the packet as destined for port 8080 and delivers it to the application listening there.

## 15.2. Transparent Proxying (Virus Scanning)

**Goal:** Redirect outbound user POP3 mail traffic (port 110) transparently to a local virus scanning proxy ( `p3scan` on port 8110) before it reaches the Internet.

- **Hook Used:** `OUTPUT` (DNAT)
- **Mechanism:** Normal users initiate a connection to an external email server. Due to an Output DNAT rule, the packet's destination is changed to the local proxy (127.0.0.1:8110).
- **API Extraction:** The proxy intercepts the packet and uses the `getsockopt` API ( `SO_IPFI_GETORIG_DST` ) to query IPFire-Wall for the original external server IP. The proxy retrieves the emails safely, scans them, and hands them to the email client.

## 15.3. External SSH Server Forwarding

**Goal:** A client connects to the IPFire-Wall router requesting SSH. The firewall redirects the connection to an internal SSH server (Host Y).

- **Hook Used:** `PRE_ROUTING` (DNAT) -> `FORWARD` -> `POST_ROUTING` (SNAT/Masquerade)
- **Mechanism:**
  - In `PRE_ROUTING` , the firewall rewrites the packet's destination IP to the internal Server Y.
  - The routing table forwards the packet.
  - In `POST_ROUTING` , if the internal server Y doesn't have a default route back through the firewall, a Source NAT (Masquerade) is applied to ensure the replies return through the firewall before proceeding to the original client.

## 15.4. DNS Interception

**Goal:** Prevent users from bypassing parental controls or organizational policies by hardcoding an external DNS server (e.g., 8.8.8.8).

- **Hook Used:** `PRE_ROUTING` / `OUTPUT`
- **Mechanism:** A destination NAT rule intercepts all UDP traffic destined for port 53. Regardless of the intended external server, the address is rewritten to point to the local network's strictly-managed DNS caching server.

---

# Chapter 16: ICMP Error NAT Translation & Table Architecture

This document covers two major advanced topics in the IPFire-Wall engine:

1. The translation logic for ICMP Error payloads associated with mapped connections (DNAT & SNAT).

2. The core data structures and architectural design of the NAT and State tracking tables, specifically focusing on `ipfi_entry_head` and hash lookups.

## Part 1: ICMP Error NAT Translation

When a router or server drops a packet or encounters a network error, it replies with an ICMP Error message (e.g., Destination Unreachable, Time Exceeded). The payload of this ICMP message contains a copy of the original IP header and the first 8 bytes of the transport layer header (the "inner header") that caused the error.

If the original packet underwent Network Address Translation (NAT) before generating the error, the ICMP error packet returning to the client must also be reverse-translated. The firewall must rewrite both the **Outer ICMP/IP Header** (to route the error to the correct recipient) and the **Inner Packaged IP Header** (so the client recognizes the error as belonging to its original connection).

### IP Terminology

We define the following standard terminology for actors in the NAT traversal:

- **A** = Client (Original Source of the traffic)
- **B** = Firewall Public IP (External-facing IP address)
- **B'** = Firewall Masquerade / Local IP (Internal-facing IP address of the firewall)
- **C** = Server (Original Destination for SNAT, or the Internal DNAT target for DNAT)

### Case 1: DNAT Forward Direction Error (Client A to Server C)

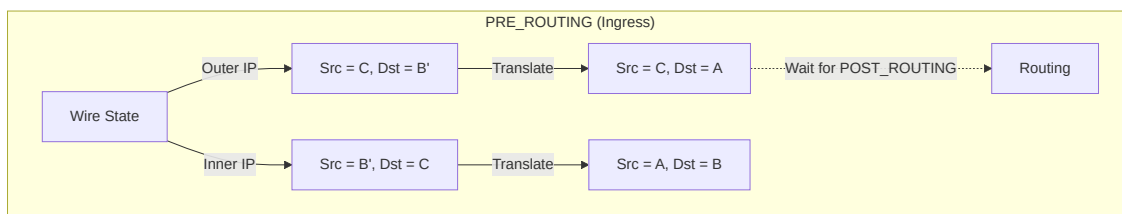
**Scenario:** Client A sends a packet to the Firewall's Public IP (B). The Firewall DNATs the packet and forwards it to the Internal Server (C). The Server (C) rejects the packet (e.g., UDP Port Unreachable) and generates an ICMP error sent back to the Firewall (B' or A directly, depending on if Masquerade was also applied).

- The originally sent Inner Packet was: `Src = A (or B')`, `Dst = C`
- The generated ICMP Error on the wire (arriving at the firewall) is:
  - Outer IP: `Src = C`, `Dst = B' (or A)`
  - Inner IP (the original packet): `Src = B' (or A)`, `Dst = C`

### Translation Flow

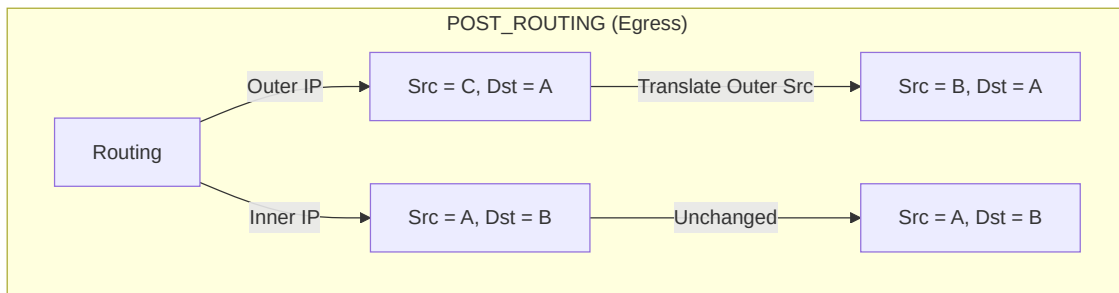
This translation is split across `PRE_ROUTING` and `POST_ROUTING`.

**1. PRE\_ROUTING Hook:** We must rewrite the **Outer Dst** and **all Inner Headers**. We DO NOT rewrite the Outer Src yet to prevent the packet from being dropped by the kernel as a "martian source" (a local IP arriving from an external interface).



**2. POST\_ROUTING Hook:** Because the inner headers are already fully de-NATed to their original state ( `Src = A`, `Dst = B` ), the `POST_ROUTING` NAT lookup recognizes this packet as matching the DNAT entry. We now

execute the final fixup: rewriting the **Outer Src** so Client A sees the error coming from the IP it was talking to (B).



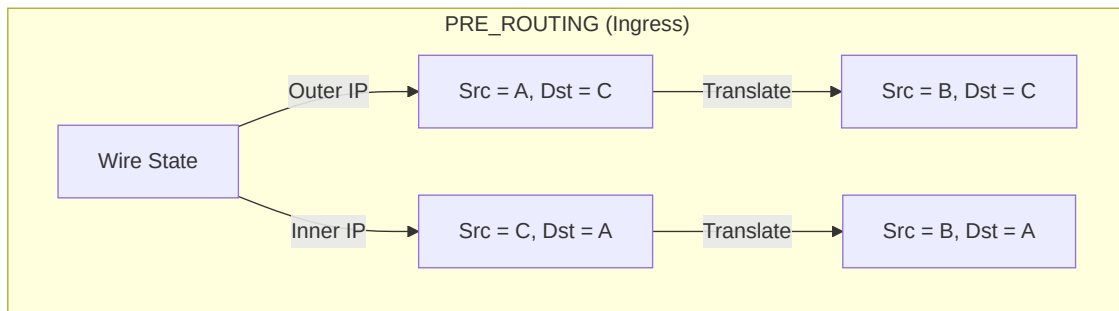
## Case 2: DNAT Reverse Direction Error (Server C back to Client A)

**Scenario:** Server C sends a reply packet back to Client A. The firewall (B') or Client A itself rejects the packet and generates an ICMP error going *towards* Server C.

- The originally sent Inner Packet was: `Src = C, Dst = A (or B')`
- The generated ICMP Error on the wire (arriving at the firewall) is:
  - Outer IP: `Src = A (or B'), Dst = C`
  - Inner IP (the original packet): `Src = C, Dst = A (or B')`

### Translation Flow

This is handled entirely in `PRE_ROUTING`. Only the Source IPs need to be un-DNATed to match the state before the original packet left the firewall.



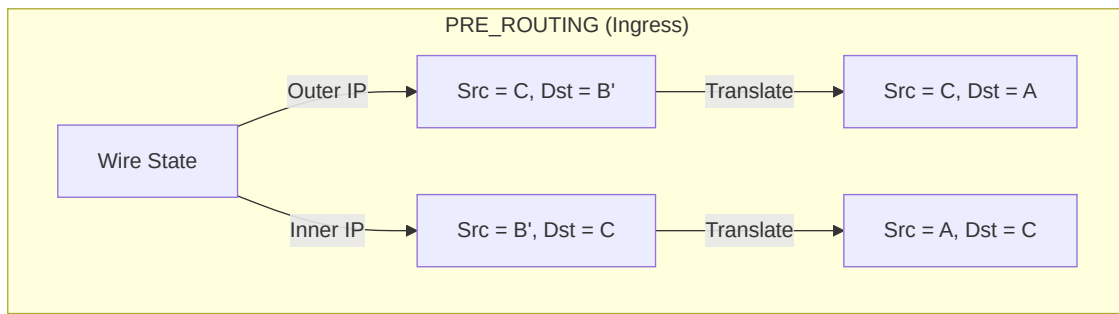
## Case 3: SNAT / Masquerade Forward Direction Error

**Scenario:** Client A sends a packet to Remote Server C. The Firewall SNATs it to its own Public IP (B'). The Server (C) rejects the packet and generates an ICMP error sent back to the Firewall (B').

- The originally sent Inner Packet was: `Src = B', Dst = C`
- The generated ICMP Error on the wire (arriving at the firewall) is:
  - Outer IP: `Src = C, Dst = B'`
  - Inner IP (the original packet): `Src = B', Dst = C`

### Translation Flow

Handled entirely in `PRE_ROUTING`. We rewrite the Outer Dst and Inner Src back to the original client IP (A).



## Appendix: Source Code

This appendix contains all the kernel module source code logically ordered.

### module\_init.h

```
#ifndef MODULE_INIT_H
#define MODULE_INIT_H

#endif // MODULE_INIT_H
```

### module\_init.c

### ipfire.h

```
#ifndef IPFI_H
#define IPFI_H

/* See ipfi.c for details and
 * use of this software.
 * (C) 2005 Giacomo S.
 */

// #include <linux/config.h>
#include <common/ipfi_structures.h>
#include <linux/inetdevice.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/netfilter.h> /* for hook registering */
#include <linux/netfilter_ipv4.h> /* for hook registering */
#include <linux/rcupdate.h>
#include <linux/version.h>
```

```

#include <net/tcp.h>
#include <net/udp.h>

extern pid_t userspace_control_pid;
extern pid_t userspace_data_pid;
extern uid_t userspace_uid;
extern struct sock *sknl_ipfi_control;
extern struct sock *sknl_ipfi_data;
extern struct sock *sknl_ipfi_gui_notifier;

#include "filter/header_check.h"

extern struct nf_hook_ops nfh_pre, nfh_in, nfh_out, nfh_fwd, nfh_post,
 nfh_defrag_pre, nfh_defrag_out;

#define NOLOCK 0
#define ACQUIRE_LOCK 1

#define IPFIRE_DEFAULT_POLICY IPFI_DROP /* denial */

#define NETLINK_IPFI_DATA MAX_LINKS - 3
#define NETLINK_IPFI_CONTROL MAX_LINKS - 2
#define NETLINK_IPFI_GUI_NOTIFIER MAX_LINKS - 1

/* network printk() should be guarded by net_ratelimit() function -- see
 * net/core/utils.c */
#define IPFI_PRINTK(args...) \
do { \
 if (net_ratelimit()) \
 printk(args); \
} while (0)

#define MODERATE_LIMIT 32
#define MAXMODERATE_ARGS 16

/* this macro avoids filling kernel log with the same message hundred of times.
 * There are MAXMODERATE_ARGS different messages that can be registered by means
 * of the enumeration moderate_print_sections below. The associated message is
 * printed by printk only every MODERATE_LIMIT times.
 * The variable used by this macro must be globally declared in the file/module
 * that uses it and its name must be `moderate_print'. It must be an unsigned int
 * array of dimension MAXMODERATE_ARGS. An example is represented in
 * ipfi_machine, where `moderate_print' is declared at the beginning of the file
 * and initialized to 0 values in the initialization method `init_translation()'
 * via the memset function.
 */
#define IPFI_MODERATE_PRINTK(id, args...) \
do { \
 if (id < MAXMODERATE_ARGS) { \
 if (moderate_print[id] % moderate_print_limit[id] == 0) { \
 moderate_print[id]++; \
 IPFI_PRINTK(args); \
 IPFI_PRINTK("[message repeated %u times. Next repetition in %u " \
 "events of the same type]\n", \
 moderate_print[id], moderate_print_limit[id]); \
 } else { \
 moderate_print[id]++; \
 } \
 } \
} while (0)

```

```

enum moderate_print_sections {
 PRINT_PROTO_UNSUPPORTED = 0,
 PRINT_SKB_ALLOC_FAILED,
 PRINT_TEST,
 /* register here other custom sections */

 END_MODERATE_LIST = MAXMODERATE_ARGS - 1,
};

enum checksum_errors {
 BAD_IP_CSUM = 1,
 BAD_TCPHEAD_CSUM,
 BAD_TCPHEAD_CHECK,
 BAD_UDPHEAD_CHECK,
 BAD_UDPHEAD_CSUM
};

#define BAD_CHECKSUM INT_MIN

/* constant for updating statistic counters */
#define IPFI_FAILED_NETLINK_USPACE 10

/* every PRINTK_WARN_IPFI_DISABLED packets a warning is logged
 * telling IPFI is disabled */
#define PRINTK_WARN_IPFI_DISABLED 20

// /* linked list containing rules */
// struct rule_list {
// ipfire_rule rule;
// struct list_head list;
//
// struct rcu_head rulelist_rcuh;
// };

/* loguser:
 if loguser >= 2 implicit denial is sent back;
 if loguser >= 4 implicit and explicit denial;
 if loguser >= 5 all filtering is sent back;
 if loguser >= 6 filtering and pre/post stuff is sent.
 commands in do_control are always sent back: loguser
 then decides to print or not information. To have information
 printed on commands received _at_startup_ loguser
 must be 7.
*/
/* loglevel
 if loglevel > 3 printk of rule added in manage_rule()
 (ipfi_netl.c);
 if loglevel > 2 printk of command received in do_control
 (ipfi_netl.c);
*/
struct ipfire_options {
 u8 nat : 1, masquerade : 1, state : 1, all_stateful : 1, free : 4;
 u8 user_allowed : 1, noflush_on_exit : 1,
 loglevel : 3, /* logging level by means of printk */
 loguser : 3; /* sending information to userspace */
 unsigned long int snatted_lifetime;
 unsigned long int dnatted_lifetime;
 unsigned long int state_lifetime;
 unsigned long int setup_shutd_state_lifetime;
 unsigned long int loginfo_lifetime;

```

```

 unsigned long int max_loginfo_entries;
 unsigned long int max_nat_entries;
 unsigned long int max_state_entries;
};

/* functions to register with netfilter hooks */
int register_hooks(void);

int check_headers(struct sk_buff *skb);
unsigned int process(void *priv, struct sk_buff *skb,
 const struct nf_hook_state *state);

/* ipfire functions */

struct response iph_in_get_response(struct sk_buff *skb, ipfi_flow *flow,
 struct info_flags *flags);

/* if rulename in src is specified, copy it to dest rulename */
inline void copy_rulename(ipfire_info_t *iit_dest,
 const ipfire_info_t *iit_src);

/* if *cnt reaches ULONG MAX, it must be reset to 0 */
inline void check_packet_num(unsigned long long *cnt);

int ipfi_response(const struct nf_hook_state *state, struct sk_buff *skb,
 ipfi_flow *_flow);

int ipfi_pre_process(struct sk_buff *skb, const ipfi_flow *flow);
int ipfi_post_process(struct sk_buff *skb, const ipfi_flow *flow);

int recalculate_ip_checksum(struct sk_buff *skb, int direction);

/* updates sent counter, sends packet to userspace and calls
 * update_kernel_stats()
 */
inline int send_packet_to_userspace_and_update_counters(
 const struct sk_buff *skb, const ipfi_flow *flow,
 const struct response *resp, const struct info_flags *flags);

/* prints just tcp checksum, for debug. To remove */
void print_check(struct sk_buff *skb);

/* Callback to free a rule removed from the linked list. */
void free_rule_rcu_call(struct rcu_head *head);

static inline void log_skb_tuple(const struct sk_buff *skb) {
 const struct iphdr *iph;
 __be32 saddr, daddr;
 if (!skb)
 return;
 iph = ip_hdr(skb);
 if (!iph)
 return;

 saddr = iph->saddr;
 daddr = iph->daddr;

 /* TCP */
 if (iph->protocol == IPPROTO_TCP) {
 const struct tcphdr *tcph;

```

```

 if (!pskb_may_pull((struct sk_buff *)skb,
 ip_hdrlen(skb) + sizeof(struct tcphdr)))
 return;

 tcph = tcphdr(skb);

 pr_info("TCP %pI4:%u -> %pI4:%u\n", &saddr, ntohs(tcph->source), &daddr,
 ntohs(tcph->dest));
}
/* UDP */
else if (iph->protocol == IPPROTO_UDP) {
 const struct udphdr *udph;

 if (!pskb_may_pull((struct sk_buff *)skb,
 ip_hdrlen(skb) + sizeof(struct udphdr)))
 return;

 udph = udphdr(skb);

 pr_info("UDP %pI4:%u -> %pI4:%u\n", &saddr, ntohs(udph->source), &daddr,
 ntohs(udph->dest));
}
/* Other protocols */
else {
 pr_info("IP proto=%u %pI4 -> %pI4\n", iph->protocol, &saddr, &daddr);
}
}

#endif

```

## ipfire.c

```

/* ip firewall Giacomo S. */
#include "ipfire.h"
#include "build.h"
#include "filter/defrag.h"
#include "filter/state/state_machine.h"
#include "globals.h"
#include "helpers/icmp_nat.h"
#include "ipfi_machine.h"
#include "mangle/tcpmss.h"
#include "module_init.h"
#include "netlink/ipfi_netl.h"
#include "netlink/message_builder.h"
#include "proc/proc.h"
#include <linux/init.h>
#include <linux/list.h>
#include <linux/module.h>
#include <linux/netfilter.h>
#include <net/net_namespace.h>
#include <net/tcp.h>
#include <net/udp.h>

#define NF_IP_PRE_ROUTING NF_INET_PRE_ROUTING
#define NF_IP_POST_ROUTING NF_INET_POST_ROUTING
#define NF_IP_LOCAL_IN NF_INET_LOCAL_IN
#define NF_IP_LOCAL_OUT NF_INET_LOCAL_OUT
#define NF_IP_FORWARD NF_INET_FORWARD

```



```

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Giacomo S. <delleceste@gmail.com>");
MODULE_DESCRIPTION(
 "IPFIRE-wall: A high-performance, stateful IPv4 packet filter.\n"
 "Features:\n"
 " - Stateful Inspection: Advanced TCP/UDP/ICMP state tracking.\n"
 " - NAT: SNAT, DNAT, and Masquerading support.\n"
 " - Lifecycle Management: Unified ipfi_entry with RCU and per-entry "
 "timers.\n"
 " - Slab Allocation: Dedicated kmem_cache for state, NAT, and log "
 "entries.\n"
 " - Smart Logging: Rate-limited, duplicate-suppressed logging system.\n"
 " - Protocol Helpers: Optimized support for Passive FTP.\n"
 " - Networking: TCP MSS clamping and IPv4 defragmentation.\n"
 " - Userspace: Netlink-based control, ruleset sync, and statistics.\n"
 "Network Namespaces (netns):\n"
 " - per_net=0 (default): Hooks registered in init_net only.\n"
 " - per_net=1: Hooks in all namespaces; tables (state, NAT, logs) are "
 "shared.\n"
 " - Primary use: Local host firewalling and cross-netns performance "
 "testing.");

/* Versione di test!
 * (C) Giacomo Strangolino, 2005-2026
 * Filtro di pacchetti con funzionalita' di NAT.
 * Il software in kernel space consta di 6 moduli interdipendenti.
 * Per problemi potete scrivere a Giacomo S.
 * posta elettronica: delleceste@gmail.com
 * Software libero :)
 * Si legga la documentazione per l'uso e l'installazione.
 * I commenti al codice e la documentazione sono in lingua inglese
 * per facilitare la lettura del codice a chi ne avesse bisogno, dal
 * momento che io stesso sono stato aiutato nello svolgimento
 * del progetto da diverse persone sparse in tutto il mondo.
 * Grazie!
 * Giacomo.
 */

/*****
 * Copyright 2005-2026 Giacomo Strangolino.
 *
 * email:
 * delleceste@gmail.com
 *
 * Web Site: www.giacomos.it
 * IPFIRE-wall home: www.giacomos.it/ipfire
 *
 *****/

/*
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation; either version 2 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 */

```

```

/*
 * Thanks to:
 * Alessandro Rubini, Alexander Harsch, Andrea Barberio, Jozsef Kadlecsek, Diego
 * Billi, Luigi Rizzo, Paul 'Rusty' Russell, Andrew Gargan, Daniele orlandi,
 * Stephen Hemminger, Kasper Dupont, Peter T. Breuer, Tauno Voipio;
 */

/* Linux kernel 2.6.9 and above were used for test, together with
 * debian GNU/Linux - www.debian.org - and
 * slackware Linux - www.slackware.org.
 */

char *policy = "drop";

module_param(policy, charp, S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP);
MODULE_PARM_DESC(default_policy, "\"accept\" or \"drop\" policy as default.\n");

/*
 * per_net - Network namespace hook registration mode.
 *
 * 0 (default): Hooks are registered in init_net only. The firewall sees
 * traffic on the host's main network stack. Packets inside
 * other namespaces (even through loopback) are invisible
 * because each namespace has its own netfilter hook chain.
 *
 * 1: Hooks are registered in every network namespace via
 * register_pernet_subsys(). All namespaces share the same
 * state, NAT, and log tables (no per-namespace isolation).
 *
 * The per_net=1 mode is primarily intended for local performance testing:
 * flood traffic between namespaces on the same host to measure firewall
 * overhead without flooding the physical network.
 */
static int per_net = 0;
module_param(per_net, int, S_IRUGO);
MODULE_PARM_DESC(
 per_net,
 "Hook registration mode: 0=init_net only (default), 1=all namespaces.\n"
 "With per_net=0, traffic in non-init namespaces is not filtered.\n");

int welcome(void);

/* packet counters */
unsigned long long int in_counter = 0;
unsigned long long int out_counter = 0;
unsigned long long int fwd_counter = 0;
unsigned long long int pre_counter = 0;
unsigned long long int post_counter = 0;

time64_t module_load_time;

/* bad checksum counter for packets received */
unsigned badcsum_cnt_rcv = 0;

struct response iph_in_get_response(struct sk_buff *skb, ipfi_flow *flow,
 struct info_flags *flags) {
 struct response response = {};
 flags->direction = flow->direction;
 /* invoke engine function passing the appropriate rule lists */
 if (flow->direction == IPFI_INPUT)

```

```

 response = ipfire_filter(&in_drop, &in_acc, &fwopts, skb, flow, flags);
else if (flow->direction == IPFI_OUTPUT)
 response = ipfire_filter(&out_drop, &out_acc, &fwopts, skb, flow, flags);
else if (flow->direction == IPFI_FWD)
 response = ipfire_filter(&fwd_drop, &fwd_acc, &fwopts, skb, flow, flags);
else
 IPFI_PRINTK("IPFIRE: iph_in_get_response(): invalid direction!\n");
return response;
}

struct nf_hook_ops nfh_pre, nfh_in, nfh_out, nfh_fwd, nfh_post, nfh_defrag_pre,
 nfh_defrag_out;

static int register_ipfire_net(struct net *net);
static void unregister_ipfire_net(struct net *net);

static int ipfire_net_init(struct net *net) { return register_ipfire_net(net); }

static void ipfire_net_exit(struct net *net) { unregister_ipfire_net(net); }

static struct pernet_operations ipfire_net_ops = {
 .init = ipfire_net_init,
 .exit = ipfire_net_exit,
};

/*
 * Hook Registration
 *
 * nf_register_net_hook() is namespace-scoped: hooks only fire for packets
 * traversing the struct net they were registered in.
 *
 * per_net=0: register_ipfire_net(&init_net) - only the host's own traffic.
 * Other namespaces have their own hook chains and are unaffected.
 *
 * per_net=1: register_pernet_subsys() - hooks fire in every namespace.
 * Useful for local performance testing: create namespaces with
 * veth pairs and flood packets between them to measure firewall
 * overhead, without sending any traffic on the physical network.
 */

#ifdef _BUILD_VERSION
#define KERNEL_MODULE_VERSION _BUILD_VERSION
#else
#define KERNEL_MODULE_VERSION "2.0.0-alpha"
#endif

#ifdef _BUILD_CODENAME
#define KERNEL_MODULE_CODENAME _BUILD_CODENAME
#else
#define KERNEL_MODULE_CODENAME "lin"
#endif

#define BUILD_DATE _BUILD_DATE
#define BUILD_SYS _BUILD_SYS

int welcome(void) {
 struct timespec64 tv_load_time;
 IPFI_PRINTK("IPFIRE: MODULE INITIALIZED [%s] codename %s built on %s, %s - "
 "Giacomo S. "
 "<delleceste@gmail.com>\n",
 KERNEL_MODULE_VERSION, KERNEL_MODULE_CODENAME, BUILD_DATE,

```

```

 BUILD_SYS);

/* set loading time into kernel stats struct */
ktime_get_real_ts64(&tv_load_time);
module_load_time = tv_load_time.tv_sec;

/* Allocate per-CPU counters */
ipfi_counters = alloc_percpu(struct ipfi_counters);
if (!ipfi_counters) {
 IPFI_PRINTK("IPFIRE: failed to allocate per-CPU counters\n");
 return -ENOMEM;
}

/* Set default policy and write it in kernel stats */
init_options(&fwopts);
set_policy(policy);
init_kernel_stats(&kstats);
kstats.policy = default_policy;
kstats.kmod_load_time = module_load_time;
if (init_procentry(PROCENT, policy) < 0)
 return -1;
set_procentry_values();
userspace_control_pid = 0;
userspace_data_pid = 0;

/* Initialize ruleset lists before any potential notifier event */
init_ruleset_heads();

/* Allocate global workqueue for entry cleanup (before component init) */
ipfire_wq = alloc_workqueue("ipfire_wq", WQ_MEM_RECLAIM, 0);
if (!ipfire_wq) {
 IPFI_PRINTK("IPFIRE: failed to allocate workqueue\n");
 return -ENOMEM;
}

init_translation();
init_log();
if (init_netl() == 0) {
 init_machine(); /* registers netdevice notifier */
 if (per_net) {
 if (register_pernet_subsys(&ipfire_net_ops) < 0) {
 IPFI_PRINTK("IPFIRE: failed to register pernet subsystem\n");
 return -1;
 }
 } else {
 register_ipfire_net(&init_net);
 }
}

return 0;
}

static int __init ini(void) { return welcome(); }

static void __exit fini(void) {
 /*
 * ===== SHUTDOWN SEQUENCE =====
 *
 * The unified lifecycle (ipfi_entry) uses a deferred-free chain:
 *
 * flush_all / timer_cb
 */
}

```

```

* → ipfi_entry_put()
* → queue_work(ipfire_wq, cleanup_work) [step A: work queued]
* → free_entry_work()
* → timer_delete_sync()
* → call_rcu(free_entry_rcu) [step B: RCU cb queued]
* → free_entry_rcu()
* → kfree(h) [step C: actual free]
*
* To safely destroy kmem_cache, we must ensure step C has completed
* for ALL entries. This requires:
*
* 1. we_are_exiting = true → stop new allocations
* 2. synchronize_net() → finish in-flight packet hooks
* 3. unregister_hooks → no more packets
* 4. fini_machine / fini_log / → flush_all queues work items (step A)
* fini_translation
* 5. destroy_workqueue(ipfire_wq) → flushes & runs all work items,
* which post call_rcu (step B)
* 6. rcu_barrier() → waits for all call_rcu callbacks
* to finish (step C – kfree done)
* 7. kmem_cache_destroy(...) → now safe, all objects freed
*/

/* --- Step 1: signal exit --- */
WRITE_ONCE(we_are_exiting, true);

/* --- Step 2: unregister hooks ---
 * Stop new packets from entering the module's paths */
if (per_net) {
 unregister_pernet_subsys(&ipfire_net_ops);
} else {
 unregister_ipfire_net(&init_net);
}

/* --- Step 3: wait for in-flight RCU readers (packet hooks) ---
 * synchronize_net() calls synchronize_rcu() internally.
 * Note: synchronize_rcu() only waits for _readers_, NOT for
 * call_rcu() _callbacks_ – that's what rcu_barrier() is for.
 * We do this AFTER unregistering hooks so we know no new
 * packets are entering. */
synchronize_net();

/* --- Step 4: flush all tables ---
 * Each fini function calls ipfi_table_flush_hash() (or flush_all for LRU
 * lists), which:
 * - iterates buckets/lists, marks entries REMOVED
 * - calls ipfi_entry_put() → queues cleanup work on ipfire_wq */
fini_machine();
fini_log();
fini_translation();

/* fini_netl(): just calls sock_release on the netlink socket */
fini_netl();
clean_proc();

/* --- Step 5: flush the workqueue ---
 * destroy_workqueue() drains all pending work items.
 * This runs free_entry_work() for every entry, which calls
 * timer_delete_sync() + call_rcu(free_entry_rcu).
 * After this, all call_rcu callbacks are _posted_ but may
 * not have _executed_ yet. */

```

```

 if (ipfire_wq) {
 destroy_workqueue(ipfire_wq);
 ipfire_wq = NULL;
 }

 /* --- Step 6: wait for all RCU callbacks ---
 * rcu_barrier() blocks until every call_rcu(free_entry_rcu)
 * posted in step 5 has finished executing (i.e., kfree done).
 * After this returns, no slab objects are in use. */
 rcu_barrier();

 /* --- Step 7: destroy slab caches ---
 * All objects have been freed, safe to tear down the caches. */
 kmem_cache_destroy(state_cache);
 kmem_cache_destroy(nat_cache);
 kmem_cache_destroy(loginfo_cache);

 if (ipfi_counters)
 free_percpu(ipfi_counters);
}

void set_policy(const char *def_policy) {
 if (strncmp(def_policy, "accept", 6) == 0) {
 kstats.policy = default_policy = IPFI_ACCEPT;
 } else {
 kstats.policy = default_policy = IPFI_DROP;
 }
}

void init_options(struct ipfire_options *opts) {
 memset(opts, 0, sizeof(struct ipfire_options));
 /* by default, users are allowed to insert their rules.
 * Administrator, to avoid users' rules, have to start
 * firewall with option noflush and disabling users */
 opts->user_allowed = 1;
}

// enum nf_ip_hook_priorities {
// NF_IP_PRI_FIRST = INT_MIN,
// NF_IP_PRI_CONNTRACK_DEFRAG = -400,
// NF_IP_PRI_RAW = -300,
// NF_IP_PRI_SELINUX_FIRST = -225,
// NF_IP_PRI_CONNTRACK = -200,
// NF_IP_PRI_MANGLE = -150,
// NF_IP_PRI_NAT_DST = -100,
// NF_IP_PRI_FILTER = 0,
// NF_IP_PRI_SECURITY = 50,
// NF_IP_PRI_NAT_SRC = 100,
// NF_IP_PRI_SELINUX_LAST = 225,
// NF_IP_PRI_CONNTRACK_CONFIRM = INT_MAX,
// NF_IP_PRI_LAST = INT_MAX,
// };

/*
 * register_ipfire_net - Initialize and register netfilter hooks for a namespace
 *
 * This function sets up the hook structures (nf_hook_ops) that define our
 * packet processing callbacks, then registers them in the given namespace.
 *
 * Returns: 0 on success, negative error code on failure
 */

```

```

static int register_ipfire_net(struct net *net) {
 int ret;

 /* Setup hook structures (using global ops for simplicity if they can be
 * shared)
 * Note: In modern kernels, nf_hook_ops can be shared if they are immutable.
 */
 /* PRE_ROUTING: Defragmentation hook (highest priority) */
 nfh_defrag_pre.pf = PF_INET;
 nfh_defrag_pre.hook = ipfi_defrag;
 nfh_defrag_pre.hooknum = NF_INET_PRE_ROUTING;
 nfh_defrag_pre.priority = NF_IP_PRI_CONNTRACK_DEFRAG;

 /* pre routing */
 nfh_pre.pf = PF_INET;
 nfh_pre.hooknum = NF_IP_PRE_ROUTING;
 nfh_pre.priority = NF_IP_PRI_NAT_DST;
 nfh_pre.hook = process;

 /* input */
 nfh_in.pf = PF_INET;
 nfh_in.hooknum = NF_IP_LOCAL_IN;
 nfh_in.priority = NF_IP_PRI_FILTER;
 nfh_in.hook = process;

 /* forward */
 nfh_fwd.pf = PF_INET;
 nfh_fwd.hooknum = NF_IP_FORWARD;
 nfh_fwd.priority = NF_IP_PRI_FILTER;
 nfh_fwd.hook = process;

 /* output */
 nfh_out.pf = PF_INET;
 nfh_out.hooknum = NF_IP_LOCAL_OUT;
 nfh_out.priority = NF_IP_PRI_FILTER;
 nfh_out.hook = process;

 /* output - defrag */
 nfh_defrag_out.pf = PF_INET;
 nfh_defrag_out.hook = ipfi_defrag;
 nfh_defrag_out.hooknum = NF_INET_LOCAL_OUT;
 nfh_defrag_out.priority = NF_IP_PRI_CONNTRACK_DEFRAG;

 /* post routing */
 nfh_post.pf = PF_INET;
 nfh_post.hooknum = NF_IP_POST_ROUTING;
 nfh_post.priority = NF_IP_PRI_NAT_SRC;
 nfh_post.hook = process;

 /* Register hooks for the given net namespace */
 ret = nf_register_net_hook(net, &nfh_defrag_pre);
 if (ret < 0)
 goto err_defrag_pre;

 ret = nf_register_net_hook(net, &nfh_pre);
 if (ret < 0)
 goto err_pre;

 ret = nf_register_net_hook(net, &nfh_in);
 if (ret < 0)
 goto err_in;

```

```

ret = nf_register_net_hook(net, &nfh_fwd);
if (ret < 0)
 goto err_fwd;

ret = nf_register_net_hook(net, &nfh_out);
if (ret < 0)
 goto err_out;

ret = nf_register_net_hook(net, &nfh_defrag_out);
if (ret < 0)
 goto err_defrag_out;

ret = nf_register_net_hook(net, &nfh_post);
if (ret < 0)
 goto err_post;

return 0;

/* Error handling - unregister in reverse order */
err_post:
 nf_unregister_net_hook(net, &nfh_defrag_out);
err_defrag_out:
 nf_unregister_net_hook(net, &nfh_out);
err_out:
 nf_unregister_net_hook(net, &nfh_fwd);
err_fwd:
 nf_unregister_net_hook(net, &nfh_in);
err_in:
 nf_unregister_net_hook(net, &nfh_pre);
err_pre:
 nf_unregister_net_hook(net, &nfh_defrag_pre);
err_defrag_pre:
 return ret;
}

static void unregister_ipfire_net(struct net *net) {
 nf_unregister_net_hook(net, &nfh_post);
 nf_unregister_net_hook(net, &nfh_defrag_out);
 nf_unregister_net_hook(net, &nfh_out);
 nf_unregister_net_hook(net, &nfh_fwd);
 nf_unregister_net_hook(net, &nfh_in);
 nf_unregister_net_hook(net, &nfh_pre);
 nf_unregister_net_hook(net, &nfh_defrag_pre);
}

int check_headers(struct sk_buff *skb) {
 if (pskb_may_pull(skb, sizeof(struct iphdr))) {
 // pskb_may_pull may realloc skb: get a fresh ptr to iphdr
 struct iphdr *ih = ip_hdr(skb);
 unsigned int ihl;
 uint8_t protocol;

 if (ih->ihl < 5 || ih->version != 4) {
 IPFI_PRINTK(
 "ipfire_core: check_headers: invalid ip hdr (ihl %d, version %d)",
 ih->ihl, ih->version);
 return -1;
 }

 ihl = ih->ihl * 4;

```



```

protocol = ih->protocol;

switch (protocol) {
case IPPROTO_TCP:
 if (!pskb_may_pull(skb, ihl + sizeof(struct tcphdr)))
 return -1;
 break;
case IPPROTO_UDP:
 if (!pskb_may_pull(skb, ihl + sizeof(struct udphdr)))
 return -1;
 break;
case IPPROTO_ICMP:
 if (!pskb_may_pull(skb, ihl + sizeof(struct icmphdr)))
 return -1;
 break;
case IPPROTO_IGMP:
 if (!pskb_may_pull(skb, ihl + sizeof(struct igmpchr)))
 return -1;
 break;
case IPPROTO_GRE:
case IPPROTO_PIM:
 /* protocols only for L2/L3 matching, no further pull needed */
 break;
default:
 /* other protocols are allowed but not specifically handled here */
 break;
}
return 0;
}
IPFI_PRINTK("ipfire_core: check_headers: malformed ip hdr");
return -1;
}

static unsigned int local_output_dnat(const struct nf_hook_state *state,
 struct sk_buff *skb, ipfi_flow *flow) {
 if (fwopts.nat != 0 && flow->direction == IPFI_OUTPUT) {
 struct info_flags flags = {};
 struct response res = {.verdict = IPFI_ACCEPT};
 struct nat_table *dnt =
 get_dnatted_count() > 0 ? lookup_nat_forward(skb, NAT_DNAT) : NULL;
 int dnat_ret = -1;

 flags.direction = flow->direction;

 if (dnt != NULL) {
 res.rule_id = dnt->rule_id;
 dnat_ret = dest_translate(skb, dnt);
 nat_put(dnt);
 } else {
 dnat_ret = dnat_translation(skb, flow, &res, &flags);
 }

 if (dnat_ret >= 0) {
 flags.nat = 1;
 struct sock *routing_sk = skb->sk ? skb->sk : state->sk;
 if (ip_route_me_harder(state->net, routing_sk, skb, RTN_UNSPEC)) {
 IPFI_PRINTK("IPFIRE: ip_route_me_harder failed after OUTPUT DNAT\n");
 return NF_DROP;
 }

 if ((userspace_data_pid) && (loguser_enabled) &&

```

```

 (is_to_send(skb, &fwopts, &res, flow, &flags) > 0)) {
 int err;
 struct sk_buff *skb_touser =
 build_info_tnlmsg(skb, flow, &res, &flags, &err);
 if (skb_touser != NULL) {
 skb_send_to_user(skb_touser, LISTENER_DATA);
 } else {
 IPFI_PRINTK(
 "IPFIRE: failed to build log message after OUTPUT DNAT\n");
 }
}
}
}
return NF_ACCEPT;
}

unsigned int process(void *priv, struct sk_buff *skb,
 const struct nf_hook_state *state) {
 unsigned int hooknum = state->hook;
 const struct net_device *in = state->in;
 const struct net_device *out = state->out;

 unsigned int ret;
 __be32 daddr;
 ipfi_flow flow = {in, out, NODIRECTION};
 // malformed packet or unsupported protocol
 if (check_headers(skb) < 0)
 return NF_DROP;
 bool no_nat = (fwopts.masquerade == 0 && fwopts.nat == 0) ||
 (list_empty(&translation_pre.list) &&
 list_empty(&translation_post.list) &&
 list_empty(&translation_out.list) &&
 get_dnatted_count() == 0 && get_snatted_count() == 0);

 switch (hooknum) {
 case NF_IP_PRE_ROUTING:
 if (no_nat)
 return NF_ACCEPT;
 IPFI_STAT_INC(pre_rcv); // stats
 daddr = ip_hdr(skb)->daddr; /* save original destination address */
 flow.direction = IPFI_INPUT_PRE;
 ret = ipfi_pre_process(skb, &flow);
 // if (ret != NF_DROP && ret != NF_STOLEN && daddr != ip_hdr(skb)->daddr) {
 // destination nat applied and destination address changed in pre routing
 // dst_release(skb_dst(skb));
 // skb_dst_set(skb, NULL);
 // }
 return ret;
 case NF_IP_LOCAL_IN:
 IPFI_STAT_INC(in_rcv);
 flow.direction = IPFI_INPUT;
 return ipfi_response(state, skb, &flow);
 case NF_IP_LOCAL_OUT:
 IPFI_STAT_INC(out_rcv);
 flow.direction = IPFI_OUTPUT;

 /* 1. Evaluate firewall rules and update state based on ORIGINAL destination
 */
 ret = ipfi_response(state, skb, &flow);
 if (ret != NF_ACCEPT)
 return ret;
 }
}

```

```

/* 2. Apply OUTPUT DNAT (which modifies the destination) */
if (!no_nat && local_output_dnat(state, skb, &flow) == NF_DROP)
 return NF_DROP;

 return NF_ACCEPT;
case NF_IP_FORWARD:
 IPFI_STAT_INC(fwd_rcv);
 flow.direction = IPFI_FWD;
 return ipfi_response(state, skb, &flow);
case NF_IP_POST_ROUTING:
 if (no_nat)
 return NF_ACCEPT;
 IPFI_STAT_INC(post_rcv);
 flow.direction = IPFI_OUTPUT_POST;
 return ipfi_post_process(skb, &flow);
default:
 return NF_DROP;
}
return NF_DROP;
}

void init_kernel_stats(struct kernel_stats *nl_kstats) {
 memset(nl_kstats, 0, sizeof(struct kernel_stats));
}

/* updates sent counter, sends packet to userspace and calls
 * update_kernel_stats()
 */
inline int send_packet_to_userspace_and_update_counters(
 const struct sk_buff *skb, const ipfi_flow *flow,
 const struct response *resp, const struct info_flags *flags) {
 int err = 0;
 struct sk_buff *skb_to_user = NULL;

 skb_to_user = build_info_t_nmsg(skb, flow, resp, flags, &err);
 if (skb_to_user != NULL) {
 IPFI_STAT_INC(sent_tou);
 if (skb_send_to_user(skb_to_user, LISTENER_DATA) < 0) {
 printk("send_packet_to_userspace_and_update_counters skb_send failed\n");
 update_kernel_stats(IPFI_FAILED_NETLINK_USPACE, resp->verdict);
 return -1;
 }
 } else if (skb_to_user == NULL) {
 IPFI_PRINTK("IPFIRE: failed to allocate memory for socket buffer in "
 "send_packet_to_userspace_and_update_counters()\n");
 return -1;
 }
 return 0;
}

int ipfi_pre_process(struct sk_buff *skb, const ipfi_flow *flow) {
 int ret = -1;
 int verdict;
 struct response resp = {};
 struct info_flags flags = {};
 flags.direction = flow->direction;
 /* in pre routing we do not do any filtering. In normal cases we will return
 * IPFI_ACCEPT as verdict in this hook. In case of errors instead, we'll
 * return IPFI_DROP, to interrupt packet processing. This happens in case of
 * checksum error in incoming packets or (unprobable) memory allocation

```

```

 * errors
 */
 verdict = NF_ACCEPT;
 /* MASQUERADE or NAT are enabled: go on! */

 /* No more kcalloc for ipfire_info_t. We use stack-based flags and response.
 */

 /* No more build_ipfire_info_from_skb or init_packet needed for translation
 * here. Translation functions now take skb and components directly.
 */

 /* if it's an ICMP error for a NATed connection, handle it directly */
 if (ip_hdr(skb)->protocol == IPPROTO_ICMP) {
 if (icmp_nat_process(skb, NF_IP_PRE_ROUTING) == 1) {
 /* The outer Dst IP changed (e.g. local .245 -> remote .25).
 * The cached routing decision (LOCAL_IN) is now stale.
 * Drop it so the kernel re-evaluates routing -> FORWARD.
 */
 ret = 1;
 }
 }

 /* if a packet comes back from a dnatted connection,
 * i.e. has been forwarded, we must de_dnat it. Dynamic
 * rules are checked in this case.
 */
 if (get_dnatted_count() > 0) {
 ret = pre_de_dnat(skb, flow, &resp, &flags);

 /* implements pre processing of the packets: DNAT.
 * check if we already have a translation for this session
 * (early lookup consolidated here)
 */
 if (ret < 0) {
 struct nat_table *dnt = lookup_nat_forward(skb, NAT_DNAT);
 if (dnt != NULL) {
 ret = dest_translate(skb, dnt);
 nat_put(dnt);
 }
 }
 }

 /* dnat_translation() requires direct match with DNAT rules inserted by user.
 */
 if (ret < 0) /* no pre_de_dnat, maybe dnat_translation */
 ret = dnat_translation(skb, flow, &resp, &flags);

 /* now let's de-snat, or de-masquerade, if no match has previously succeeded
 */
 if (ret < 0 && get_snatted_count() > 0)
 ret = pre_de_snat(skb, flow, &resp, &flags);

 /* checksum is calculated inside set_pairs_in_skb(), ipfi_translation.c */
 if (ret >= 0) {
 resp.verdict = IPFI_ACCEPT;
 flags.nat = 1;

 /* we send to userspace old packet, to let see how translation changed it,
 * if loguser is greater than 5 */
 if ((userspace_data_pid) && (loguser_enabled) &&

```

```

 (is_to_send(skb, &fwopts, &resp, flow, &flags)))
 send_packet_to_userspace_and_update_counters(skb, flow, &resp, &flags);
else /* not sent because of log level */
 IPFI_STAT_INC(not_sent);
} else if (ret == BAD_CHECKSUM) {
 if (we_are_exiting) {
 IPFI_STAT_INC(not_sent);
 return NF_DROP;
 }

 IPFI_STAT_INC(bad_checksum_in);
 IPFI_PRINTK(
 "IPFIRE: bad checksum on incoming packet %llu over %llu received.\n",
 this_cpu_read(ipfi_counters->bad_checksum_in),
 this_cpu_read(ipfi_counters->pre_rcv));
 /* send a packet signaling an error */
 send_packet_to_userspace_and_update_counters(skb, flow, &resp, &flags);
 verdict = NF_DROP;
}
/* No more kfree(packet) needed */
return verdict;
}

int ipfi_post_process(struct sk_buff *skb, const ipfi_flow *flow) {
 struct response resp = {.verdict = IPFI_ACCEPT};
 struct info_flags flags = {};
 flags.direction = flow->direction;
 /* do post routing tasks. Checksumming is performed in set_pairs_in_skb(),
 * the manipulation function inside ipfi_translation.c
 */
 int de_dnat_done = -1, snat_done = -1;

 /* stats - use per-CPU counters to avoid cache-line bouncing */
 IPFI_STAT_INC(post_rcv);
 /* No more kcalloc for ipfire_info_t. */

 /* MASQUERADE and SNAT now take components directly. No
 * build_ipfire_info_from_skb needed here. */

 /* if it's an ICMP error for a NATed connection, handle it directly */
 if (ip_hdr(skb)->protocol == IPPROTO_ICMP) {
 if (icmp_nat_process(skb, NF_IP_POST_ROUTING) == 1) {
 /* Translation applied, accept packet immediately */
 IPFI_STAT_INC(post_rcv);
 snat_done = 1;
 goto send_touser;
 }
 }

 if (get_dnatted_count() > 0) {
 /*
 * Consolidate: both post_snat_dynamic() and de_dnat_translation()
 * do lookup_nat_idx(NAT_DNAT, NAT_IDX_POSTNAT, skb), which probes
 * the exact same hash bucket. Do ONE lookup and handle both cases:
 * - dynamic masquerade (forward packet through DNAT entry)
 * - de-DNAT (reply packet returning from DNAT target)
 */
 struct nat_table *dntmp = lookup_nat_idx(NAT_DNAT, NAT_IDX_POSTNAT, skb);
 if (dntmp) {
 /* Try dynamic masquerade first (forward direction) */
 if (snat_dynamic_table_match(dntmp, skb) > 0) {

```

```

 dntmp->state = state_machine(skb, dntmp->state, 0);
 ipfi_entry_update_timer(&dntmp->h, dntmp->protocol, dntmp->state);
 snat_done = snat_dynamic_translate(skb, dntmp, flow);

 /* Update/Add Reply index with the new Masquerade address */
 {
 u32 reply_key = get_nat_tuple_hash(dntmp->new_addr, dntmp->new_port,
 dntmp->our_ifaddr,
 dntmp->old_sport, dntmp->protocol);
 nat_add_index(dntmp, NAT_IDX_REPLY, reply_key);
 }

 nat_put(dntmp);
 goto send_touser;
}

/* Try de-DNAT (reverse direction) */
if (de_dnat_table_match(dntmp, skb) > 0) {
 dntmp->state = state_machine(skb, dntmp->state, 1);
 ipfi_entry_update_timer(&dntmp->h, dntmp->protocol, dntmp->state);
 de_dnat_done = de_dnat(skb, dntmp);
}
nat_put(dntmp);
}
}

/* check if we already have a translation for this session
 * (early lookup consolidated here, covers both SNAT and Masquerade)
 */
if (get_snatted_count() > 0) {
 struct nat_table *snt = lookup_nat_forward(skb, NAT_SNAT);
 if (snt != NULL) {
 snat_done = snat_packet(skb, snt);
 nat_put(snt);
 goto send_touser;
 }
}

/* masquerade_translation() scans masquerade_post list */
if ((snat_done = masquerade_translation(skb, flow, &resp, &flags)) >= 0)
 goto send_touser;

/* snat_translation scans translation_post list */
snat_done = snat_translation(skb, flow, &resp, &flags);

send_touser:

if ((snat_done >= 0) || (de_dnat_done >= 0)) {
 flags.nat = 1U;
 if (snat_done >= 0)
 flags.snat = 1U;
 if ((userspace_data_pid) && (loguser_enabled) &&
 (is_to_send(skb, &fwopts, &resp, flow, &flags)))
 send_packet_to_userspace_and_update_counters(skb, flow, &resp, &flags);
 else /* not sent because of log level */
 IPFI_STAT_INC(not_sent);
}

/* Don't check for bad checksum on outgoing packets.
 * See netfilter ip_conntrack_proto_tcp.c comment */

/* checksumming... */
return NF_ACCEPT;

```

```

}

int ipfi_response(const struct nf_hook_state *state, struct sk_buff *skb,
 ipfi_flow *flow) {
 /* 0.98.4: When we have to DNAT in output direction, we have to
 * check if the original destination is allowed by the
 * filter rules. Imagine we have setup a proxy web and the
 * user wants to block connections to www.badsite.com.
 * If the output connections are redirected to the proxy
 * by means of an OUTPUT DNAT rule, the connection will
 * pass through also if it is not desired.
 */
 struct info_flags flags = {};
 struct response res = iph_in_get_response(skb, flow, &flags);
 /* decide according to loguser if to send or not feedback */
 if ((userspace_data_pid) && (loguser_enabled)) {
 if ((is_to_send(skb, &fwopts, &res, flow, &flags) > 0)) {
 int err;
 struct sk_buff *skb_touser =
 build_info_t_nlmsg(skb, flow, &res, &flags, &err);
 if (skb_touser == NULL) /* shouldn't happen :r */
 IPFI_PRINTK("IPFIRE: failed to allocate memory for socket buffer in "
 "iph_in_get_response()\n");
 else if (skb_send_to_user(skb_touser, LISTENER_DATA) < 0)
 update_kernel_stats(IPFI_FAILED_NETLINK_USPACE, res.verdict);
 }

 /* Qt-GUI: if gui_notifier_enabled we send the info if there is no match
 * (popup asking the user the verdict for the new seen packet - if
 * directions are in or out -) OR if there was a rule matching and a
 * notification was requested for tha packets matching _that_ rule.
 */
 if (gui_notifier_enabled &&
 ((res.verdict == IPFI_IMPLICIT && flow->direction != IPFI_FWD) ||
 (res.verdict != IPFI_IMPLICIT && res.notify))) {
 printk("GUI NOTIFIER ENABLED!!\n");
 /* previous socket buffer sent to userspace: the kernel will have freed
 * it... so the pointer skb_touser is no more valid: create a new socket
 * buffer with the same ipfire_info_t contents: this will be sent via
 * the notifier socket.
 */
 int err = 0;
 struct sk_buff *skb_touser =
 build_info_t_nlmsg(skb, flow, &res, &flags, &err);
 if (skb_touser != NULL &&
 skb_send_to_user(skb_touser, GUI_NOTIF_DATA) < 0) {
 IPFI_PRINTK("failed to send message to userspace via netlink: %d\n",
 err);
 update_kernel_stats(IPFI_FAILED_NETLINK_USPACE, res.verdict);
 }
 }
 }

 } else /* packets not sent because of log level */ {
 IPFI_STAT_INC(not_sent);
 }

 /* update the sum of the packets processed */
 IPFI_STAT_INC(sum);

 /* update statistics */
 update_kernel_stats(flow->direction, res.verdict);
}

```

```

/* Map internal IPFI verdicts to netfilter verdicts:
 * IPFI_ACCEPT (2) -> NF_ACCEPT (1)
 * IPFI_DROP (1) -> NF_DROP (0)
 * IPFI_IMPLICIT (0) -> apply default_policy
 */
if (res.verdict == IPFI_ACCEPT)
 return NF_ACCEPT;
else if (res.verdict == IPFI_DROP)
 return NF_DROP;
else {
 /* IPFI_IMPLICIT: apply default policy */
 if (default_policy == IPFI_ACCEPT)
 return NF_ACCEPT;
 return NF_DROP;
}
}

/* no more needed since 1.96 */
int recalculate_ip_checksum(struct sk_buff *skb, int direction) {
 int len;
 int datalen;
 int iph_sum_old;
 struct iphdr *iph;
 struct tcphdr *th;
 struct udphdr *uh;

 if (skb == NULL)
 return network_header_null("recalculate_ip_checksum()",
 "socket buffer NULL");

 len = skb->len;
 iph = ip_hdr(skb);

 datalen = skb->len - iph->ihl * 4;
 iph_sum_old = iph->check;
 iph->check = 0;
 iph->check = ip_fast_csum((u8 *)iph, iph->ihl);

 /* tcp */
 switch (iph->protocol) {
 case IPPROTO_TCP:
 th = (struct tcphdr *)((void *)iph + iph->ihl * 4);
 th->check = 0;
 th->check = tcp_v4_check(datalen, iph->saddr, iph->daddr,
 skb_checksum(skb, iph->ihl * 4, datalen, 0));

 break;
 case IPPROTO_UDP:
 uh = (struct udphdr *)((void *)iph + iph->ihl * 4);
 uh->check = 0;
 uh->check = udp_v4_check(datalen, iph->saddr, iph->daddr,
 skb_checksum(skb, iph->ihl * 4, datalen, 0));

 break;
 default:
 break;
 }

 /* see include/skbuff.h: "B.Checksumming on output":
 * CHECKSUM_NONE: skb is checksummed by protocol or csum is not required.
 * If checksum is calculated here, no need to recalculate it. */
 if ((direction == IPFI_OUTPUT) || (direction == IPFI_OUTPUT_POST))
 skb->ip_summed = CHECKSUM_NONE;
 return 0;
}

```



```

}

module_init(ini);
module_exit(fini);

```

## ipfi\_macros.h

```

/* macros in ip_input.c, ip_output.c, ip_forward.c */

#define IPFI_INPUT_FUNCTION() \
ipfi_packet_input_counter++; \
 if(ipfi_input_processing != NULL) /* pre routing processing */ \
{ \
 printk("INPUT: LOCAL_DELIVER() [LOCAL_IN]: %lu\n", ipfi_packet_input_counter); \
 \
 if(ipfi_input_processing(skb, ipfi_packet_input_counter, IPFI_INPUT) == \
IPFI_DROP) \
 return IPFI_DROP; \
} \
else \
{ \
 if(ipfi_packet_input_counter % PRINTK_WARN_IPFI_DISABLED == 0) \
 printk("IPFI: WARNING: FIREWALL DISABLED [input packet n. %lu]\n", \
 ipfi_packet_input_counter); \
} \

#define IPFI_PRE_FUNCTION() \
ipfi_packet_pre_counter++; \
if(ipfi_pre_processing != NULL) /* pre routing processing */ \
{ \
 printk("INPUT: IP_RCV [LOCAL_PRE]: %lu\n", ipfi_packet_pre_counter); \
 if(ipfi_pre_processing(skb, ipfi_packet_pre_counter, IPFI_INPUT_PRE) == \
IPFI_DROP) \
 return IPFI_DROP; \
} \
else \
{ \
 if(ipfi_packet_pre_counter % PRINTK_WARN_IPFI_DISABLED == 0) \
 printk("IPFI: WARNING: FIREWALL DISABLED [input pre packet n. %lu]\n", \
 ipfi_packet_pre_counter); \
} \

#define IPFI_OUTPUT_FUNCTION() \
ipfi_packet_output_counter++; \
 if(ipfi_output_processing != NULL) /* pre routing processing */ \
{ \
 if(ipfi_output_processing(skb, ipfi_packet_output_counter, IPFI_OUTPUT) \
 == IPFI_DROP) \
 return IPFI_DROP; \
} \
else \
{ \
 if(ipfi_packet_output_counter % PRINTK_WARN_IPFI_DISABLED == 0) \
 printk("IPFI: WARNING: FIREWALL DISABLED [output packet n. %lu]\n", \
 ipfi_packet_output_counter); \
} \

```

```

#define IPFI_POST_FUNCTION() \
ipfi_post_packet_counter++; \
 printk("IPFI_FINISH_OUTPUT() [POSTROUTING]: %lu\n", ipfi_post_packet_counter); \
 if(ipfi_post_processing == NULL) \
{ \
 if(ipfi_post_packet_counter % PRINTK_WARN_IPFI_DISABLED == 0) \
 printk("IPFI: WARNING FIREWALL DISABLED [output post packet n.%lu]\n", \
 ipfi_post_packet_counter); \
} \
 else \
{ \
 if(ipfi_post_processing(skb, ipfi_post_packet_counter, IPFI_OUTPUT_POST) == \
IPFI_DROP) \
 return IPFI_DROP; \
}

#define IPFI_FWD_FUNCTION() \
ipfi_packet_forward_counter ++; \
if(ipfi_forward_processing != NULL) /* pre routing processing */ \
{ \
 if(ipfi_forward_processing(skb, ipfi_packet_forward_counter, IPFI_FWD) == \
IPFI_DROP) \
 return IPFI_DROP; \
} \
else \
{ \
 if(ipfi_packet_forward_counter % PRINTK_WARN_IPFI_DISABLED == 0) \
 printk("IPFI: WARNING: FIREWALL DISABLED [forward packet n. %lu]\n", \
 ipfi_packet_forward_counter); \
}

```

## ipfi\_entry.h

```

/* ipfi_entry.h - Common lifecycle header for all ipfire table entries
 *
 * Every table entry struct (state_table, nat_table) embeds an
 * ipfi_entry_head as its FIRST member. The lifecycle functions in
 * table_lifecycle.c operate on this embedded header.
 */
#ifndef IPFI_ENTRY_H
#define IPFI_ENTRY_H

#include <linux/hashtable.h>
#include <linux/jhash.h>
#include <linux/list.h>
#include <linux/percpu_counter.h>
#include <linux/rcupdate.h>
#include <linux/refcount.h>
#include <linux/spinlock.h>
#include <linux/timer.h>
#include <linux/workqueue.h>

/* Bit index for the "entry has been removed from its list" flag */
#define IPFI_ENTRY_REMOVED 0

extern struct workqueue_struct *ipfire_wq;

```

```

/**
 * struct ipfi_entry_head - common header embedded in every table entry
 *
 * This must be the FIRST member of any table entry struct, so that
 * container_of(head, struct whatever, h) works uniformly and the
 * pointer to the head IS the pointer to the entry.
 */
struct ipfi_entry_head {
 struct timer_list timer;
 struct work_struct cleanup_work;
 struct rcu_head rcuh;
 struct list_head lnode; /* list linkage (used by loginfo LRU) */
 struct hlist_node hnode; /* hash linkage */
 refcount_t refcnt;
 unsigned long status;
 unsigned long last_timer_update;
};

/* ===== Lifecycle functions (table_lifecycle.c) ===== */

/**
 * ipfi_entry_init - Initialise lifecycle fields of a new entry.
 * @h: pointer to the embedded head
 * @timeout_s: initial timeout in seconds
 *
 * Sets up the timer, work struct, refcount (at 0 - caller must set it),
 * status (cleared), and initial expiry.
 */
void ipfi_entry_init(struct ipfi_entry_head *h, unsigned int timeout_s,
 void (*timeout_fn)(struct timer_list *));

/**
 * ipfi_entry_arm_timer - Start the timer after entry has been added to its
 * list.
 */
static inline void ipfi_entry_arm_timer(struct ipfi_entry_head *h) {
 mod_timer(&h->timer, h->timer.expires);
}

/**
 * ipfi_entry_hold / ipfi_entry_put - refcount management.
 *
 * When refcount drops to zero, _put queues the cleanup work which
 * syncs the timer and then schedules RCU-delayed kfree.
 */
static inline void ipfi_entry_hold(struct ipfi_entry_head *h) {
 refcount_inc(&h->refcnt);
}

/**
 * ipfi_entry_hold_rcu - Try to acquire a reference in RCU context.
 * Returns true if successful, false if the entry is already dead (0 refcnt).
 */
static inline bool ipfi_entry_hold_rcu(struct ipfi_entry_head *h) {
 return refcount_inc_not_zero(&h->refcnt);
}

void ipfi_entry_put(struct ipfi_entry_head *h);

/**
 * ipfi_entry_update_timer - Throttled timer refresh.

```

```

* @h: entry head
* @proto: protocol (IPPROTO_*)
* @state: current connection state (int, passed to get_timeout_by_state)
*
* Skips if REMOVED is set or if last update was <5 s ago.
*/
void ipfi_entry_update_timer(struct ipfi_entry_head *h, int proto, int state);

/**
* ipfi_entry_remove - Unlink an entry from its container (under caller's lock).
* @h: entry head
* @counter: pointer to the entry counter (decremented)
*
* List build: does list_del_rcu + set_bit(REMOVED) + counter-- + put.
* Hash build: does hlist_del_rcu + set_bit(REMOVED) + counter-- + put.
* The caller MUST hold the protecting spinlock.
*
* The thread that wins the test_and_set_bit(REMOVED) race calls
* ipfi_entry_put(h) internally. The loser is a complete no-op.
* Callers must NOT call ipfi_entry_put() after this function.
*
* ipfi_entry_put() calls queue_work(), which is safe from BH-disabled
* spinlock context (queue_work takes an internal irqsave lock, never sleeps).
*/
void ipfi_entry_remove(struct ipfi_entry_head *h,
 struct percpu_counter *counter);

/**
* ipfi_table_flush_all - Remove and put all lnode-linked entries.
*
* Process-context only (sleepable).
*/
int ipfi_table_flush_all(struct list_head *list, spinlock_t *lock,
 struct percpu_counter *counter);

/**
* ipfi_table_flush_hash - Remove and put all hnode-linked entries.
*
* Used by the state table and log table.
* Process-context only (sleepable).
*/
int ipfi_table_flush_hash(struct hlist_head *ht, unsigned int nbuckets,
 spinlock_t *lock, struct percpu_counter *counter);

/**
* ipfi_table_flush_hash_bucketed - Remove and put all hnode-linked entries
* using per-bucket spinlocks.
*
* Process-context only (sleepable).
*/
int ipfi_table_flush_hash_bucketed(struct hlist_head *ht, unsigned int nbuckets,
 spinlock_t *locks,
 struct percpu_counter *counter);

#endif

/* IPFI_ENTRY_H */

```

## table\_lifecycle.c

```
/* table_lifecycle.c - Unified lifecycle management for ipfire table entries
 *
 * Provides refcounting, timer management, RCU-safe destruction, and
 * bulk flush for any struct that embeds an ipfi_entry_head as its
 * first member (state_table, nat_table, ipfire_logininfo).
 *
 * - List build (default off): uses list_head / lnode for O(N) traversal
 * - Hash build (default on): uses hlist_node / hnode for O(1) lookups
 *
 * =====
 * OWNERSHIP AND DELETION CHAIN
 * =====
 *
 * Every entry starts with refcount = 1 (the "container reference").
 * The lifecycle is:
 *
 * CREATED (refcnt=1)
 * |
 * ▼
 * LINKED into list/hash + timer armed
 * |
 * ▼ timer fires —or— eviction (logininfo only)
 * ipfi_entry_remove() [under table spinlock]
 * | test_and_set_bit(REMOVED) ← atomic "winner" gate
 * | hlist_del_rcu / list_del_rcu
 * | counter--
 * | ipfi_entry_put() ← winner puts, loser is no-op
 * | ↳ refcount → 0
 * | ↳ queue_work(ipfire_wq, cleanup_work)
 * | ↳ [workqueue / process context]
 * | timer_delete_sync()
 * | call_rcu(free_entry_rcu)
 * | ↳ [RCU callback]
 * | kfree(h)
 *
 * KEY INVARIANTS
 * =====
 *
 * 1. REMOVED bit is set exactly once (test_and_set_bit).
 * 2. ipfi_entry_put is called exactly once (by the winner of the bit race).
 * 3. kfree happens only after: timer is dead AND RCU grace period has
 * elapsed, guaranteeing no CPU is still touching the object.
 * 4. queue_work() is safe from BH-disabled spinlock context because it
 * uses spin_lock_irqsave internally and never sleeps.
 *
 * LOGININFO SPECIAL CASE (see log.c)
 * =====
 *
 * logininfo entries maintain TWO simultaneous links in hash mode:
 * lnode → active_logi_list (LRU eviction, O(1) tail access)
 * hnode → logininfo_hashtable (deduplication lookup, O(1))
 *
 * ipfi_entry_remove only handles hnode. The lnode is managed explicitly
 * by the logininfo timer callback and eviction path, both under
 * logininfo_list_lock, guarded by a REMOVED pre-check to prevent
 * double list_del_rcu (which would write to LIST_POISON addresses).
 *
 * =====
 */
```

```

#include "ipfi_entry.h"
#include <linux/bitops.h>
#include <linux/jiffies.h>
#include <linux/list.h>
#include <linux/percpu_counter.h>
#include <linux/rculist.h>
#include <linux/slab.h>
#include <linux/spinlock.h>

/* Forward declaration: timeout helper from state_timeout.c */
unsigned int get_timeout_by_state(int protocol, int state);

/* ---- RCU callback: final kfree ---- */

static void free_entry_rcu(struct rcu_head *rcu) {
 struct ipfi_entry_head *h = container_of(rcu, struct ipfi_entry_head, rcuh);
 /*
 * All entries are allocated via kmem_cache_alloc.
 * kfree() handles kmem_cache objects correctly on
 * modern kernels (6.x) via virt_to_slab().
 */
 kfree(h);
}

/* ---- Work handler: process-context cleanup ---- */

static void free_entry_work(struct work_struct *work) {
 struct ipfi_entry_head *h =
 container_of(work, struct ipfi_entry_head, cleanup_work);

 /* Safe in process context */
 timer_delete_sync(&h->timer);

 /* After RCU grace period, kfree the entry */
 call_rcu(&h->rcuh, free_entry_rcu);
}

/* ---- Public API ---- */

void ipfi_entry_init(struct ipfi_entry_head *h, unsigned int timeout_s,
 void (*timeout_fn)(struct timer_list *)) {
 INIT_WORK(&h->cleanup_work, free_entry_work);
 timer_setup(&h->timer, timeout_fn, 0);
 h->timer.expires = jiffies + HZ * timeout_s;
 h->status = 0;
 h->last_timer_update = jiffies;
 /* refcnt is NOT set here – caller is responsible
 * (refcount_set before or after init as appropriate). */
}

void ipfi_entry_put(struct ipfi_entry_head *h) {
 if (!refcount_dec_and_test(&h->refcnt))
 return;

 /*
 * Defer final cleanup via workqueue so we don't risk kfree'ing
 * while other contexts (RCU readers, timers) may touch the object.
 */
 if (!ipfire_wq) {
 WARN_ON(1);
 }
}

```

```

 queue_work(system_wq, &h->cleanup_work);
} else {
 queue_work(ipfire_wq, &h->cleanup_work);
}
}

void ipfi_entry_update_timer(struct ipfi_entry_head *h, int proto, int state) {
 if (unlikely(test_bit(IPFI_ENTRY_REMOVED, &h->status)))
 return;

 if (time_after(jiffies, READ_ONCE(h->last_timer_update) + (5 * HZ))) {
 unsigned int timeout;

 /* Safely get a reference while under RCU/pointer validity */
 if (!ipfi_entry_hold_rcu(h))
 return;

 /* Re-check removal bit under the reference */
 if (unlikely(test_bit(IPFI_ENTRY_REMOVED, &h->status))) {
 ipfi_entry_put(h);
 return;
 }

 timeout = get_timeout_by_state(proto, state);
 mod_timer(&h->timer, jiffies + HZ * timeout);
 WRITE_ONCE(h->last_timer_update, jiffies);

 ipfi_entry_put(h);
 }
}

/**
 * ipfi_entry_remove - Unlink an entry from its container (under caller's lock).
 *
 * CRITICAL: test_and_set_bit(REMOVED) MUST happen BEFORE the del_rcu call.
 * Only the thread that successfully sets the bit unlinks the entry.
 */
void ipfi_entry_remove(struct ipfi_entry_head *h,
 struct percpu_counter *counter) {
 if (test_and_set_bit(IPFI_ENTRY_REMOVED, &h->status))
 return; /* loser: complete no-op */

 /* Entries are tracked by hnode in hash mode */
 hlist_del_rcu(&h->hnode);
 percpu_counter_add_batch(counter, -1, 1);

 /*
 * Winner always puts. queue_work() is safe from BH-disabled spinlock
 * context: it uses spin_lock_irqsave internally and never sleeps.
 */
 ipfi_entry_put(h);
}

/* =====
 * List-mode bulk flush (always compiled – used by NAT tables
 * regardless of whether hash mode is active for the state table)
 * ===== */

int ipfi_table_flush_all(struct list_head *list, spinlock_t *lock,
 struct percpu_counter *counter) {
 struct ipfi_entry_head *h, *tmp;

```

[illegible]



```

struct ipfi_entry_head *h;
struct hlist_node *tmp;
unsigned int bkt;
int count;

for (bkt = 0; bkt < nbuckets; bkt++) {
 spin_lock_bh(&locks[bkt]);
 hlist_for_each_entry_safe(h, tmp, &ht[bkt], hnode) {
 if (!test_and_set_bit(IPFI_ENTRY_REMOVED, &h->status)) {
 hlist_del_rcu(&h->hnode);
 ipfi_entry_put(h);
 }
 }
 spin_unlock_bh(&locks[bkt]);
}

/* Since we don't have a global lock during flush, we just sum up what's left
 * and reset. This is typically used during unload where traffic flow is
 * stopped anyway. */
count = percpu_counter_sum(counter);
percpu_counter_set(counter, 0);

return count;
}

```

## filter/filter\_engine.h

```

#ifndef IPFI_FILTER_ENGINE_H
#define IPFI_FILTER_ENGINE_H

#include <linux/skbuff.h>
#include <common/ipfi_structures.h>
#include "ipfire.h" /* for ipfire_options if needed, or forward declare */

/* struct ipfire_options is defined in ipfi.h usually or ipfi_structures?
 * It's in ipfi.h based on typical usage. Let's check ipfi.h content if needed.
 * For now assuming it's available via ipfi.h inclusion.
 */

struct response ipfire_filter(const ipfire_rule *denied,
 const ipfire_rule *allowed,
 const struct ipfire_options *ipfi_opts,
 struct sk_buff *skb, const ipfi_flow *flow,
 struct info_flags *flags);

#endif

```

## filter/filter\_engine.c

```

/* filter/filter_engine.c: Main filtering engine for ipfire-wall */

#include <common/ipfi_structures.h>
#include <linux/icmp.h>
#include <linux/inetdevice.h>
#include <linux/ip.h>

```

```

#include <linux/list.h>
#include <linux/module.h>
#include <linux/skbuff.h>
#include <linux/tcp.h>
#include <linux/udp.h>

struct state_table;
#include "globals.h"
#include "ipfi_machine.h"
#include "ipfire.h"
#include "mangle/mangle.h"
#include "netlink/ipfi_netl.h"
#include "netlink/message_builder.h"
#include "state/state_machine.h"
#include "state/state_table.h"

struct response ipfire_filter(const ipfire_rule *dropped,
 const ipfire_rule *allowed,
 const struct ipfire_options *ipfi_opts,
 struct sk_buff *skb, const ipfi_flow *flow,
 struct info_flags *flags) {
 struct response response = {
 .verdict = IPFI_IMPLICIT,
 };
 ipfire_rule *rule;
 short pass;
 short res;
 short drop = 0;
 struct state_table *newtable = NULL;
 struct iphdr *iph = ip_hdr(skb);

 if (flow->direction == IPFI_INPUT || flow->direction == IPFI_OUTPUT ||
 flow->direction == IPFI_FWD) {
 {
 __u8 ftp_tmp = flags->ftp;
 response = check_state(skb, flow, &ftp_tmp);
 flags->ftp = ftp_tmp;
 }
 pass = response.verdict;
 if (pass > 0) {
 response.state = 1U;
 return response;
 }
 }

 rcu_read_lock();
 list_for_each_entry_rcu(rule, &dropped->list, list) {
 if ((res = direction_filter(flow->direction, rule)) < 0)
 goto next_drop_rule;
 else if (res > 0)
 drop = 1;

 if ((res = device_filter(rule, flow->in, flow->out)) < 0)
 goto next_drop_rule;
 else if (res > 0)
 drop = 1;

 if ((res = ip_layer_filter(iph, rule, flow->direction, flow->in,
 flow->out)) < 0)
 goto next_drop_rule;
 else if (res > 0)

```

```

 drop = 1;

 if (iph->protocol == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)((void *)iph + iph->ihl * 4);
 if ((res = ipfi_tcp_filter(th, rule)) < 0)
 goto next_drop_rule;
 else if (res > 0)
 drop = 1;
 } else if (iph->protocol == IPPROTO_UDP) {
 struct udphdr *uh = (struct udphdr *)((void *)iph + iph->ihl * 4);
 if ((res = udp_filter(uh, rule)) < 0)
 goto next_drop_rule;
 else if (res > 0)
 drop = 1;
 } else if (iph->protocol == IPPROTO_ICMP) {
 struct icmphdr *ih = (struct icmphdr *)((void *)iph + iph->ihl * 4);
 if ((res = icmp_filter(ih, rule)) < 0)
 goto next_drop_rule;
 else if (res > 0)
 drop = 1;
 }

 if (drop > 0) {
 response.state = 0;
 response.verdict = IPFI_DROP;
 response.notify = rule->notify;
 response.rule_id = rule->rule_id;
 rcu_read_unlock();
 return response;
 }

next_drop_rule:;
}

pass = 0;
list_for_each_entry_rcu(rule, &allowed->list, list) {
 if ((res = direction_filter(flow->direction, rule)) < 0)
 goto next_pass_rule;
 else if (res > 0)
 pass = 1;

 int in_lo = 0, out_lo = 0;
 if (flow->in != NULL && strcmp(flow->in->name, "lo") == 0)
 in_lo = 1;
 if (flow->out != NULL && strcmp(flow->out->name, "lo") == 0)
 out_lo = 1;

 if ((res = device_filter(rule, flow->in, flow->out)) < 0) {
 if (in_lo) {
 printk("loopback in: failed device filters\n");
 }
 if (out_lo) {
 printk("loopback out: failed device filters\n");
 }
 goto next_pass_rule;
 } else if (res > 0)
 pass = 1;

 if ((res = ip_layer_filter(iph, rule, flow->direction, flow->in,
 flow->out)) < 0)
 goto next_pass_rule;
}

```

```

else if (res > 0)
 pass = 1;

if (iph->protocol == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)((void *)iph + iph->ihl * 4);
 if ((res = ipfi_tcp_filter(th, rule)) < 0)
 goto next_pass_rule;
 else if (res > 0)
 pass = 1;
} else if (iph->protocol == IPPROTO_UDP) {
 struct udphdr *uh = (struct udphdr *)((void *)iph + iph->ihl * 4);
 if ((res = udp_filter(uh, rule)) < 0)
 goto next_pass_rule;
 else if (res > 0)
 pass = 1;
} else if (iph->protocol == IPPROTO_ICMP) {
 struct icmphdr *ih = (struct icmphdr *)((void *)iph + iph->ihl * 4);
 if ((res = icmp_filter(ih, rule)) < 0)
 goto next_pass_rule;
 else if (res > 0)
 pass = 1;
}

if (pass > 0) {
 response.state = 0U;
 response.verdict = IPFI_ACCEPT;
 response.notify = rule->notify;
 response.rule_id = rule->rule_id;
 response.nolog = rule->nflags.nolog; /* propagate nolog flag */
}

if ((pass > 0) && ((rule->state) || (ipfi_opts->all_stateful)) &&
 (ipfi_opts->state)) {
 if (flow->direction == IPFI_INPUT || flow->direction == IPFI_OUTPUT ||
 flow->direction == IPFI_FWD) {
 newtable = keep_state(skb, rule, flow);
 response.state = 1U;
 }
}

if (pass > 0) {
 rcu_read_unlock();
 if (newtable != NULL) {
 add_state_table_to_list(newtable);
 state_put(newtable);
 }
 if (mangle_skb(&rule->pkmangle, skb, flow, 0) < 0) {
 IPFI_PRINTK("IPFIRE: ipfire_filter(): MSS mangle failed for rule\n");
 }
 return response;
}
next_pass_rule;;
}
rcu_read_unlock();
return response;
}

struct state_table *keep_state(const struct sk_buff *skb,
 const ipfire_rule *p_rule,
 const ipfi_flow *flow) {
 ipfire_info_t *ipfi_info_warn;
 if (p_rule == NULL)

```

```

 return NULL;

if (percpu_counter_read(&state_tables_counter) >= max_state_entries) {
 ipfi_info_warn =
 (ipfire_info_t *)kmalloc(sizeof(ipfire_info_t), GFP_ATOMIC);
 if (ipfi_info_warn != NULL) {
 memset(ipfi_info_warn, 0, sizeof(ipfire_info_t));
 ipfi_info_warn->flags.state_max_entries = 1;
 struct sk_buff *skbi = build_info_t_packet(ipfi_info_warn);
 if (skbi != NULL && skb_send_to_user(skbi, LISTENER_DATA) < 0)
 IPFI_PRINTK("IPFIRE: error notifying maximum number of state entries "
 "to user\n");
 else if (skbi == NULL)
 IPFI_PRINTK(
 "IPFIRE: failed to allocate socket buffer space in keep_state()\n");
 kfree(ipfi_info_warn);
 } else
 IPFI_PRINTK(
 "IPFIRE: memory allocation error in keep_state, ipfi_machine.c\n");

 IPFI_PRINTK("IPFIRE: reached maximum count for STATE entries: %lld\n",
 (long long)percpu_counter_read(&state_tables_counter));
 return NULL;
}

struct state_table *state_t =
 (struct state_table *)kmem_cache_alloc(state_cache, GFP_ATOMIC);
if (!state_t) {
 IPFI_PRINTK("IPFIRE: memory allocation error in keep_state "
 "(kmem_cache_alloc failed)\n");
 return NULL;
}
memset(state_t, 0, sizeof(struct state_table));
refcount_set(&state_t->h.refcnt, 1); // initial refcount
if (fill_net_table_fields(state_t, skb, flow) < 0) {
 IPFI_PRINTK("IPFIRE: fill_net_table_fields failed, ipfi_machine.c\n");
 kmem_cache_free(state_cache, state_t);
 return NULL;
} else if (set_state(skb, state_t, 0) < 0) {
 IPFI_PRINTK("IPFIRE: invalid state when adding new state entry!\n");
 kmem_cache_free(state_cache, state_t);
 return NULL;
}

if (p_rule->nflags.ftp) {
 state_t->ftp = FTP_LOOK_FOR;
}

state_t->rule_id = p_rule->rule_id;
state_t->notify = p_rule->notify;
state_t->nolog = p_rule->nflags.nolog;
state_t->admin = !p_rule->owner;

return state_t;
}

void fill_state_info(struct state_info *stinfo, const struct state_table *stt) {
 stinfo->saddr = stt->saddr;
 stinfo->daddr = stt->daddr;
 stinfo->sport = stt->sport;
 stinfo->dport = stt->dport;
 stinfo->protocol = stt->protocol;
 stinfo->state = stt->state;
 stinfo->direction = stt->direction;
}

```

```

stinfo->ftp = stt->ftp;
stinfo->rule_id = stt->rule_id;

/* Calculate remaining timeout from timer */
if (timer_pending(&stt->h.timer)) {
 long remaining = stt->h.timer.expires - jiffies;
 stinfo->timeout = remaining > 0 ? (unsigned int)(remaining / HZ) : 0;
} else {
 stinfo->timeout = 0;
}

strncpy(stinfo->in_devname, stt->in_devname, IFNAMSIZ);
strncpy(stinfo->out_devname, stt->out_devname, IFNAMSIZ);
stinfo->notify = stt->notify;
stinfo->admin = stt->admin;
}

int get_dev_ifaddr(__u32 *addr, int direction, const struct net_device *in,
 const struct net_device *out) {
 switch (direction) {
 case IPFI_INPUT_PRE:
 case IPFI_INPUT:
 if (in) {
 if (get_ifaddr_by_name(in->name, addr) < 0) {
 IPFI_PRINTK("IPFIRE: direction: input no interface matching name %s!\n",
 in->name);
 return -1;
 }
 } else {
 return -1;
 }
 break;
 case IPFI_OUTPUT_POST:
 case IPFI_OUTPUT:
 if (out) {
 if (get_ifaddr_by_name(out->name, addr) < 0) {
 printk("IPFIRE: direction: output: no interface matching name %s!\n",
 out->name);
 return -1;
 }
 } else {
 return -1;
 }
 break;
 default:
 printk("IPFIRE: cannot get my address for direction %d!\n", direction);
 return -1;
 }
 return 0;
}

int get_ifaddr_by_name(const char *ifname, __u32 *addr) {
 struct net_device *pnet_device;
 struct in_device *pin_device;
 struct in_ifaddr *inet_ifaddr;

 rcu_read_lock();
 for_each_netdev_rcu(&init_net, pnet_device) {
 if ((netif_running(pnet_device)) && (pnet_device->ip_ptr != NULL) &&
 (strcmp(pnet_device->name, ifname) == 0)) {
 pin_device = (struct in_device *)pnet_device->ip_ptr;

```

```

 inet_ifaddr = pin_device->ifa_list;
 if (inet_ifaddr == NULL) {
 IPFI_PRINTK("ifa_list is null!\n");
 break;
 }
 *addr = (inet_ifaddr->ifa_local);
 rcu_read_unlock();
 return 1;
}
}

rcu_read_unlock();
return -1;
}

int add_ftp_dynamic_rule(struct state_table *ftpt) {
 int ret;

 if (percpu_counter_read(&state_tables_counter) >= max_state_entries) {
 IPFI_PRINTK("IPFIRE: reached maximum count for STATE entries "
 "(adding FTP rule): %lld\n",
 (long long)percpu_counter_read(&state_tables_counter));
 return -1;
 }
 ftpt->state.state = FTP_NEW;

 if (ftpt->ftp != FTP_DEFINED) {
 IPFI_PRINTK("IPFIRE: ftp support: you shouldn't be here without "
 "FTP_DEFINED set!\n");
 return -1;
 }

 ret = add_state_table_to_list(ftpt);
 if (ret < 0) {
 IPFI_PRINTK("IPFIRE FTP: add_state_table_to_list failed: %d\n", ret);
 return -1;
 }
 return 0;
}

```

## filter/rule\_match.h

```

#ifndef IPFI_RULE_MATCH_H
#define IPFI_RULE_MATCH_H

#include <linux/skbuff.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/udp.h>
#include <linux/icmp.h>
#include <linux/netdevice.h>
#include <common/ipfi_structures.h>

int direction_filter(int direction, const ipfire_rule *r);
int device_filter(const ipfire_rule *r, const struct net_device *in, const struct
net_device *out);
int address_match(const struct iphdr *iph, const ipfire_rule *r, int direction, const
struct net_device *in, const struct net_device *out);

```

```

int port_match(const struct tcphdr *tcph, const struct udphdr *udph, const ipfire_rule
*r, short protocol);
int ip_layer_filter(const struct iphdr *iph, const ipfire_rule *r, int direction, const
struct net_device *in, const struct net_device *out);
int ipfi_tcp_filter(const struct tcphdr *tcph, const ipfire_rule *r);
int udp_filter(const struct udphdr *udph, const ipfire_rule *r);
int icmp_filter(const struct icmphdr *icmph, const ipfire_rule *r);

#endif

```

## filter/rule\_match.c

```

/* filter/rule_match.c: Rule matching logic for ipfire-wall */

#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/udp.h>
#include <linux/icmp.h>
#include <linux/netdevice.h>
#include "ipfire.h"
#include "ipfi_machine.h"

inline int direction_filter(int direction, const ipfire_rule * r)
{
 if (r->nflags.direction)
 {
 if (r->direction != direction)
 return -1;
 return 1;
 }
 return 0;
}

int device_filter(const ipfire_rule * r,
 const struct net_device *in,
 const struct net_device *out)
{
 /* don't bother if user fills in output rule within input context
 * or viceversa */
 if (r->nflags.indev && in != NULL) {
 if (strncmp(in->name, r->devpar.in_devname, IFNAMSIZ) == 0)
 return 1;
 else
 return -1;
 }

 if (r->nflags.outdev && out != NULL) {
 if (strncmp(out->name, r->devpar.out_devname, IFNAMSIZ) == 0)
 return 1;
 else
 return -1;
 }
 return 0;
}

int address_match(const struct iphdr * iph,
 const ipfire_rule * r,
 int direction,

```



```

 const struct net_device *in,
 const struct net_device *out)
{
 int match = 0;
 int i, addr_in_list;
 __u32 source_address, p_source_address = 0;
 __u32 dest_address, p_dest_address = 0;

 /* source address */
 /* Set correct source and destination address: the one corresponding
 * to the interface in packet if MYADDR was specified, the dotted
 * decimal one if ADDR is specified in flags */
 if (r->nflags.src_addr == MYADDR) {
 if (get_dev_ifaddr(&p_source_address, direction, in, out) < 0) {
 return -1;
 }
 source_address = p_source_address;
 }
 else
 source_address = r->ip.ipsrc[0];

 /* initialize addr_in list */
 addr_in_list = 0;
 /* a single ip is given: it must match exactly */
 if ((r->nflags.src_addr) && (r->parmean.samean == SINGLE))
 {
 if (source_address != iph->saddr)
 return -1;
 else
 match = 1;
 }
 /* an interval is given: src address in packet must be contained in it.
 * Addresses cannot be mine */
 else if ((r->nflags.src_addr == ONEADDR) && (r->parmean.samean == INTERVAL))
 {
 if ((ntohl(r->ip.ipsrc[0]) <= ntohl(iph->saddr)) && (ntohl(r->ip.ipsrc[1]) >=
ntohl(iph->saddr))) {
 match = 1;
 }
 else
 return -1;
 }
 /* address in packet must be different from address in rule */
 else if ((r->nflags.src_addr) && (r->parmean.samean == DIFFERENT_FROM))
 {
 if (source_address != iph->saddr)
 match = 1;
 else
 return -1;
 }
 /* finally: src address in packet must be not included in rule source ip interval */
 else if ((r->nflags.src_addr == ONEADDR) && (r->parmean.samean ==
INTERVAL_DIFFERENT_FROM))
 {
 if ((ntohl(r->ip.ipsrc[0]) <= ntohl(iph->saddr)) && (ntohl(r->ip.ipsrc[1]) >=
ntohl(iph->saddr)))
 return -1;
 else /* saddr of iphead not contained: ok */
 match = 1;
 }
 else if ((r->nflags.src_addr == ONEADDR) && (r->parmean.samean == MULTI))

```

```

{
 for(i = 0; i < MAXMULTILEN && r->ip.ipsrc[i] != 0; i++)
 {
 if(r->ip.ipsrc[i] == iph->saddr)
 {
 addr_in_list = 1;
 break; /* no need to go further on */
 }
 }
 if(addr_in_list == 1)
 match = 1;
 else
 return -1;
}
else if ((r->nflags.src_addr == ONEADDR) && (r->parmean.samean == MULTI_DIFFERENT))
{
 match = 1; /* suppose packet->iphead.saddr is different from any element of the
list */
 for(i = 0; i < MAXMULTILEN && r->ip.ipsrc[i] != 0; i++)
 {
 if(r->ip.ipsrc[i] == iph->saddr) /* one element of the list matches */
 return -1; /* then if one element matches source address, we must leave
*/
 }
}
/* destination address */
if (r->nflags.dst_addr == MYADDR)
{
 if (get_dev_ifaddr(&p_dest_address, direction, in, out) < 0)
 {
 return -1;
 }
 dest_address = p_dest_address;
}
else
 dest_address = r->ip.ipdst[0];

if ((r->nflags.dst_addr > 0) && (r->parmean.damean == SINGLE))
{
 if (dest_address != iph->daddr)
 return -1;
 else
 match = 1;
}
else if ((r->nflags.dst_addr == ONEADDR) && (r->parmean.damean == INTERVAL))
{
 if ((ntohl(r->ip.ipdst[0]) <= ntohl(iph->daddr)) && (ntohl(r->ip.ipdst[1]) >=
ntohl(iph->daddr)))
 match = 1;
 else
 return -1;
}
else if ((r->nflags.dst_addr > 0) && (r->parmean.damean == DIFFERENT_FROM))
{
 if (dest_address != iph->daddr)
 match = 1;
 else
 return -1;
}
else if ((r->nflags.dst_addr == ONEADDR) && (r->parmean.damean ==
INTERVAL_DIFFERENT_FROM))

```

```

{
 if ((ntohl(r->ip.ipdst[0]) <= ntohl(iph->daddr)) && (ntohl(r->ip.ipdst[1]) >=
ntohl(iph->daddr)))
 return -1;
 else
 match = 1;
}
/* list of IP addresses */
else if ((r->nflags.dst_addr == ONEADDR) && (r->parmean.damean == MULTI))
{
 /* re initialize addr_in list before parsing destination address */
 addr_in_list = 0;
 for(i = 0; i < MAXMULTILEN && r->ip.ipdst[i] != 0; i++)
 {
 if(r->ip.ipdst[i] == iph->daddr)
 {
 addr_in_list = 1;
 break; /* no need to go further on */
 }
 }
 if(addr_in_list == 1)
 match = 1;
 else
 return -1;
}
else if ((r->nflags.dst_addr == ONEADDR) && (r->parmean.damean == MULTI_DIFFERENT))
{
 match = 1; /* suppose packet->iphead.daddr is different from any element of the
list */
 for(i = 0; i < MAXMULTILEN && r->ip.ipdst[i] != 0; i++)
 {
 if(r->ip.ipdst[i] == iph->daddr) /* one element of the list matches */
 return -1; /* then if one element matches destination address, we must
leave with -1 */
 }
}
return match;
}

int port_match(const struct tcphdr *tcph,
 const struct udphdr *udph,
 const ipfire_rule * r,
 short protocol)
{
 int match = 0, sport_found = 0, dport_found = 0;
 int i;
 u16 sport;
 u16 dport;
 switch (protocol)
 {
 case IPPROTO_TCP:
 sport = tcph->source;
 dport = tcph->dest;
 break;
 case IPPROTO_UDP:
 sport = udph->source;
 dport = udph->dest;
 break;
 default:
 IPFI_PRINTK("IPFIRE: port_match(): invalid protocol %d!\n",
 protocol);
 }
}

```

```

 return -1;
 break;
 }
 /* source port */
 /* a single port is given: it must match exactly */
 if ((r->nflags.src_port) && (r->parmean.spmean == SINGLE))
 {
 if (r->tp.sport[0] != sport)
 return -1;
 else
 match = 1;
 }
 /* an interval is given: src port in packet must be contained in it */
 else if ((r->nflags.src_port) && (r->parmean.spmean == INTERVAL))
 {
 if ((ntohs(r->tp.sport[0]) <= ntohs(sport)) &&
 (ntohs(r->tp.sport[1]) >= ntohs(sport)))
 match = 1;
 else
 return -1;
 }
 /* port in packet must be different from port in rule */
 else if ((r->nflags.src_port) && (r->parmean.spmean == DIFFERENT_FROM))
 {
 if (r->tp.sport[0] != sport)
 match = 1;
 else
 return -1;
 }
 /* port must not be inside rule interval */
 else if ((r->nflags.src_port) && (r->parmean.spmean == INTERVAL_DIFFERENT_FROM))
 {
 if ((ntohs(r->tp.sport[0]) <= ntohs(sport))
 && (ntohs(r->tp.sport[1]) >= ntohs(sport)))
 return -1;
 else
 match = 1;
 }
 /* a list of ports to check */
 else if ((r->nflags.src_port) && (r->parmean.spmean == MULTI))
 {
 for(i = 0; i < MAXMULTILEN; i++)
 {
 /* if one element is zero, leave */
 if(r->tp.sport[i] == 0)
 break;
 if(ntohs(r->tp.sport[i]) == ntohs(sport)) /* one element matches */
 {
 sport_found = 1;
 break;
 }
 }
 if(sport_found == 1) /* found a matching port */
 match = 1;
 else
 return -1;
 }
 else if ((r->nflags.src_port) && (r->parmean.spmean == MULTI_DIFFERENT))
 {
 for(i = 0; i < MAXMULTILEN; i++)
 {

```

```

 /* if one element is zero, leave */
 if(r->tp.sport[i] == 0)
 break;
 if(ntohs(r->tp.sport[i]) == ntohs(sport)) /* one element matches, the check
fails */
 return -1;
 }
 /* We have left the cycle without returning, so the match is positive */
 match = 1;
}
/* destination port */
if ((r->nflags.dst_port) && (r->parmean.dpmean == SINGLE))
{
 if (r->tp.dport[0] != dport)
 return -1;
 else
 match = 1;
}
else if ((r->nflags.dst_port) && (r->parmean.dpmean == INTERVAL))
{
 if ((ntohs(r->tp.dport[0]) <= ntohs(dport)) &&
 (ntohs(r->tp.dport[1]) >= ntohs(dport)))
 match = 1;
 else
 return -1;
}
else if ((r->nflags.dst_port) && (r->parmean.dpmean == DIFFERENT_FROM))
{
 if (r->tp.dport[0] != dport)
 match = 1;
 else
 return -1;
}
else if ((r->nflags.dst_port) && (r->parmean.dpmean == INTERVAL_DIFFERENT_FROM))
{
 if ((ntohs(r->tp.dport[0]) <= ntohs(dport))
 && (ntohs(r->tp.dport[1]) >= ntohs(dport)))
 return -1;
 else
 match = 1;
}
/* a list of (destination) ports to check */
else if ((r->nflags.dst_port) && (r->parmean.dpmean == MULTI))
{
 for(i = 0; i < MAXMULTILEN; i++)
 {
 /* if one element is zero, leave */
 if(r->tp.dport[i] == 0)
 break;
 if(ntohs(r->tp.dport[i]) == ntohs(dport)) /* one element matches */
 {
 dport_found = 1;
 break;
 }
 }
 if(dport_found == 1) /* found a matching port */
 match = 1;
 else
 return -1;
}
else if ((r->nflags.dst_port) && (r->parmean.dpmean == MULTI_DIFFERENT))

```

```

{
 for(i = 0; i < MAXMULTILEN; i++)
 {
 /* if one element is zero, leave */
 if(r->tp.dport[i] == 0)
 break;
 if(ntohs(r->tp.dport[i]) == ntohs(dport)) /* one element matches, the check
fails */
 return -1;
 }
 /* We have left the cycle without returning, so the match is positive */
 match = 1;
}

return match;
}

int ip_layer_filter(const struct iphdr *iph, const ipfire_rule* r, int direction,
 const struct net_device *in, const struct net_device *out)
{
 int match = 0;
 if ((match = address_match(iph, r, direction, in, out)) < 0) {
 return -1;
 }
 if (iph && r->nflags.proto) { // filter by proto
 if (r->ip.protocol != iph->protocol)
 return -1;
 else
 match = 1;
 }
 if (r->nflags.tot_len)
 {
 if (r->ip.total_length != iph->tot_len)
 return -1;
 else
 match = 1;
 }
 if (r->nflags.tos)
 {
 if (r->ip.tos != iph->tos)
 return -1;
 else
 match = 1;
 }
 return match;
}

int ipfi_tcp_filter(const struct tcphdr *tcph, const ipfire_rule * r)
{
 int match = 0;
 /* check tcp specific fields */
 if ((match = port_match(tcph, NULL, r, IPPROTO_TCP)) < 0)
 {
 return -1;
 }
 if (r->nflags.fin)
 {
 if (r->tp.fin != tcph->fin)
 return -1;
 else
 match = 1;
 }

```

```

}
if (r->nflags.syn)
{
 if (r->tp.syn != tcph->syn)
 return -1;
 else
 match = 1;
}
if (r->nflags.rst)
{
 if (r->tp.rst != tcph->rst)
 return -1;
 else
 match = 1;
}
if (r->nflags.psh)
{
 if (r->tp.psh != tcph->psh)
 return -1;
 else
 match = 1;
}
if (r->nflags.ack)
{
 if (r->tp.ack != tcph->ack)
 return -1;
 else
 match = 1;
}
if (r->nflags.urg)
{
 if (r->tp.urg != tcph->urg)
 return -1;
 else
 match = 1;
}
return match;
}

int udp_filter(const struct udphdr *udph, const ipfire_rule * r)
{
 int match = 0;
 /* check tcp specific fields */
 if ((match = port_match(NULL, udph, r, IPPROTO_UDP)) < 0)
 return -1;
 return match;
}

int icmp_filter(const struct icmphdr * icmph, const ipfire_rule * r)
{
 int match = 0;
 /* icmp specific fields */
 if (r->nflags.icmp_type)
 {
 if (r->icmp_p.type != icmph->type)
 return -1;
 else
 match = 1;
 }
 if (r->nflags.icmp_code)
 {

```

```

 if (r->icmp_p.code != icmp_h->code)
 return -1;
 else
 match = 1;
 }
 if (r->nflags.icmp_echo_id)
 {
 if (r->icmp_p.echo_id != icmp_h->un.echo.id)
 return -1;
 else
 match = 1;
 }
 if (r->nflags.icmp_echo_seq)
 {
 if (r->icmp_p.echo_seq != icmp_h->un.echo.sequence)
 return -1;
 else
 match = 1;
 }
 /* disabled icmp_frag_mtu. */
 return match;
}

```

## filter/header\_check.h

```

#ifndef IPFI_HEADER_CHECK_H
#define IPFI_HEADER_CHECK_H

/* Prints an error message and always returns -1.
 * It is used to signal on syslog a network IP or TCP or UDP
 * header null and to return the -1 as error code.
 */
int network_header_null(const char *funcname, const char *message);

#endif

```

## filter/header\_check.c

```

#include "filter/header_check.h"
#include <linux/kernel.h>

int network_header_null(const char *funcname, const char *message) {
 printk("IPFIRE: %s: %s\n", funcname, message);
 return -1;
}

```

## filter/defrag.h

```

#ifndef IPFI_DEFRAG_H
#define IPFI_DEFRAG_H

```



```

int ipfi_gather_frags(struct sk_buff *skb, u_int32_t user);

unsigned int ipfi_defrag(void *priv,
 struct sk_buff *skb,
 const struct nf_hook_state *state);

#endif

```

## filter/defrag.c

```

#include <linux/types.h>
#include <linux/ip.h>
#include <linux/netfilter.h>
#include <linux/module.h>
#include <linux/skbuff.h>
#include <net/route.h>
#include <net/ip.h>
/// toglier
#include <linux/tcp.h>

#include "ipfire.h"
#include "filter/defrag.h"

/* Returns new sk_buff, or NULL */
int ipfi_gather_frags(struct sk_buff *skb, u_int32_t user)
{
 int err;

 skb_orphan(skb);

 local_bh_disable();
 /* Process an incoming IP datagram fragment. - net/ipv4/ip_fragment.c */
 err = ip_defrag(&init_net, skb, user);
 local_bh_enable();

 if (!err)
 ip_send_check(ip_hdr(skb)); /* Generate a checksum for an outgoing IP datagram.
(net/ipv4/ip_output.c) */

 return err;
}

unsigned int ipfi_defrag(void *priv,
 struct sk_buff *skb,
 const struct nf_hook_state *state)
{
 unsigned int hooknum = state->hook;
 /* Gather fragments. */
 /* Fragment Offset: 13-bits used to identify where each of the fragments belong at
the time of reassembly.
 * IP_MF = 0x2000, IP_OFFSET = 0x1FFF -> ored make 13 bits.
 * NF_STOLEN means don't continue to process the packet and not deallocate it.
 * NF_DROP, equivalent to IPFI_ACCEPT, means don't continue to process the packet,
but deallocate it.
 */
 if (ip_hdr(skb)->frag_off & htons(IP_MF | IP_OFFSET))
 {

```

```

 /* include/net/ip.h defines IP_DEFRAG_CONNTRACK_IN and
 IP_DEFRAG_CONNTRACK_OUT: might suit */
 if (ipfi_gather_frags(skb,
 hooknum == NF_INET_PRE_ROUTING ? IP_DEFRAG_CONNTRACK_IN :
 IP_DEFRAG_CONNTRACK_OUT))
 {
 return NF_STOLEN;
 }
 return NF_ACCEPT;
 }
}

```

## nat/nat.h

```

#ifndef IPFI_NAT_H
#define IPFI_NAT_H

#include <common/ipfi_structures.h>
#include <linux/refcount.h>
#include <linux/slab.h>
#include <linux/timer.h>
#include <linux/workqueue.h>

#define SO_IPFI_GETORIG_DST 200 /* a number */

#define SNAT_ENTRY 0
#define DNAT_ENTRY 1

#define DNAT_HASH_BITS 10
#define SNAT_HASH_BITS 10

typedef struct {
 __u32 saddr, daddr;
 __u16 sport, dport;
 /* 0 is not valid */
 short int valid;
} net_quadruplet;

#define IPFI_NAT_REMOVED 0

struct pkt_manip_info {
 __u8 sa : 1, sp : 1, da : 1, dp : 1,
 direction : 4; /* 4 bits as in ipfire_rule */
};

extern struct workqueue_struct *ipfire_wq;

/* Common functions */
int init_translation(void);
void fini_translation(void);

net_quadruplet get_quad_from_skb(const struct sk_buff *skb);

/* given source and destination addresses and ports, this function sets them
 * in socket buffer passed as parameter. In case of TCP or UDP protocols,
 * returns 1, 0 for ICMP, -1 in case of errors
 */
int manip_skb(struct sk_buff *skb, __u32 saddr, __u16 sport, __u32 daddr,

```

```

 __u16 dport, struct pkt_manip_info pf);

/* checks ip and transport checksums, returning 0 if correct,
 * a negative value picked from enum checksum_errors (ipfi.h) otherwise */
int check_checksums(const struct sk_buff *skb);

/* prints the checksum error message according to the checksum_error enum
 * defined in ipfi.h and returns -1. Introduced in 0.99.2
 */
int csum_error_message(const char *origin, int enum_code);

/* skb contains the fields taken from sk_buff, r is the translation rule.
 * In this function, the skb must match the rule provided by user for
 * DNAT or SNAT (or MASQUERADE). 1 is returned on success
 */
int translation_rule_match(const struct sk_buff *skb, const ipfi_flow *flow,
 const struct info_flags *flags,
 const ipfire_rule *r);

/* returns 1 if network address is a private one conforming
 * to rfc 1918, 0 otherwise */
inline int private_address(__u32 addr);

/* returns 1 if packet comes from a public host and
 * gets redirected to an internal host. */
int public_to_private_address(const struct sk_buff *skb,
 const ipfire_rule *transrule);

/* copies address in *address, looking for devices with name
 * equal to device name in skb. Returns 1 or -1 in case of failure */
__u32 get_ifaddr(const struct sk_buff *skb, const struct net_device *dev);

#endif /* IPFI_NAT_H */

```

## nat/nat\_engine.c

```

/* nat/nat_engine.c: NAT engine entry point for ipfire-wall */

#include "../filter/state/state_machine.h"
#include "globals.h"
#include "ipfi_machine.h"
#include "ipfire.h"
#include "nat.h"
#include "nat_table.h"
#include "netlink/ipfi_netl.h"
#include <linux/hashtable.h>
#include <linux/icmp.h>
#include <linux/ip.h>
#include <linux/jhash.h>
#include <linux/module.h>
#include <linux/netfilter_ipv4.h>
#include <linux/skbuff.h>
#include <linux/tcp.h>
#include <linux/udp.h>
#include <net/tcp.h>
#include <net/udp.h>

int translation_rule_match(const struct sk_buff *skb, const ipfi_flow *flow,

```

```

 const struct info_flags *flags,
 const ipfire_rule *r) {
struct iphdr *iph = ip_hdr(skb);
if (r->direction != flags->direction) {
 return -1;
}

if (r->ip.protocol != iph->protocol) {
 return -1;
}

if (r->nflags.indev &&
 (flow->in == NULL ||
 strncmp(r->devpar.in_devname, flow->in->name, IFNAMSIZ) != 0)) {
 return -1;
}
if (r->nflags.outdev &&
 (flow->out == NULL ||
 strncmp(r->devpar.out_devname, flow->out->name, IFNAMSIZ) != 0)) {
 return -1;
}

if (address_match(iph, r, flags->direction, flow->in, flow->out) < 0) {
 return -1;
}

switch (iph->protocol) {
case IPPROTO_TCP: {
 struct tcphdr *th = (struct tcphdr *)((void *)iph + iph->ihl * 4);
 if (port_match(th, NULL, r, IPPROTO_TCP) < 0) {
 return -1;
 }
 break;
}
case IPPROTO_UDP: {
 struct udphdr *uh = (struct udphdr *)((void *)iph + iph->ihl * 4);
 if (port_match(NULL, uh, r, IPPROTO_UDP) < 0) {
 return -1;
 }
 break;
}
}
return 1;
}

int manip_skb(struct sk_buff *skb, __u32 saddr, __u16 sport, __u32 daddr,
 __u16 dport, struct pkt_manip_info mi) {

 struct iphdr *ipheader;
 unsigned int l4hdroff;
 struct tcphdr *ptcphead = NULL;
 struct udphdr *pudphead = NULL;
 unsigned int writable_len = 0;
 __u32 oldaddr = 0, newaddr = 0;
 __u16 oldport = 0, newport = 0;

 /*
 * Checksum verification is NOT done here.
 * Callers (dnat_translation, etc.) already call check_checksums()
 * before invoking manip_skb, so verifying again would be double work.
 */

```

```

if (!skb)
 return -1;
ipheader = ip_hdr(skb);
if (ipheader == NULL)
 return -1;
l4hdroff = ip_hdrlen(skb);

if ((mi.sa || mi.da || mi.sp || mi.dp) && ipheader->protocol == IPPROTO_TCP)
 writable_len = l4hdroff + sizeof(struct tcp_hdr);
else if ((mi.sa || mi.da || mi.sp || mi.dp) &&
 ipheader->protocol == IPPROTO_UDP)
 writable_len = l4hdroff + sizeof(struct udphdr);
else
 writable_len = l4hdroff;

if (skb_ensure_writable(skb, writable_len))
 return -1;

/* skb_ensure_writable may reallocate: refresh pointers.
 * No need for pskb_may_pull - skb_ensure_writable already linearised
 * and made writable up to writable_len which covers L4 headers. */
ipheader = ip_hdr(skb);
if (ipheader == NULL)
 return -1;
l4hdroff = ip_hdrlen(skb);

if (ipheader->protocol == IPPROTO_TCP)
 ptcphdr = tcp_hdr(skb);
else if (ipheader->protocol == IPPROTO_UDP)
 pudphdr = udp_hdr(skb);

if (mi.sa) {
 oldaddr = ipheader->saddr;
 newaddr = saddr;
 ipheader->saddr = saddr;
} else if (mi.da) {
 oldaddr = ipheader->daddr;
 newaddr = daddr;
 ipheader->daddr = daddr;
}

if (mi.sp || mi.dp) {
 switch (ipheader->protocol) {
 case IPPROTO_TCP:
 if (mi.sp && ptcphdr) {
 oldport = ptcphdr->source;
 newport = sport;
 ptcphdr->source = newport;
 } else if (mi.dp && ptcphdr) {
 oldport = ptcphdr->dest;
 newport = dport;
 ptcphdr->dest = newport;
 }
 break;
 case IPPROTO_UDP:
 if (mi.sp && pudphdr) {
 oldport = pudphdr->source;
 newport = sport;
 pudphdr->source = newport;
 } else if (mi.dp && pudphdr) {

```

```

 oldport = pudphd->dest;
 newport = dport;
 pudphd->dest = newport;
 }
 break;
}
}

if (mi.sa || mi.da) {
 csum_replace4(&iphd->check, oldaddr, newaddr);
 switch (iphd->protocol) {
 case IPPROTO_TCP:
 inet_proto_csum_replace4(&ptcpd->check, skb, oldaddr, newaddr, 1);
 break;
 case IPPROTO_UDP:
 if (pudphd->check || skb->ip_summed == CHECKSUM_PARTIAL) {
 inet_proto_csum_replace4(&pudphd->check, skb, oldaddr, newaddr, 1);
 if (!pudphd->check)
 pudphd->check = CSUM_MANGLED_0;
 }
 break;
 }
}

if (mi.sp || mi.dp) {
 switch (iphd->protocol) {
 case IPPROTO_UDP:
 if (pudphd->check || skb->ip_summed == CHECKSUM_PARTIAL) {
 inet_proto_csum_replace2(&pudphd->check, skb, oldport, newport, 0);
 if (!pudphd->check)
 pudphd->check = CSUM_MANGLED_0;
 }
 break;
 case IPPROTO_TCP:
 inet_proto_csum_replace2(&ptcpd->check, skb, oldport, newport, 0);
 break;
 }
}

return 1;
}

net_quadruplet get_quad_from_skb(const struct sk_buff *skb) {
 struct iphdr *iphdr;
 net_quadruplet netquad;
 memset(&netquad, 0, sizeof(net_quadruplet));
 if (skb == NULL)
 return netquad;
 iphdr = ip_hdr(skb);
 if (iphdr == NULL)
 return netquad;
 netquad.saddr = iphdr->saddr;
 netquad.daddr = iphdr->daddr;
 netquad.valid = 1;
 switch (iphdr->protocol) {
 case IPPROTO_TCP: {
 struct tcphdr *th = (struct tcphdr *)((void *)iphdr + iphdr->ihl * 4);
 netquad.sport = th->source;
 netquad.dport = th->dest;
 break;
 }
 case IPPROTO_UDP: {

```

```

 struct udphdr *uh = (struct udphdr *)((void *)iphdr + iphdr->ihl * 4);
 netquad.sport = uh->source;
 netquad.dport = uh->dest;
 break;
}
case IPPROTO_ICMP: {
 netquad.sport = 0;
 netquad.dport = 0;
 break;
}
}
return netquad;
}

int get_original_dest(struct sock *sk, int optval, void __user *user,
 int *len) {
 struct inet_sock *inet;
 struct sockaddr_in *sin;
 if (strcmp(sk->sk_prot->name, "TCP"))
 return -ENOPROTOOPT;
 if ((unsigned int)*len < sizeof(struct sockaddr_in))
 return -EINVAL;
 sin = kmalloc(sizeof(struct sockaddr_in), GFP_ATOMIC);
 if (!sin)
 return -ENOMEM;
 inet = (struct inet_sock *)sk;
 net_quadruplet n4 = {inet->inet_rcv_saddr, inet->inet_daddr, inet->inet_sport,
 inet->inet_dport, 1};
 if (lookup_dnat_table_and_getorigdst(&n4, sin) < 0) {
 kfree(sin);
 return -ENOENT;
 }
 sin->sin_family = AF_INET;
 if (copy_to_user(user, sin, sizeof(struct sockaddr_in)) != 0) {
 kfree(sin);
 return -EFAULT;
 }
 kfree(sin);
 return 0;
}

int get_orig_from_dnat_entry(const struct nat_table *dnt,
 const net_quadruplet *n4,
 struct sockaddr_in *sin) {
 if (dnt->protocol != IPPROTO_TCP)
 return -1;
 if ((dnt->old_saddr == n4->daddr) && (dnt->new_addr == n4->saddr) &&
 (dnt->old_sport == n4->dport) && (dnt->new_port == n4->sport)) {
 sin->sin_addr.s_addr = dnt->old_daddr;
 sin->sin_port = dnt->old_dport;
 return 1;
 }
 return -1;
}

int lookup_dnat_table_and_getorigdst(const net_quadruplet *n4,
 struct sockaddr_in *sin) {
 struct nat_table *dntmp;
 rcu_read_lock_bh();
 {
 unsigned int bkt;

```

```

 u32 key = get_nat_tuple_hash(n4->saddr, n4->sport, n4->daddr, n4->dport,
 IPPROTO_TCP);
 bkt = key & ((1 << NAT_HASH_BITS) - 1);

 hlist_for_each_entry_rcu(dntmp,
 &nat_hashtables[NAT_DNAT][NAT_IDX_REPLY][bkt],
 h_indices[NAT_IDX_REPLY - 1]) {
 if (get_orig_from_dnat_entry(dntmp, n4, sin) == 1) {
 rcu_read_unlock_bh();
 return 0;
 }
 }
 rcu_read_unlock_bh();
 return -1;
}

inline int private_address(__u32 addr) {
 __u32 haddr = ntohl(addr);
 if ((haddr >= 0x0a000000) && (haddr <= 0x0affffff))
 return 1;
 if ((haddr >= 0xac100000) && (haddr <= 0xac1ffffff))
 return 1;
 if ((haddr >= 0xc0a80000) && (haddr <= 0xc0a8ffff))
 return 1;
 return 0;
}

inline int public_to_private_address(const struct sk_buff *skb,
 const ipfire_rule *ipfr) {
 struct iphdr *iphead = ip_hdr(skb);
 if (iphead == NULL)
 return -1;
 if (private_address(iphead->saddr))
 return 0;
 if (private_address(ipfr->newaddr))
 return 1;
 return 0;
}

int csum_error_message(const char *origin, int enum_code) {
 IPFI_PRINTK("IPFIRE: %s: checksum error: %d\n", origin, enum_code);
 return BAD_CHECKSUM;
}

int check_checksums(const struct sk_buff *skb) {
 int datalen;
 struct iphdr *iph = ip_hdr(skb);
 if (iph == NULL)
 return -1;
 /* Skip verification for packets with offloaded checksums (e.g. veth,
 * xen-vif). In virtual environments, the stack often avoids calculating
 * checksums until the packet leaves the physical interface. Verifying them
 * here would fail and drop legitimate packets. */
 if (skb->ip_summed == CHECKSUM_PARTIAL)
 return 0;
 /* If the NIC/driver already verified the checksum, trust it.
 * CHECKSUM_UNNECESSARY means hardware confirmed correctness. */
 if (skb->ip_summed == CHECKSUM_UNNECESSARY)
 return 0;
 datalen = skb->len - iph->ihl * 4;

```



```

if (ip_fast_csum((u8 *)iph, iph->ihl) != 0)
 return -BAD_IP_CSUM;
switch (iph->protocol) {
case IPPROTO_TCP: {
 if (csum_tcpudp_magic(iph->saddr, iph->daddr, datalen, IPPROTO_TCP,
 skb->ip_summed == CHECKSUM_COMPLETE
 ? skb->csum
 : skb_checksum(skb, iph->ihl * 4, datalen, 0)))
 return -BAD_TCPHEAD_CSUM;
 break;
}
case IPPROTO_UDP: {
 struct udphdr *uh = (struct udphdr *)((void *)iph + iph->ihl * 4);
 if (!uh->check)
 return 0;
 if (csum_tcpudp_magic(iph->saddr, iph->daddr, datalen, IPPROTO_UDP,
 skb->ip_summed == CHECKSUM_COMPLETE
 ? skb->csum
 : skb_checksum(skb, iph->ihl * 4, datalen, 0)))
 return -BAD_UDPHEAD_CSUM;
 break;
}
}
return 0;
}

static struct nf_sockopt_ops so_getoriginal_dst = {
 .pf = PF_INET,
 .get_optmin = SO_IPFI_GETORIG_DST,
 .get_optmax = SO_IPFI_GETORIG_DST + 1,
 .get = &get_original_dest,
};

int init_translation(void) {
 init_nat_tables();
 return nf_register_sockopt(&so_getoriginal_dst);
}

void fini_translation(void) {
 fini_nat_tables();
 might_sleep();
 nf_unregister_sockopt(&so_getoriginal_dst);
}

```

## nat/nat\_table.h

```

/* nat/nat_table.h - Unified NAT table structure
 *
 * Replaces the separate snatted_table and dnatted_table structs.
 * Every NAT entry (SNAT or DNAT) is represented as a struct nat_table
 * with a type flag.
 */
#ifndef IPFI_NAT_TABLE_H
#define IPFI_NAT_TABLE_H

#include "../ipfi_entry.h"
#include "nat.h"
#include <common/ipfi_structures.h>

```

```

#include <linux/netdevice.h>
#include <linux/percpu_counter.h>

enum nat_type { NAT_SNAT = 0, NAT_DNAT = 1 };
enum nat_index {
 NAT_IDX_ORIG = 0,
 NAT_IDX_POSTNAT = 1,
 NAT_IDX_REPLY = 2,
 NAT_IDX_COUNT = 3
};

struct nat_table {
 struct ipfi_entry_head h; /* MUST be first. h->hnode is ORIG index. */
 struct hlist_node h_indices[NAT_IDX_COUNT - 1]; /* POSTNAT and REPLY */

 /* Original packet fields */
 __u32 old_saddr, old_daddr;
 __u16 old_sport, old_dport;

 /* Translated fields:
 * SNAT: new_addr = new source addr, new_port = new source port
 * DNAT: new_addr = new dest addr, new_port = new dest port
 */
 __u32 new_addr;
 __u16 new_port;

 __u32 our_ifaddr; /* DNAT dynamic SNAT reverse, 0 for SNAT */

 __u8 direction : 4, external : 3, nolog : 1;
 __u8 protocol;
 __u8 state;
 enum nat_type type;

 uint32_t rule_id;
 unsigned int position;

 u32 keys[NAT_IDX_COUNT];
 unsigned int bkts[NAT_IDX_COUNT];
 u8 active_indices; /* bitmask of enum nat_index */

 int in_ifindex, out_ifindex;
 char in_devname[IFNAMSIZ];
 char out_devname[IFNAMSIZ];
 char rulename[RULENAMELEN]; /* DNAT only, empty for SNAT */
};

/* ---- Inline hold/put delegating to lifecycle ---- */

static inline void nat_hold(struct nat_table *nt) { ipfi_entry_hold(&nt->h); }

static inline bool nat_hold_rcu(struct nat_table *nt) {
 return ipfi_entry_hold_rcu(&nt->h);
}

static inline void nat_put(struct nat_table *nt) { ipfi_entry_put(&nt->h); }

/* ---- NAT table functions (nat_table.c) ---- */

/* Hash functions */
u32 get_snat_hash(__u32 new_saddr, __u16 new_sport, __u32 old_daddr,
 __u16 old_dport, __u8 proto);

```

```

u32 get_dnat_hash(__u32 old_saddr, __u16 old_sport, __u32 new_daddr,
 __u16 new_dport, __u8 proto);
u32 get_nat_tuple_hash(__u32 saddr, __u16 sport, __u32 daddr, __u16 dport,
 __u8 proto);

/* Lookup:
 * All lookup functions search RCU-protected hash tables.
 *
 * lookup_nat_idx:
 * - Internally handles rcu_read_lock_bh().
 * - Returns entry with a reference hold (nat_hold_rcu).
 * - Caller MUST call nat_put() when finished.
 */
struct nat_table *lookup_nat_idx(enum nat_type type, enum nat_index idx,
 const struct sk_buff *skb);

/* lookup_nat_forward:
 * - Internally handles rcu_read_lock_bh().
 * - Wraps lookup_nat_idx(NAT_IDX_ORIG).
 * - Updates timer and state internally.
 * - Returns entry with a reference hold.
 */
struct nat_table *lookup_nat_forward(const struct sk_buff *skb,
 enum nat_type type);

int fill_nat_entry_fields(struct nat_table *entry, const struct sk_buff *skb,
 const ipfi_flow *flow, const struct response *resp,
 const struct info_flags *flags,
 const ipfire_rule *rule, enum nat_type type);

/* Supplemental index management */
void nat_add_index(struct nat_table *nt, enum nat_index idx, __u32 key);

/* Compare */
int compare_nat_entries(const struct nat_table *a, const struct nat_table *b);

/* Timer callback */
void handle_nat_entry_timeout(struct timer_list *t);

/* Flush */
int free_nat_tables(enum nat_type type);

/* Init/Fini */
int init_nat_tables(void);
void fini_nat_tables(void);

/* Access to per-type globals – implemented as arrays indexed by nat_type.
 * In hash mode, entries are stored in nat_hashtables and nat_counters.
 * In list mode, entries are stored in nat_lists and nat_counters.
 * Locks and counters are always present.
 */
#ifdef NAT_HASH_BITS
#define NAT_HASH_BITS 8
#endif

extern spinlock_t nat_bucket_locks[2][NAT_IDX_COUNT][1 << NAT_HASH_BITS];
extern struct percpu_counter nat_counters[2];
extern struct kmem_cache *nat_cache;
extern struct hlist_head nat_hashtables[2][NAT_IDX_COUNT][1 << NAT_HASH_BITS];

```

```
#endif /* IPFI_NAT_TABLE_H */
```

## nat/nat\_table.c

```
/* nat/nat_table.c - Unified NAT table management for ipfire-wall
 *
 * Merges the former snat_table.c and dnat_table.c into a single
 * implementation. All SNAT and DNAT entries share the same struct
 * nat_table and lifecycle functions from table_lifecycle.c.
 */

#include "nat_table.h"
#include "../filter/state/state_machine.h"
#include "../globals.h"
#include "ipfi_machine.h"
#include "ipfire.h"
#include "nat.h"
#include <linux/bitops.h>
#include <linux/gfp.h>
#include <linux/hashtable.h>
#include <linux/jhash.h>
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/timer.h>

/* ---- Per-type globals ---- */

/* Hash tables: two arrays of hlist_head, one per nat_type */
/* (defined in common/globals.c via globals.h extern) */
/* (defined in common/globals.c via globals.h extern) */
extern struct percpu_counter nat_counters[2];
extern struct kmem_cache *nat_cache;

void fini_nat_tables(void) {
 int s, d;
 s = free_nat_tables(NAT_SNAT);
 d = free_nat_tables(NAT_DNAT);
 percpu_counter_destroy(&nat_counters[NAT_SNAT]);
 percpu_counter_destroy(&nat_counters[NAT_DNAT]);
 IPFI_PRINTK("IPFIRE: NAT tables freed: snat=%d dnat=%d\n", s, d);
 /* kmem_cache_destroy deferred to ipfire.c::fini() after
 * destroy_workqueue + rcu_barrier ensure all kfree callbacks ran. */
}

int init_nat_tables(void) {
 nat_cache = kmem_cache_create("ipfi_nat", sizeof(struct nat_table), 0,
 SLAB_HWCACHE_ALIGN, NULL);

 if (!nat_cache) {
 IPFI_PRINTK("IPFIRE: failed to create NAT slab cache\n");
 return -ENOMEM;
 }
 {
 unsigned int i, j;
 for (i = 0; i < (1 << NAT_HASH_BITS); i++) {
 for (j = 0; j < NAT_IDX_COUNT; j++) {
 INIT_HLIST_HEAD(&nat_hashtables[NAT_SNAT][j][i]);
 INIT_HLIST_HEAD(&nat_hashtables[NAT_DNAT][j][i]);
 }
 }
 }
}
```

```

 spin_lock_init(&nat_bucket_locks[NAT_SNAT][j][i]);
 spin_lock_init(&nat_bucket_locks[NAT_DNAT][j][i]);
 }
}
}
percpu_counter_init(&nat_counters[NAT_SNAT], 0, GFP_KERNEL);
percpu_counter_init(&nat_counters[NAT_DNAT], 0, GFP_KERNEL);
return 0;
}

/* ---- Hash functions ---- */

u32 get_snat_hash(__u32 new_saddr, __u16 new_sport, __u32 old_daddr,
 __u16 old_dport, __u8 proto) {
 __u32 a1 = new_saddr, a2 = old_daddr;
 __u16 p1 = new_sport, p2 = old_dport;
 if (a1 > a2 || (a1 == a2 && p1 > p2)) {
 swap(a1, a2);
 swap(p1, p2);
 }
 if (proto == IPPROTO_ICMP)
 return jhash_3words(a1, a2, 0, proto);
 return jhash_3words(a1, a2, ((u32)p1 << 16) | p2, proto);
}

u32 get_nat_tuple_hash(__u32 saddr, __u16 sport, __u32 daddr, __u16 dport,
 __u8 proto) {
 __u32 a1 = saddr, a2 = daddr;
 __u16 p1 = sport, p2 = dport;
 if (a1 > a2 || (a1 == a2 && p1 > p2)) {
 swap(a1, a2);
 swap(p1, p2);
 }
 if (proto == IPPROTO_ICMP)
 return jhash_3words(a1, a2, 0, proto);
 return jhash_3words(a1, a2, ((u32)p1 << 16) | p2, proto);
}

u32 get_dnat_hash(__u32 old_saddr, __u16 old_sport, __u32 new_daddr,
 __u16 new_dport, __u8 proto) {
 return get_nat_tuple_hash(old_saddr, old_sport, new_daddr, new_dport, proto);
}

/* Supplemental index management */
void nat_add_index(struct nat_table *nt, enum nat_index idx, __u32 key) {
 unsigned int new_bkt, old_bkt;

 if (idx <= NAT_IDX_ORIG || idx >= NAT_IDX_COUNT)
 return;

 new_bkt = key & ((1 << NAT_HASH_BITS) - 1);

 /* Case 1: Entry already has this index. Check if it needs moving. */
 if (nt->active_indices & (1 << idx)) {
 if (nt->keys[idx] == key)
 return; /* Bucket same, nothing to do (idempotency) */

 /* Identity changed (e.g. masquerading updated our_ifaddr).
 * Move entry to the new bucket. */
 old_bkt = nt->bkets[idx];

```

```

/* Unlink from old bucket */
spin_lock_bh(&nat_bucket_locks[nt->type][idx][old_bkt]);
hlist_del_rcu(&nt->h_indices[idx - 1]);
nt->active_indices &= ~(1 << idx);
spin_unlock_bh(&nat_bucket_locks[nt->type][idx][old_bkt]);
}

/* Case 2: Add to new bucket (or relocated bucket) */
spin_lock_bh(&nat_bucket_locks[nt->type][idx][new_bkt]);
nt->keys[idx] = key;
nt->bkts[idx] = new_bkt;
nt->active_indices |= (1 << idx);

hlist_add_head_rcu(&nt->h_indices[idx - 1],
 &nat_hashtables[nt->type][idx][new_bkt]);
spin_unlock_bh(&nat_bucket_locks[nt->type][idx][new_bkt]);
}

/* ---- Compare ---- */

int compare_nat_entries(const struct nat_table *a, const struct nat_table *b) {
 return (a->type == b->type) && (a->protocol == b->protocol) &&
 (a->old_saddr == b->old_saddr) && (a->old_daddr == b->old_daddr) &&
 (a->old_sport == b->old_sport) && (a->old_dport == b->old_dport) &&
 (a->new_addr == b->new_addr) && (a->new_port == b->new_port) &&
 (a->direction == b->direction);
}

/* ---- Fill network fields ---- */

int fill_nat_entry_fields(struct nat_table *entry, const struct sk_buff *skb,
 const ipfi_flow *flow, const struct response *resp,
 const struct info_flags *flags,
 const ipfire_rule *rule, enum nat_type type) {
 struct iphdr *iph = ip_hdr(skb);

 memset(entry, 0, sizeof(struct nat_table));
 entry->type = type;
 entry->protocol = iph->protocol;
 entry->old_saddr = iph->saddr;
 entry->old_daddr = iph->daddr;
 entry->direction = flags->direction;
 entry->nolog = rule->nflags.nolog;
 entry->external = flags->external;
 entry->rule_id = resp->rule_id;
 entry->position = percpu_counter_read(&nat_counters[type]);

 /* Initialize index metadata for NAT_IDX_ORIG */
 entry->active_indices = (1 << NAT_IDX_ORIG);
 entry->keys[NAT_IDX_ORIG] = get_nat_tuple_hash(
 iph->saddr,
 (iph->protocol == IPPROTO_TCP || iph->protocol == IPPROTO_UDP)
 ? ((struct tcphdr *)((void *)iph + iph->ihl * 4))->source
 : 0,
 iph->daddr,
 (iph->protocol == IPPROTO_TCP || iph->protocol == IPPROTO_UDP)
 ? ((struct tcphdr *)((void *)iph + iph->ihl * 4))->dest
 : 0,
 iph->protocol);
 entry->bkts[NAT_IDX_ORIG] =
 entry->keys[NAT_IDX_ORIG] & ((1 << NAT_HASH_BITS) - 1);

```

```

if (flow->in)
 strncpy(entry->in_devname, flow->in->name, IFNAMSIZ);
if (flow->out)
 strncpy(entry->out_devname, flow->out->name, IFNAMSIZ);

if (iph->protocol == IPPROTO_TCP || iph->protocol == IPPROTO_UDP) {
 struct tcphdr *th = (struct tcphdr *)((void *)iph + iph->ihl * 4);
 entry->old_sport = th->source;
 entry->old_dport = th->dest;
} else if (iph->protocol == IPPROTO_ICMP) {
 entry->old_sport = 0;
 entry->old_dport = 0;
}

if (type == NAT_SNAT) {
 /* Default: keep original source */
 entry->new_addr = iph->saddr;
 entry->new_port =
 (iph->protocol == IPPROTO_TCP || iph->protocol == IPPROTO_UDP)
 ? entry->old_sport
 : 0;
 if (rule->nflags.newaddr)
 entry->new_addr = rule->newaddr;
 if (rule->nflags.newport)
 entry->new_port = rule->newport;
} else { /* NAT_DNAT */
 /* Default: keep original dest */
 entry->new_addr = iph->daddr;
 entry->new_port =
 (iph->protocol == IPPROTO_TCP || iph->protocol == IPPROTO_UDP)
 ? entry->old_dport
 : 0;
 if (rule->nflags.newaddr)
 entry->new_addr = rule->newaddr;
 if (rule->nflags.newport)
 entry->new_port = rule->newport;
}

return 0;
}

/* ---- Timer callback ---- */

void handle_nat_entry_timeout(struct timer_list *t) {
 struct nat_table *nt = timer_container_of(nt, t, h.timer);
 enum nat_type type = nt->type;
 int i;
 struct {
 unsigned int bkt;
 enum nat_index idx;
 } locks[NAT_IDX_COUNT];
 int lock_count = 0;

 /* Identify active locks and order them to avoid deadlock */
 for (i = 0; i < NAT_IDX_COUNT; i++) {
 if (nt->active_indices & (1 << i)) {
 locks[lock_count].bkt = nt->bkets[i];
 locks[lock_count].idx = i;
 lock_count++;
 }
 }
}

```

```

}

/* Bubble sort by (idx, bkt) for consistent locking order */
for (i = 0; i < lock_count - 1; i++) {
 for (int j = 0; j < lock_count - i - 1; j++) {
 if (locks[j].idx > locks[j + 1].idx ||
 (locks[j].idx == locks[j + 1].idx &&
 locks[j].bkt > locks[j + 1].bkt)) {
 swap(locks[j], locks[j + 1]);
 }
 }
}

/* Acquire locks */
for (i = 0; i < lock_count; i++) {
 spin_lock_bh(&nat_bucket_locks[type][locks[i].idx][locks[i].bkt]);
}

/* Unlink from all indices */
if (test_and_set_bit(IPFI_ENTRY_REMOVED, &nt->h.status))
 goto unlock;

hlist_del_rcu(&nt->h.hnode); /* ORIG */
for (i = 0; i < NAT_IDX_COUNT - 1; i++) {
 if (nt->active_indices & (1 <= (i + 1))) {
 hlist_del_rcu(&nt->h_indices[i]);
 }
}

percpu_counter_add_batch(&nat_counters[type], -1, 1);
ipfi_entry_put(&nt->h);

unlock:
 for (i = lock_count - 1; i >= 0; i--) {
 spin_unlock_bh(&nat_bucket_locks[type][locks[i].idx][locks[i].bkt]);
 }
}

/* ---- Forward match (for early lookup in existing sessions) ---- */

/* ---- Lookup: by specific index (O(1)) ----
 *
 * This function handles RCU read-side protection internally.
 * It searches the specified index and returns a pointer to the entry
 * with a reference count hold (using nat_hold_rcu).
 *
 * The caller MUST call nat_put() when the entry is no longer needed.
 */
struct nat_table *lookup_nat_idx(enum nat_type type, enum nat_index idx,
 const struct sk_buff *skb) {
 struct nat_table *tmp;
 struct iphdr *iph = ip_hdr(skb);
 net_quadruplet nq;
 u32 key;
 unsigned int bkt;

 if (unlikely(!iph))
 return NULL;

 nq = get_quad_from_skb(skb);
 if (unlikely(!nq.valid))

```



```

 return NULL;

key =
 get_nat_tuple_hash(nq.saddr, nq.sport, nq.daddr, nq.dport, iph->protocol);
bkt = key & ((1 << NAT_HASH_BITS) - 1);

rcu_read_lock_bh();
if (idx == NAT_IDX_ORIG) {
 hlist_for_each_entry_rcu(tmp, &nat_hashtables[type][idx][bkt], h.hnode) {
 if (iph->protocol == tmp->protocol) {
 if (nq.saddr == tmp->old_saddr && nq.sport == tmp->old_sport &&
 nq.daddr == tmp->old_daddr && nq.dport == tmp->old_dport) {
 if (nat_hold_rcu(tmp)) {
 rcu_read_unlock_bh();
 return tmp;
 }
 }
 }
 }
} else {
 /* Use the appropriate index in h_indices */
 int h_idx = (int)idx - 1;
 if (h_idx == 0) { /* POSTNAT */
 hlist_for_each_entry_rcu(tmp, &nat_hashtables[type][idx][bkt],
 h_indices[0]) {
 if (iph->protocol == tmp->protocol) {
 if (tmp->type == NAT_DNAT) {
 /* DNAT POSTNAT: A -> C (Destination is new) */
 if (nq.saddr == tmp->old_saddr && nq.sport == tmp->old_sport &&
 nq.daddr == tmp->new_addr && nq.dport == tmp->new_port) {
 if (nat_hold_rcu(tmp)) {
 rcu_read_unlock_bh();
 return tmp;
 }
 }
 }
 /* DNAT POSTNAT Reverse: C -> A (Reply from DNAT target to original
 * source) */
 if (nq.saddr == tmp->new_addr && nq.sport == tmp->new_port &&
 nq.daddr == tmp->old_saddr && nq.dport == tmp->old_sport) {
 if (nat_hold_rcu(tmp)) {
 rcu_read_unlock_bh();
 return tmp;
 }
 }
 }
 }
 } else {
 /* SNAT POSTNAT: C -> B (Source is new) */
 if (nq.saddr == tmp->new_addr && nq.sport == tmp->new_port &&
 nq.daddr == tmp->old_daddr && nq.dport == tmp->old_dport) {
 if (nat_hold_rcu(tmp)) {
 rcu_read_unlock_bh();
 return tmp;
 }
 }
 }
}
} else { /* REPLY */
 hlist_for_each_entry_rcu(tmp, &nat_hashtables[type][idx][bkt],
 h_indices[1]) {
 if (iph->protocol == tmp->protocol) {
 if (tmp->type == NAT_DNAT) {

```

```

/* DNAT REPLY: C -> B (Reply to DNATed destination)
 * Match criteria for packets returning from a DNAT target server
 * (C). Used in:
 * - PREROUTING (pre_de_dnat): Identifies replies to keep state.
 * - POSTROUTING (de_dnat_translation): Restores the original
 * destination address of the forward packet as the source of this
 * reply, so the client (B) sees the gateway's IP.
 */
if (tmp->direction == IPFI_OUTPUT) {
/* OUTPUT DNAT REPLY: C -> A
 * Packet generated locally (A -> B), translated to A -> C.
 * Reply is C -> A. It comes back to the local host's old_saddr.
 */
if (nq.saddr == tmp->new_addr && nq.sport == tmp->new_port &&
 nq.daddr == tmp->old_saddr && nq.dport == tmp->old_sport) {
 if (nat_hold_rcu(tmp)) {
 rcu_read_unlock_bh();
 return tmp;
 }
} else {
if (nq.saddr == tmp->new_addr && nq.sport == tmp->new_port &&
 (nq.daddr == tmp->old_daddr ||
 (tmp->our_ifaddr && nq.daddr == tmp->our_ifaddr))) {
 if (nat_hold_rcu(tmp)) {
 rcu_read_unlock_bh();
 return tmp;
 }
}
}
} else {
/* SNAT REPLY: B -> C (Reply to SNATed source)
 * Match criteria for packets returning to an SNATed source (C).
 * Used in:
 * - PREROUTING (pre_de_snat): The FIRST step in reverse processing.
 * Correctly identifies the reply packet and restores the original
 * client IP as the destination, before any DNAT reverse
 * processing.
 */
if (nq.saddr == tmp->old_daddr && nq.sport == tmp->old_dport &&
 nq.daddr == tmp->new_addr && nq.dport == tmp->new_port) {
 if (nat_hold_rcu(tmp)) {
 rcu_read_unlock_bh();
 return tmp;
 }
}
}
}
}
}
}
}
rcu_read_unlock_bh();
return NULL;
}

/* ---- Forward match (for original direction) ---- */

/* ---- Lookup: by entry comparison (for duplicate detection) ---- */

/* ---- Lookup: by forward 5-tuple (for existing session reuse) ----
 *

```

```

* Wraps lookup_nat_idx for the original direction.
* Internally handles RCU protection and state/timer updates.
*
* Returns entry with reference hold.
*/
struct nat_table *lookup_nat_forward(const struct sk_buff *skb,
 enum nat_type type) {
 struct nat_table *nt = lookup_nat_idx(type, NAT_IDX_ORIG, skb);
 if (nt) {
 nt->state = state_machine(skb, nt->state, 0);
 ipfi_entry_update_timer(&nt->h, nt->protocol, nt->state);
 }
 return nt;
}

/* ---- Flush ---- */

int free_nat_tables(enum nat_type type) {
 struct nat_table *nt;
 struct hlist_node *tmp;
 unsigned int bkt;

 /* Supplemental indices must be cleared first.
 * We don't call ipfi_entry_put here because the final ORIG flush
 * will handle entry destruction.
 */
 for (bkt = 0; bkt < (1 << NAT_HASH_BITS); bkt++) {
 /* 1. Flush POSTNAT index */
 spin_lock_bh(&nat_bucket_locks[type][NAT_IDX_POSTNAT][bkt]);
 hlist_for_each_entry_safe(nt, tmp,
 &nat_hashtables[type][NAT_IDX_POSTNAT][bkt],
 h_indices[NAT_IDX_POSTNAT - 1]) {
 /* Only the thread that marks the entry as REMOVED (either the
 * final ORIG flush or a timer) should handle supplemental unlinking.
 * However, during module unload we clear everything. We check the
 * status to avoid double-deletion if a timer is firing. */
 if (!test_bit(IPFI_ENTRY_REMOVED, &nt->h.status)) {
 hlist_del_rcu(&nt->h_indices[NAT_IDX_POSTNAT - 1]);
 nt->active_indices &= ~(1 << NAT_IDX_POSTNAT);
 }
 }
 spin_unlock_bh(&nat_bucket_locks[type][NAT_IDX_POSTNAT][bkt]);

 /* 2. Flush REPLY index */
 spin_lock_bh(&nat_bucket_locks[type][NAT_IDX_REPLY][bkt]);
 hlist_for_each_entry_safe(nt, tmp,
 &nat_hashtables[type][NAT_IDX_REPLY][bkt],
 h_indices[NAT_IDX_REPLY - 1]) {
 if (!test_bit(IPFI_ENTRY_REMOVED, &nt->h.status)) {
 hlist_del_rcu(&nt->h_indices[NAT_IDX_REPLY - 1]);
 nt->active_indices &= ~(1 << NAT_IDX_REPLY);
 }
 }
 spin_unlock_bh(&nat_bucket_locks[type][NAT_IDX_REPLY][bkt]);
 }

 /* 3. Flush ORIG index (standard traversal of h.hnode) */
 return ipfi_table_flush_hash_bucketed(
 nat_hashtables[type][NAT_IDX_ORIG], 1 << NAT_HASH_BITS,

```

```

nat_bucket_locks[type][NAT_IDX_ORIG], &nat_counters[type]));
}

```

## nat/translation.h

```

/* DEPRECATED: usage of this file is deprecated.
 * Please include nat/nat.h, nat/snat/snat.h or nat/dnat/dnat.h instead.
 * This file will be removed in future.
 */
#ifndef IPFI_TRANSLATION_DEPRECATED_H
#define IPFI_TRANSLATION_DEPRECATED_H
//warning "ipfi_translation.h is deprecated. Use nat/nat.h instead."
#endif

```

## mangle/mangle.h

```

#ifndef IPFI_MANGLE_H
#define IPFI_MANGLE_H

#include "ipfire.h"

/**
 * mangle_skb() - Apply packet manipulations based on firewall rule settings
 * @pm: packet manipulation structure from the firewall rule
 * @skb: socket buffer to modify
 * @flow: flow information (direction, interfaces)
 * @reverse: flag indicating if this is a reverse-direction packet
 *
 * Returns: <0 on error, 0 if no manipulation needed/applied, >0 if manipulation
succeeded
 */
int mangle_skb(const struct packet_manip* pm, struct sk_buff *skb,
 const ipfi_flow *flow, short reverse);

/** @return != 0 if some field is set inside packet_manip *pm, 0 otherwise
 * @param pm pointer to struct packet_manip to evaluate.
 *
 * The function simply memcmps pm with a zero filled struct packet_manip, to see if
 * some field of the structure pm has been set.
 */
int some_manip_enabled(const struct packet_manip *pm);

#endif

```

## mangle/mangle.c

```

#include "mangle/mangle.h"
#include "mangle/tcpmss.h"

/**
 * mangle_skb() - Apply packet manipulations based on firewall rule settings

```

```

* @pm: packet manipulation structure from the firewall rule
* @skb: socket buffer to modify
* @flow: flow information (direction, interfaces)
* @reverse: flag indicating if this is a reverse-direction packet
*
* This function examines the packet_manip structure from a firewall rule
* and applies any enabled manipulations. Currently supports:
* - TCP MSS clamping (for PMTU discovery)
*
* Returns: <0 on error, 0 if no manipulation needed/applied, >0 if manipulation
succeeded
*/
int mangle_skb(const struct packet_manip* pm, struct sk_buff *skb,
 const ipfi_flow *flow, short reverse)
{
 int ret = 0;

 /* TCP Maximum Segment Size manipulation
 * Only applied if:
 * 1. MSS mangling is enabled in the rule (pm->mss.enabled)
 * 2. Packet is suitable (TCP SYN or SYN/ACK)
 */
 if(pm->mss.enabled && packet_suitable_for_mss_change(skb, flow, reverse) > 0)
 ret = tcpmss_mangle_packet(skb, pm->mss.option, pm->mss.mss);

 return ret;
}

int some_manip_enabled(const struct packet_manip *pm)
{
 struct packet_manip zero_manip;
 memset(&zero_manip, 0, sizeof(zero_manip));
 /* memcmp returns 0 if the two memory areas match */
 return memcmp(pm, &zero_manip, sizeof(struct packet_manip));
}

```

## mangle/tcpmss.h

```

#ifndef IPFI_TCPMSS_H
#define IPFI_TCPMSS_H

#include "ipfire.h"
#include <linux/types.h>
#include <linux/skbuff.h>

u_int32_t get_net_mtu(__u32 address);

int tcpmss_mangle_packet(struct sk_buff *skb, short unsigned int option, __u16 mss);

int packet_suitable_for_mss_change(const struct sk_buff *skb, const ipfi_flow *flow,
short reverse);

int optlen(const u_int8_t *opt, unsigned int offset);

#endif

```

## mangle/tcpmss.c

```
#include "mangle/tcpmss.h"
#include <net/tcp.h> /* TCPOLEN_MSS... */
#include <linux/version.h>

/* private manip function */
int __tcpmss_mangle_packet(struct sk_buff *skb, short unsigned int option, __u16 mss);

/* TCPOLEN_MSS: length of the tcp header mss option, from include/net/tcp.h
 * TCPOPT_MSS: segment size negotiating, from include/net/tcp.h
 * TCPOPT_NOP: padding, from include/net/tcp.h
 */

/**
 * packet_suitable_for_mss_change() - Determine if packet is suitable for TCP MSS
manipulation
 * @skb: socket buffer containing the packet
 * @flow: flow information (direction, interfaces)
 * @reverse: flag indicating if this is a reverse-direction packet (reply)
 *
 * TCP MSS (Maximum Segment Size) mangling is needed for Path MTU Discovery (PMTUD).
 * When packets traverse networks with different MTUs (e.g., PPPoE with 1492 vs Ethernet
1500),
 * the firewall may need to clamp MSS to prevent fragmentation issues.
 *
 * MSS mangling is ONLY applicable to TCP SYN packets because:
 * - MSS option is negotiated during connection establishment (SYN, SYN/ACK)
 * - Once negotiated, it cannot be changed mid-connection
 * - Mangling non-SYN packets would break the connection
 *
 * Returns: 1 if packet is suitable for MSS manipulation, 0 otherwise
 */
int packet_suitable_for_mss_change(const struct sk_buff *skb,
 const ipfi_flow *flow, short reverse)
{
 struct iphdr *iph;
 struct tcphdr *tcph;

 /* Extract IP header from socket buffer */
 iph = ip_hdr(skb);
 if (!iph)
 return 0;

 /* MSS mangling only applies to TCP protocol */
 if (iph->protocol != IPPROTO_TCP)
 return 0;

 /* Extract TCP header (located iph->ihl * 4 bytes after IP header) */
 tcph = (struct tcphdr *)((void *)iph + iph->ihl * 4);

 /* CASE 1: Outgoing SYN packet (connection initiation)
 * - SYN=1, ACK=0 indicates initial SYN packet from client
 * - This is where initial MSS is advertised
 * - We may need to clamp it to prevent PMTU issues
 */
 if (tcph->syn == 1 && tcph->ack == 0)
 return 1;
}
```

```

/* CASE 2: Forwarded SYN/ACK packet in reverse direction (server reply)
 * - Direction is FORWARD (packet being routed)
 * - SYN=1, ACK=1 indicates SYN/ACK reply from server
 * - reverse=1 means this is the return packet of an established flow
 * - Server's MSS also needs clamping to match path MTU
 */
if (flow->direction == IPFI_FWD && tcph->syn == 1 &&
 tcph->ack == 1 && reverse)
 return 1;

/* Diagnostic: SYN/ACK in forward but not reverse - shouldn't happen normally
 * This indicates a potential state tracking issue
 */
if (flow->direction == IPFI_FWD && tcph->syn == 1 &&
 tcph->ack == 1 && !reverse)
 IPFI_PRINTK("IPFIRE: SYN+ACK packet not marked as reverse - unexpected!\\n");

/* All other packets (data, ACK, FIN, etc.) are not suitable for MSS mangling */
return 0;
}

int optlen(const u_int8_t *opt, unsigned int offset)
{
 /* Beware zero-length options: make finite progress */
 if (opt[offset] <= TCPOPT_NOP || opt[offset+1] == 0)
 return 1;
 else
 return opt[offset+1];
}

/* code taken from xt_TCPMSS.c, net/netfilter */
u_int32_t get_net_mtu(__u32 address)
{
 /* generic internet flow structure */
 struct rtable *rt;
 u_int32_t mtu = ~0U;
 /* ipv4 only */
 struct flowi4 fl4 = {
 .daddr = address,
 };

 rt = ip_route_output_key(&init_net, &fl4);
 if (!IS_ERR(rt)) {
 mtu = dst_mtu(&rt->dst);
 ip_rt_put(rt);
 }
 return mtu;
}

/**
 * tcpmss_mangle_packet() - Mangle TCP MSS option in a packet
 * @skb: socket buffer containing the packet to modify
 * @tcpmss_option: type of MSS manipulation (MSS_VALUE or ADJUST_MSS_TO_PMTU)
 * @mss: target MSS value (only used if tcpmss_option == MSS_VALUE)
 *
 * This is the main entry point for TCP MSS manipulation. It:
 * 1. Calls __tcpmss_mangle_packet() to modify the MSS option in TCP header
 * 2. If successful, updates the IP header total_length and checksum
 *
 * The function may enlarge the packet if the MSS option needs to be added.
 * In this case, ret > 0 indicates how many bytes were added.

```

```

*
* Returns: <0 on error, 0 if no change needed, >0 if packet was enlarged
*/
int tcpmss_mangle_packet(struct sk_buff *skb, short unsigned int tcpmss_option, __u16
mss)
{
 int ret, newlen;
 struct iphdr *iph;

 /* Perform the actual MSS mangling */
 ret = __tcpmss_mangle_packet(skb, tcpmss_option, mss);

 /* If packet was enlarged (MSS option was added), update IP header */
 if (ret > 0) {
 iph = ip_hdr(skb);
 /* Socket buffer has been enlarged by 'ret' bytes
 * Calculate new total length: old_length + bytes_added
 */
 newlen = htons(ntohs(iph->tot_len) + ret);

 /* Update IP header checksum to reflect new total length
 * csum_replace2(): efficiently updates checksum by replacing old value with new
 */
 csum_replace2(&iph->check, iph->tot_len, newlen);
 iph->tot_len = newlen;
 }
 return ret;
}

int __tcpmss_mangle_packet(struct sk_buff *skb,
 short unsigned int tcpmss_option,
 __u16 mss/*,
 ipfire_info_t *info*/)
{
 struct tcphdr *tcph;
 struct iphdr *iph;
 unsigned int tcplen, i, tcphoff, mtu_minlen;
 __be16 oldval;
 __u16 newmss;
 u_int16_t oldmss;
 u32 net_mtu;
 u8 *opt;
 mtu_minlen = sizeof(struct iphdr) + sizeof(struct tcphdr);

 if (skb_ensure_writable(skb, skb->len))
 {
 IPFI_PRINTK("IPFI: __tcpmss_mangle_packet(): skb_ensure_writable failed\n");
 return -1;
 }
 /* reload pointers after skb_make_writable() call */
 iph = ip_hdr(skb);
 tcphoff = iph->ihl * 4;
 tcplen = skb->len - tcphoff;
 tcph = (struct tcphdr *) (skb_network_header(skb) + tcphoff);

 /* (1) Since it passed flags test in tcp match, we know it is is
 * not a fragment, and has data >= tcp header length. SYN
 * packets should not contain data: if they did, then we risk
 * running over MTU, sending Frag Needed and breaking things
 * badly. --RR

```



```

*
* (1) Quoted from net/netfilter/xt_TCPMSS.c
*/
if (tcplen != tcph->doff * 4)
{
 IPFI_PRINTK("IPFIRE: __tcpmss_mangle_packet(): bad tcp header length (%u
bytes)\n", skb->len);
 return -1;
}

if(tcpmss_option == ADJUST_MSS_TO_PMTU) {
 struct dst_entry *dst_en = skb_dst(skb);
 net_mtu = get_net_mtu(iph->saddr);
 if(dst_mtu(dst_en) <= mtu_minlen)
 {
 IPFI_PRINTK("__tcpmss_mangle_packet(): unknown or invalid skb PATH-MTU
(%u)\n", dst_mtu(dst_en));
 return -1;
 }
 if(net_mtu <= mtu_minlen)
 {
 IPFI_PRINTK("__tcpmss_mangle_packet(): unknown or invalid calculated PATH-MTU
(%u)\n", net_mtu);
 return -1;
 }
 newmss = min(dst_mtu(dst_en), net_mtu) - mtu_minlen;
}
else
{
 newmss = mss;
}

/* first byte of the tcp header */
opt = (u_int8_t *)tcph;
/* cycle: start: i: first byte after the end of the tcp header;
* test: i: less than tcp header data offset (start of data) - the number of
32 bit words in the TCP Header
* increment: i: length of option
*/
for (i = sizeof(struct tcphdr); i < tcph->doff * 4; i += optlen(opt, i))
{
 /* TCPOLEN_MSS: length of the tcp header mss option, from include/net/tcp.h
 * TCPOPT_MSS: option 'segment size negotiating', from include/net/tcp.h
 * Maximum Segment Size

 +-----+-----+-----+-----+
 |00000010|00000100| max seg size |
 +-----+-----+-----+-----+
 Kind Length:4
 *
 * note: tcph->doff * 4 - i must be greater than or equal TCPOLEN_MSS (4 bytes:
1:kind,2:length,3,4:mss)
 */
 if (opt[i] == TCPOPT_MSS && tcph->doff * 4 - i >= TCPOLEN_MSS && opt[i+1] ==
TCPOLEN_MSS)
 {
 oldmss = (opt[i+2] << 8) | opt[i+3];
 // IPFI_PRINTK("oldmss: %u ----- new mss
(possible): %u [i = %d]\n", oldmss, newmss, i);
 /* Never increase MSS, even when setting it, as
 * doing so results in problems for hosts that rely

```

```

 * on MSS being set correctly.
 */
 // info->manipinfo.pmanip.mss.enabled = 1;
 if (oldmss <= newmss)
 {
 // info->manipinfo.pmanip.mss.old_lesssthan = 1;
 // info->manipinfo.pmanip.mss.mss = oldmss;
 return 0;
 }
 /* else: fill in the new mss */
 opt[i+2] = (newmss & 0xff00) >> 8;
 opt[i+3] = newmss & 0x00ff;
 // info->manipinfo.pmanip.mss.mss = newmss;
 /* update checksum */
 inet_proto_csum_replace2(&tcph->check, skb,
 htons(oldmss), htons(newmss),
 0);

 return 0;
 }
}

IPFI_PRINTK("IPFIRE: mss option not found. Adding\n");
/*
 * MSS Option not found ?! add it..
 */
if (skb_tailroom(skb) < TCPOLEN_MSS)
{
 if (pskb_expand_head(skb, 0, TCPOLEN_MSS - skb_tailroom(skb), GFP_ATOMIC))
 return -1;
 tcph = (struct tcphdr *) (skb_network_header(skb) + tcphoff);
}

/* net/core/skbuff.c: skb_put - add data to a buffer.
 * Extends the used data area of the buffer of an amount TCPOLEN_MSS
 */
skb_put(skb, TCPOLEN_MSS);
opt = (u_int8_t *) tcph + sizeof(struct tcphdr);
/* (tcplen = skb->len - tcphoff) move memory pointed by opt (tcph + sizeof(struct
tcphdr) = start of data
 * into opt + TCPOLEN_MSS
 */
memmove(opt + TCPOLEN_MSS, opt, tcplen - sizeof(struct tcphdr));

inet_proto_csum_replace2(&tcph->check, skb, htons(tcplen), htons(tcplen +
TCPOLEN_MSS), 1);
opt[0] = TCPOPT_MSS;
opt[1] = TCPOLEN_MSS;
opt[2] = (newmss & 0xff00) >> 8;
opt[3] = newmss & 0x00ff;

inet_proto_csum_replace4(&tcph->check, skb, 0, ((__be32 *)opt), 0);

oldval = ((__be16 *)tcph)[6]; /* old value calculation to use in csum_replace..
below */
tcph->doff += TCPOLEN_MSS/4; /* new start of data position */
inet_proto_csum_replace2(&tcph->check, skb, oldval, ((__be16 *)tcph)[6], 0);
// info->manipinfo.pmanip.mss(skb_enlarged = 1;
return TCPOLEN_MSS;
}

```

```
EXPORT_SYMBOL(packet_suitable_for_mss_change);
EXPORT_SYMBOL(tcpmss_mangle_packet);
```

## logging/log.h

[illegible]

```
#endif
```

## logging/log.c

```
/* ipfi_log.c: a packet to be logged is really sent to userspace only if
 * it is not identical to a one previously sent. This reduces kernel/user
 * communication load */

/*****
 * Copyright 2005 Giacomo
 * jacum@libero.it
 *****/

#include "logging/log.h"
#include "globals.h"
#include <linux/gfp.h>
#include <linux/hashtable.h>
#include <linux/jhash.h>
#include <linux/percpu_counter.h>
#include <linux/slab.h>

/*
 * DESIGN:
 *
 * Log entries follow the unified ipfi_entry lifecycle (table_lifecycle.c):
 *
 * 1. ALLOCATION: kmem_cache_alloc from a dedicated slab cache.
 * 2. LIFECYCLE: Each entry has an individual timer that fires on
 * expiration. On duplicate match, the timer is refreshed
 * via ipfi_entry_update_timer() using IPPROTO_IPFI_LOG
 * as sentinel protocol (throttled to 5s).
 * 3. REMOVAL: Timer callback calls ipfi_entry_remove + ipfi_entry_put.
 * ipfi_entry_put defers to workqueue -> timer_delete_sync
 * -> call_rcu -> kfree (handles kmem_cache objects on 6.x).
 * 4. MAX CAP: loginfo_entry_counter is checked before allocation.
 * If at capacity, the oldest entry is evicted.
 */

/* ---- Dedicated slab cache for ipfire_loginfo entries ---- */
struct kmem_cache *loginfo_cache;

/**
 * get_log_hash - compute the hash bucket key for a log-deduplication entry.
 * @saddr: IPv4 source address (network byte order)
 * @daddr: IPv4 destination address (network byte order)
 * @proto: IP protocol number (IPPROTO_TCP, IPPROTO_UDP, IPPROTO_ICMP, ...)
 *
 * DESIGN: 3-TUPLE KEY vs. 5-TUPLE (state/NAT tables)
 * -----
 * State and NAT tables use the full 5-tuple (saddr, daddr, sport, dport,
 * proto) as their hash key. This is correct for those tables because each
 * lookup must retrieve *one specific connection*: sport and dport together
 * make every flow unique, so the extra discrimination cuts collision chains
 * to near-zero even at high connection rates.
 *
 * The log-deduplication table has a different contract. packet_not_seen()
 * never returns a single authoritative entry; it scans every entry in the
 */
```

```

* bucket and delegates the actual equality test to compare_logininfo_packets(),
* which performs a full content comparison (IPs, ports, flags, direction,
* rule ID, connection state). The hash key here is only a *pre-filter*: its
* job is to route the lookup to the right bucket quickly, not to identify the
* entry by itself.
*
* Using a 3-tuple (saddr, daddr, proto) trades a slightly longer per-bucket
* scan in exchange for two concrete advantages:
*
* 1. PROTOCOL GENERALITY. ICMP, IGMP, GRE, PIM and other protocols carry
* no port numbers. A 5-tuple key would require special-casing (zero-
* filling sport/dport) for each of these, and would still place all
* portless packets from the same host pair in the same bucket – so the
* extra complexity buys nothing. The 3-tuple handles all protocols
* uniformly with a single code path.
*
* 2. LOG SEMANTIC. A log entry represents "a class of recently-seen
* packets between this src/dst pair using this protocol". Two TCP
* connections from the same client to the same server on different
* ephemeral ports are still the same logging event from a firewall
* visibility standpoint. Grouping them in the same bucket is
* semantically appropriate; the full comparison in
* compare_logininfo_packets() still distinguishes them when needed.
*
* Note: with LOG_HASH_BITS=7 (128 buckets) and a typical logininfo table
* that stays well below max_logininfo_entries, per-bucket chains are short
* regardless. A migration to a 5-tuple key (to mirror state/NAT) is a
* valid future optimisation for very large deployments.
*/
static u32 get_log_hash(__be32 saddr, __be32 daddr, __u8 proto) {
 return jhash_3words((__u32)saddr, (__u32)daddr, proto, 0);
}

/* ---- Timer callback ---- */

static void handle_logininfo_timeout(struct timer_list *t) {
 struct ipfire_logininfo *li = timer_container_of(li, t, h.timer);
 struct ipfi_entry_head *h = &li->h;

 spin_lock_bh(&logininfo_list_lock);
 /*
 * BUG FIX: check REMOVED *before* list_del_rcu.
 *
 * logininfo_evict_oldest() runs under the same lock and does:
 * list_del_rcu(lnode) → ipfi_entry_remove (sets REMOVED, puts internally)
 *
 * If eviction already ran, lnode.prev/next are LIST_POISON. Calling
 * list_del_rcu again writes to LIST_POISON address → memory corruption
 * → silent hard panic. Guard with the REMOVED bit which is set atomically
 * by ipfi_entry_remove under this same lock.
 */
 if (!test_bit(IPFI_ENTRY_REMOVED, &h->status))
 list_del_rcu(&h->lnode);
 ipfi_entry_remove(h, &logininfo_entry_counter); /* puts internally if winner */
 spin_unlock_bh(&logininfo_list_lock);
}

/* ---- Allocation ---- */

static struct ipfire_logininfo *logininfo_alloc(void) {
 struct ipfire_logininfo *li;

```

```

 li = kmem_cache_alloc(logininfo_cache, GFP_ATOMIC);
 if (!li)
 return NULL;

 memset(li, 0, sizeof(*li));
 ipfi_entry_init(&li->h, logininfo_lifetime, handle_logininfo_timeout);
 refcount_set(&li->h.refcnt, 1);

 return li;
}

/* ---- Eviction (when at capacity) ---- */

// must be called under spinlock_bh
static void logininfo_evict_oldest(void) {
 struct ipfire_logininfo *oldest;

 /* Caller MUST hold logininfo_list_lock */
 if (list_empty(&active_logi_list))
 return;

 /* lnode removal: ipfi_entry_remove removes hnode in hash mode, so we
 * remove lnode first to keep active_logi_list consistent before remove. */
 oldest = list_last_entry(&active_logi_list, struct ipfire_logininfo, h.lnode);
 list_del_rcu(&oldest->h.lnode);
 ipfi_entry_remove(&oldest->h, &logininfo_entry_counter); /* puts internally */

 /*
 * BUG FIX: cancel a pending timer before releasing ownership.
 *
 * timer_delete() cancels the timer if it is pending (not yet running).
 * If it is already running on another CPU, that CPU is blocked on
 * logininfo_list_lock (we hold it), so it will check REMOVED (now set)
 * when it gets the lock and will skip list_del_rcu; ipfi_entry_remove
 * will be a no-op (REMOVED bit already set → no put).
 *
 * timer_delete without _sync is safe here: we hold a BH-disabled
 * spinlock and timer_delete_sync would deadlock.
 */
 timer_delete(&oldest->h.timer);
}

/* ---- Header extraction helpers (moved from ipfire.c) ---- */

static void build_tcpiph_usermess(const struct tcphdr *tcph,
 ipfire_info_t *ipfi_info) {
 /* we fill in our userspace information */
 memcpy(&(ipfi_info->packet.transport_header.tcphead), tcph,
 sizeof(struct tcphdr));
}

static void build_udph_usermess(const struct udphdr *p_udphdr,
 ipfire_info_t *ipfi_info) {
 memcpy(&(ipfi_info->packet.transport_header).udphdr, p_udphdr,
 sizeof(*p_udphdr));
}

static void build_icmph_usermess(const struct icmphdr *p_icmphdr,
 ipfire_info_t *ipfi_info) {
 memcpy(&(ipfi_info->packet.transport_header).icmphdr, p_icmphdr,

```

```

 sizeof(*p_icmphead));
}

static void build_igmp_header(const struct igmp_hdr *p_igmphead,
 ipfire_info_t *ipfi_info) {
 memcpy(&(ipfi_info->packet.transport_header).igmphead, p_igmphead,
 sizeof(*p_igmphead));
}

static int copy_headers(const struct sk_buff *skb, ipfire_info_t *fireinfo) {
 struct ip_hdr *iph;
 iph = ip_hdr(skb);
 /* protocol information */
 fireinfo->packet.ip.protocol = iph->protocol;
 fireinfo->packet.ip.saddr = iph->saddr;
 fireinfo->packet.ip.daddr = iph->daddr;
 /* internet header */
 /* tcp, udp icmp headers? */
 if (iph->protocol == IPPROTO_TCP)
 build_tcp_header((struct tcp_hdr *)((void *)iph + iph->ihl * 4),
 fireinfo);
 else if (iph->protocol == IPPROTO_UDP)
 build_udp_header((struct udp_hdr *)((void *)iph + iph->ihl * 4),
 fireinfo);
 else if (iph->protocol == IPPROTO_ICMP)
 build_icmp_header((struct icmp_hdr *)((void *)iph + iph->ihl * 4),
 fireinfo);
 else if (iph->protocol == IPPROTO_IGMP)
 build_igmp_header((struct igmp_hdr *)((void *)iph + iph->ihl * 4),
 fireinfo); /* since 0.98.7 */
 else {
 /* Unknown protocol: allow logging but skip transport header details. */
 return 0;
 }
 return 0;
}

int build_ipfire_info_from_skb(const struct sk_buff *skb, const ipfi_flow *flow,
 const struct response *res,
 const struct info_flags *flags,
 ipfire_info_t *dest) {
 if (copy_headers(skb, dest) < 0)
 return -1;

 dest->flags = *flags;
 dest->flags.direction = flow->direction;

 if (flow->in)
 strncpy(dest->netdevs.in_devname, flow->in->name, IFNAMSIZ);
 if (flow->out)
 strncpy(dest->netdevs.out_devname, flow->out->name, IFNAMSIZ);
 dest->response = *res;
 return 0;
}

inline int add_packet_to_infolist(const struct sk_buff *skb,
 const struct response *res,
 const ipfi_flow *flow,
 const struct info_flags *flags) {
 struct ipfire_loginfo *ipli;

```

```

if (unlikely(READ_ONCE(we_are_exiting)))
 return -EBUSY;

ipli = loginfo_alloc();
if (unlikely(!ipli))
 return -ENOMEM;

if (build_ipfire_info_from_skb(skb, flow, res, flags, &ipli->info) < 0) {
 ipfi_entry_put(&ipli->h);
 return -1;
}

spin_lock_bh(&loginfo_list_lock);

if (unlikely(we_are_exiting)) {
 spin_unlock_bh(&loginfo_list_lock);
 ipfi_entry_put(&ipli->h);
 return -EBUSY;
}

/* Enforce max entries cap */
if (percpu_counter_read(&loginfo_entry_counter) >= max_loginfo_entries) {
 loginfo_evict_oldest();
}

/* Always add to active list for LRU ordering (eviction uses list_last_entry)
 */
list_add_rcu(&ipli->h.lnode, &active_logi_list);
{
 u32 key =
 get_log_hash(ipli->info.packet.ip.saddr, ipli->info.packet.ip.daddr,
 ipli->info.packet.ip.protocol);
 hash_add_rcu(loginfo_hashtable, &ipli->h.lnode, key);
}
percpu_counter_add_batch(&loginfo_entry_counter, 1, 1);

/*
 * BUG FIX: hold an extra reference across the lock → arm-timer gap.
 *
 * After spin_unlock_bh(), another CPU can see the entry in the list,
 * call loginfo_evict_oldest(), drop refcount to 0, and queue the
 * cleanup work which eventually calls kfree(). If the workqueue runs
 * before ipfi_entry_arm_timer() below, mod_timer() fires on freed
 * memory (UAF). The extra hold keeps refcount ≥ 2 until we arm the
 * timer; the matching put releases it.
 */
ipfi_entry_hold(&ipli->h);
spin_unlock_bh(&loginfo_list_lock);

/* Arm the timer AFTER the entry is on the list */
ipfi_entry_arm_timer(&ipli->h);

/* Release the temporary arm-guard hold */
ipfi_entry_put(&ipli->h);

return 0;
}

inline int iph_compare(const struct iphdr *skb_iphdr, const ipfire_info_t *p2) {
 const struct ip_id *ip = &p2->packet.ip;
 return (skb_iphdr->saddr == ip->saddr) && (skb_iphdr->daddr == ip->daddr);
}

```



```

}

inline int tcp_compare(const struct tcphdr *tcp1, const ipfire_info_t *p2) {
 const struct tcphdr tcp2 = p2->packet.transport_header.tcphead;
 return (tcp1->source == tcp2.source) && (tcp1->dest == tcp2.dest) &&
 (tcp1->fin == tcp2.fin) && (tcp1->syn == tcp2.syn) &&
 (tcp1->ack == tcp2.ack) && (tcp1->urg == tcp2.urg) &&
 (tcp1->rst == tcp2.rst) && (tcp1->psh == tcp2.psh);
}

inline int udp_compare(const struct udphdr *udp1, const ipfire_info_t *p2) {
 struct udphdr udp2;
 udp2 = p2->packet.transport_header.udphead;
 return (udp1->source == udp2.source) && (udp1->dest == udp2.dest);
}

inline int icmp_compare(const struct icmphdr *ich1, const ipfire_info_t *p2) {
 struct icmphdr ich2;
 ich2 = p2->packet.transport_header.icmphead;
 return (ich1->type == ich2.type) && (ich1->code == ich2.code);
}

inline int igmp_compare(const struct igmp_hdr *igh1, const ipfire_info_t *p2) {
 struct igmp_hdr igh2;
 igh2 = p2->packet.transport_header.igmphead;
 return (igh1->type == igh2.type) && (igh1->code == igh2.code) &&
 (igh1->group == igh2.group);
}

int packet_matches_log_entry(const struct sk_buff *skb,
 const struct response *res, const ipfi_flow *flow,
 const struct info_flags *flags,
 const ipfire_info_t *p2) {
 struct iphdr *iph = ip_hdr(skb);

 if (iph->protocol != p2->packet.ip.protocol) {
 return -1;
 }
 if (res->st.state != p2->response.st.state) {
 return -1;
 }
 if (flow->direction != p2->flags.direction || flags->nat != p2->flags.nat ||
 flags->snat != p2->flags.snat) {
 return -1;
 }

 if (res->verdict != p2->response.verdict) {
 return -1;
 }
 const char *in_name = flow->in ? flow->in->name : "";
 if (strncmp(in_name, p2->netdevs.in_devname, IFNAMSIZ) != 0) {
 return -1;
 }
 const char *out_name = flow->out ? flow->out->name : "";
 if (strncmp(out_name, p2->netdevs.out_devname, IFNAMSIZ) != 0) {
 return -1;
 }

 if (!iph_compare(iph, p2)) {
 return -1;
 }
}

```

```

switch (iph->protocol) {
case IPPROTO_TCP: {
 struct tcphdr *th = (struct tcphdr *)((void *)iph + iph->ihl * 4);
 if (!tcph_compare(th, p2))
 return -1;
 break;
}
case IPPROTO_UDP: {
 struct udphdr *uh = (struct udphdr *)((void *)iph + iph->ihl * 4);
 if (!udph_compare(uh, p2))
 return -1;
 break;
}
case IPPROTO_ICMP: {
 struct icmphdr *ih = (struct icmphdr *)((void *)iph + iph->ihl * 4);
 if (!icmph_compare(ih, p2))
 return -1;
 break;
}
case IPPROTO_IGMP: {
 struct igmpchr *gh = (struct igmpchr *)((void *)iph + iph->ihl * 4);
 if (!igmpchr_compare(gh, p2))
 return -1;
 break;
}
case IPPROTO_GRE:
case IPPROTO_PIM:
 /* comparison is limited to the checks above, no GRE header comparison is
 * done, no PIM info inspected */
 break;
default:
 IPFI_MODERATE_PRINTK(
 PRINT_PROTO_UNSUPPORTED,
 "IPFIRE: ipfi_log.c: packet_matches_log_entry(): unsupported "
 "protocol %d.\n",
 iph->protocol);
}
return 0;
}

inline int compare_loginfo_packets(const struct sk_buff *skb,
 const struct response *res,
 const ipfi_flow *flow,
 const struct info_flags *flags,
 const ipfire_info_t *packet2) {
 if (packet_matches_log_entry(skb, res, flow, flags, packet2) == 0)
 return 1;
 return 0;
}

inline int packet_not_seen(const struct sk_buff *skb,
 const struct response *res, const ipfi_flow *flow,
 const struct info_flags *flags, int chk_state) {
 struct ipfire_loginfo *loginfo;

 /* Short circuit */
 if (percpu_counter_read(&loginfo_entry_counter) == 0)
 return 1;

 rcu_read_lock_bh();
 {

```



```

percpu_counter_destroy(&loginfo_entry_counter);
/* kmem_cache_destroy deferred to ipfire.c::fini() after
 * destroy_workqueue + rcu_barrier ensure all kfree callbacks ran. */
}

```

## helpers/ftp.h

```

#ifndef IPFI_FTP_H
#define IPFI_FTP_H

/* See ipfi.c for details and
 * use of this software.
 * (C) 2005 Giacomo S.
 */

#include "../nat/nat.h"
#include "ipfi_machine.h"
#include "ipfire.h"
#include "netlink/ipfi_netl.h"

typedef struct {
 __u32 ftp_addr;
 __u16 ftp_port;
 short valid;
} ftp_info;

/* returns 1 if a new entry is added, 0 if the skb data do not
 * contain ftp 227 information about ip and port,
 * -1 if an error occurs */
struct state_table *ftp_support(struct state_table *tentry,
 const struct sk_buff *skb);

/* if skb data contain ftp address and port, allocate and return the new entry
 * to be added to the dynamic tables list */
struct state_table *
packet_contains_ftp_params(const struct sk_buff *skb,
 const struct state_table *orig_entry,
 char *ftp_buffer);

/* just inspect if skb contains 227 command ("Entering passive mode") */
int data_start_with_227(const struct sk_buff *skb, char *ftp_buffer);

/* returns a new kmallocated struct state_table. It is the copy of the
 * original ftp table, with the new address and port.
 * _Remember_ to initialize a new timer and to add the rule at the tail
 * of the list in the calling function. */
struct state_table *
get_params_and_alloc_newentry(const struct state_table *orig, char *ftp_buffer);

/** inspects skb data and retrieves ftp address and port.
 * @return a structure of type ftp_info, which has the flag valid
 * set to 1 if it is valid, 0 if something failed. The caller must
 * check against the valid flag.
 */
ftp_info get_ftpaddr_and_port(char *ftp_buffer);

/* takes ftp string and fills in integers representing ip and port */
int get_u8s(__u8 *a1, __u8 *a2, __u8 *a3, __u8 *a4, __u8 *p1, __u8 *p2);

```

```

/* checks a bit of syntax in buffer related to 227 command */
inline int check_buf(const char *ftpcmd);

int clean_ftp_command(char *cleaned, char *ftp_buffer);

void rehash_ftp_expectation(struct state_table *entry, __be16 new_sport);

struct state_table *lookup_ftp_expectation(const struct sk_buff *skb,
 const struct iphdr *iph,
 __be16 sport, __be16 dport,
 short *reverse,
 const ipfi_flow *flow);

#endif

```

## helpers/ftp.c

```

/* ip firewall Giacomo S.
 * Passive ftp support module */

/*****
 * Copyright 2005 Giacomo
 * jacum@libero.it
 *****/

/*
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation; either version 2 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 */
/* see ipfi.c for details */

#include "helpers/ftp.h"
#include "../filter/state/state_table.h"
#include "globals.h"
#include "ipfi_machine.h"
#include "ipfire.h"
#include "netlink/ipfi_netl.h"
#include <linux/list.h>
#include <linux/module.h>

#define FTPBUF 256
#define CLEANEDBUF 128

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Giacomo S. <jacum@libero.it>");
MODULE_DESCRIPTION("Passive FTP support module");

/* Removed thread-unsafe global: char ftp_buffer[FTPBUF]; */

/* returns 1 if a new entry is added, 0 if the skb data do not

```

```

* contain ftp 227 information about ip and port,
* -1 if an error occurs
* Comes from:
* -iph_in_get_response();
* - ipfire_filter();
* - check_state();
* skb already checked in ipfi_response() against `NULL' value
*/
struct state_table *ftp_support(struct state_table *tentry,
 const struct sk_buff *skb) {
 char ftp_buffer[FTPBUF];
 return packet_contains_ftp_params(skb, tentry, ftp_buffer);
}

/* if skb data contain ftp address and port, allocate and return the new entry
* to be added to the dynamic tables list */
struct state_table *
packet_contains_ftp_params(const struct sk_buff *skb,
 const struct state_table *orig_entry,
 char *ftp_buffer) {
 struct state_table *newt = NULL;
 if (data_start_with_227(skb, ftp_buffer) > 0)
 newt = get_params_and_alloc_newentry(orig_entry, ftp_buffer);
 return newt;
}

/* just inspect if skb contains 227 command ("Entering passive mode").
* Handles multi-response segments: when FTP commands are pipelined (e.g.
* USER/PASS/PASV/QUIT sent at once), the server may reply with several
* response lines concatenated in a single TCP segment. We must search for
* the "227" line anywhere in the payload, not just at byte 0.
*
* On success, ftp_buffer contains ONLY the "227 ...(A,B,C,D,p,q)\r\n" line
* (subsequent lines are stripped) so that check_buf / get_ftpaddr_and_port
* can parse it correctly.
*/
int data_start_with_227(const struct sk_buff *skb, char *ftp_buffer) {
 unsigned int dataoff, datalen;
 char *data_ptr;
 char *p, *eol;
 struct tcphdr *th;
 struct iphdr *iph;

 iph = ip_hdr(skb);

 /* a packet arrives here if protocol is TCP, see check_state() */
 th = (struct tcphdr *)((void *)iph + iph->ihl * 4);
 dataoff = iph->ihl * 4 + th->doff * 4;
 /* If there is no data in the buffer, return -1 */
 if (dataoff >= skb->len)
 return -1;
 datalen = skb->len - dataoff;

 if (datalen > FTPBUF - 1)
 datalen = FTPBUF - 1;

 data_ptr = skb_header_pointer(skb, dataoff, datalen, ftp_buffer);

 if (data_ptr == NULL) {
 IPFI_PRINTK("IPFIRE: fb_ptr NULL! (ipfi_ftp.c)\n");
 return -1;
 }
}

```

```

}

if (data_ptr != ftp_buffer)
 memcpy(ftp_buffer, data_ptr, datalen);

ftp_buffer[datalen] = '\0';

/* 227 () A,B,C,D,p,q -> minimal string representing a 227 command */
if (datalen < 16)
 return 0;

/* Search for "227" at the start of any response line.
 * FTP response lines are delimited by \r\n, so a mid-buffer 227 is
 * always preceded by \n (or it is at position 0). */
p = ftp_buffer;
while ((p = strstr(p, "227")) != NULL) {
 /* Accept if it's at position 0 or preceded by \n */
 if (p == ftp_buffer || *(p - 1) == '\n') {
 /* Isolate this single line: find the \n that ends it */
 eol = strchr(p, '\n');
 if (eol) {
 (eol + 1) = '\0'; / keep the \n, remove whatever follows */
 }
 /* Now move the 227 line to the start of ftp_buffer */
 if (p != ftp_buffer)
 memmove(ftp_buffer, p, strlen(p) + 1);

 IPFI_PRINTK("IPFIRE FTP: found 227 line: %s", ftp_buffer);
 return 1;
 }
 p++; /* advance past this non-matching "227" occurrence */
}
return 0;
}

/* returns a new kmallocated struct state_table. It is the copy of the
 * original ftp table, with the new address and port.
 * _Remember_ to initialize a new timer and to add the rule at the tail
 * of the list in the calling function. */
struct state_table *
get_params_and_alloc_newentry(const struct state_table *orig,
 char *ftp_buffer) {
 ftp_info ftpi;
 struct state_table *newt = NULL;
 ftpi = get_ftpaddr_and_port(ftp_buffer);

 if (ftpi.valid) {
 newt = (struct state_table *)kmem_cache_alloc(state_cache, GFP_ATOMIC);
 if (!newt) {
 IPFI_PRINTK("failed to allocate space for the ftp state table!\n");
 return NULL;
 }
 /* to start, copy old table into new one */
 memset(newt, 0, sizeof(struct state_table));
 refcount_set(&newt->h.refcnt, 1);
 newt->saddr = orig->saddr;
 newt->sport = 0; /* Client data connection uses unknown ephemeral port */
 newt->direction = orig->direction;
 newt->notify = orig->notify;
 newt->admin = orig->admin;
 newt->protocol = orig->protocol;
 }
}

```

```

newt->h.status = orig->h.status;
if (orig->in_devname[0]) {
 strncpy(newt->in_devname, orig->in_devname, IFNAMSIZ - 1);
 newt->in_devname[IFNAMSIZ - 1] = '\0';
}
if (orig->out_devname[0]) {
 strncpy(newt->out_devname, orig->out_devname, IFNAMSIZ - 1);
 newt->out_devname[IFNAMSIZ - 1] = '\0';
}
newt->state = orig->state;
newt->daddr = ftpi.ftp_addr;
newt->dport = ftpi.ftp_port;
newt->ftp = FTP_DEFINED;
newt->rule_id = orig->rule_id;
newt->related = 1;
}
/* return the new allocated state table or NULL */
return newt;
}

/* checks a bit of syntax in buffer related to 227 command.
 * Note: some FTP servers append characters after ')' before \r\n, e.g.:
 * 227 Entering Passive Mode (85,188,1,133,213,231).\r\n
 * so we search backwards for ')' rather than requiring it at a fixed offset. */
inline int check_buf(const char *ftpcmd) {
 int len = strlen(ftpcmd);
 int j;
 unsigned i = 0;
 unsigned commas = 0, parenthesis = 0;
 if (len - 3 < 0 || len > FTPBUF)
 return -1;
 if (ftpcmd[len - 1] != '\n')
 return -1;
 if (ftpcmd[len - 2] != '\r')
 return -1;
 /* Search backwards from \r for ')'; allow trailing chars like '.' */
 for (j = len - 3; j >= 0; j--) {
 if (ftpcmd[j] == ')')
 break;
 }
 if (j < 0)
 return -1;
 for (i = 0; i < len && i < FTPBUF; i++) {
 if (ftpcmd[i] == ',')
 commas++;
 else if (ftpcmd[i] == '(')
 parenthesis++;
 }
 if ((commas != 5) || (parenthesis != 1))
 return -1;

 return 1;
}

/* takes ftp string and fills in integers representing ip and port */
int clean_ftp_command(char *cleaned, char *ftp_buffer) {
 unsigned i = 0, j = 0;

 if (check_buf(ftp_buffer) < 0) {
 IPFI_PRINTK("IPFIRE: bad format for 227 ftp command: \"%s\"\n", ftp_buffer);
 return -1;
 }

```



```

}

while ((i < FTPBUF) && (ftp_buffer[i] != '\0') && (ftp_buffer[i] != '('))
 i++;
/* reached '(', ftp_buffer[i] points to '(' */
i++; /* pass '(' */
while ((i < FTPBUF) && (j < CLEANEDBUF - 1) && (ftp_buffer[i] != '\0') &&
 (ftp_buffer[i] != ' ')) {
 cleaned[j] = ftp_buffer[i];
 i++;
 j++;
}
cleaned[j] = '\0'; /* Terminate string */

return 1;
}

/* inspects skb data and retrieves ftp address and port.
 * returns a structure of type ftp_info, which has the flag valid
 * set to 1 if it is valid, 0 if something failed. The caller must
 * check against the valid flag.
 */
ftp_info get_ftpaddr_and_port(char *ftp_buffer) {
 ftp_info ftpi, invalid_ftpinfo;
 char cleaned[CLEANEDBUF];
 __u8 a1, a2, a3, a4, p1, p2;
 __u32 n = 0, m = 0, o = 0;

 memset(&ftpi, 0, sizeof(ftp_info));
 memset(&invalid_ftpinfo, 0, sizeof(invalid_ftpinfo));

 /* validate the ftp_info aimed at containing a valid result */
 ftpi.valid = 1;

 if (clean_ftp_command(cleaned, ftp_buffer) < 0) {
 IPFI_PRINTK("IPFIRE: error cleaning ftp buffer!\n");
 return invalid_ftpinfo;
 }

 if (sscanf(cleaned, "%hhu,%hhu,%hhu,%hhu,%hhu,%hhu", &a1, &a2, &a3, &a4, &p1,
 &p2) != 6) {
 IPFI_PRINTK("IPFIRE: sscanf failed on cleaned buffer: %s\n", cleaned);
 return invalid_ftpinfo;
 }
 /* compute address */
 ftpi.ftp_addr = a4;
 n = a1;
 n = n << 24;
 m = a2;
 m = m << 16;
 o = a3;
 o = o << 8;
 ftpi.ftp_addr = ftpi.ftp_addr + n + m + o;
 ftpi.ftp_addr = htonl(ftp_i.ftp_addr);
 /* port */
 n = p1;
 n = n << 8;
 ftpi.ftp_port = p2 + n;
 ftpi.ftp_port = htons(ftp_i.ftp_port);

 IPFI_PRINTK("IPFIRE FTP: Parsed PASV response: IP %pI4, Port %u\n",

```

```

 &ftpi.ftp_addr, ntohs(ftpi.ftp_port));

 return ftpi;
}

/*
 * rehash_ftp_expectation – one-shot optimisation for FTP passive data flows.
 *
 * When a passive-mode FTP 227 response is parsed, we create an expectation
 * entry with sport=0 (wildcard) because the client's ephemeral data port is
 * unknown at that point. This entry is hashed into the sport=0 bucket.
 *
 * On the first data-connection packet, lookup_ftp_expectation() finds the
 * entry via the sport=0 bucket (secondary lookup). The caller may then
 * invoke this function to:
 *
 * 1. Fill in the real client source port (new_sport).
 * 2. Move the entry from the sport=0 bucket to the correct 5-tuple bucket.
 * 3. Set ftp = FTP_ESTABLISHED so subsequent packets hit the normal O(1)
 * hash path and bypass the secondary lookup entirely.
 *
 * This is purely an optimisation – without it, every data packet would go
 * through the secondary lookup, which is still correct but O(bucket_size)
 * per packet. For FTP traffic volumes, either approach is acceptable.
 *
 * Lock ordering: when two distinct buckets must be locked, the lower-numbered
 * bucket is always locked first to prevent AB-BA deadlocks.
 *
 * NOT WIRED UP YET – call site needs to be added once the caller decides
 * to use this optimisation.
 */
void rehash_ftp_expectation(struct state_table *entry, __be16 new_sport) {
 __u32 old_key = get_state_hash(entry->saddr, entry->daddr, 0, entry->dport,
 entry->protocol);

 /* MUST use hash_min() – same function as
 * hash_add_rcu/hash_for_each_possible_rcu */
 unsigned int old_bkt = hash_min(old_key, STATE_HASH_BITS);

 __u32 new_key = get_state_hash(entry->saddr, entry->daddr, new_sport,
 entry->dport, entry->protocol);
 unsigned int new_bkt = hash_min(new_key, STATE_HASH_BITS);

 if (old_bkt == new_bkt) {
 spin_lock_bh(&state_bucket_locks[old_bkt]);
 entry->sport = new_sport;
 entry->ftp = FTP_ESTABLISHED;
 spin_unlock_bh(&state_bucket_locks[old_bkt]);
 } else {
 /* Lock ordering to prevent deadlocks */
 if (old_bkt < new_bkt) {
 spin_lock_bh(&state_bucket_locks[old_bkt]);
 spin_lock_bh(&state_bucket_locks[new_bkt]);
 } else {
 spin_lock_bh(&state_bucket_locks[new_bkt]);
 spin_lock_bh(&state_bucket_locks[old_bkt]);
 }

 hlist_del_rcu(&entry->h.hnode);
 entry->sport = new_sport;
 entry->ftp = FTP_ESTABLISHED;
 hash_add_rcu(state_hashtable, &entry->h.hnode, new_key);
 }
}

```

```

 if (old_bkt < new_bkt) {
 spin_unlock_bh(&state_bucket_locks[new_bkt]);
 spin_unlock_bh(&state_bucket_locks[old_bkt]);
 } else {
 spin_unlock_bh(&state_bucket_locks[old_bkt]);
 spin_unlock_bh(&state_bucket_locks[new_bkt]);
 }
 }
}

/*
 * lookup_ftp_expectation - secondary hash lookup for FTP passive data flows.
 *
 * The FTP helper creates expectation entries with sport=0 (wildcard) because
 * the client's ephemeral source port is unknown when parsing the 227 response.
 * These entries live in the hash bucket keyed by:
 * get_state_hash(client_ip, server_data_ip, 0, server_data_port, TCP)
 *
 * For passive FTP, the client opens the data connection to the server:
 * src = client_ip : client_ephemeral (sport of the outgoing SYN)
 * dst = server_data_ip : server_data_port (dport of the outgoing SYN)
 *
 * So we probe the hash with dport (= server data port), which matches
 * the expectation's stored dport. The sport wildcard (0) is handled
 * inside direct_state_match() when ftp == FTP_DEFINED.
 *
 * Must be called under rcu_read_lock_bh().
 *
 * Returns the matched state_table entry, or NULL if no expectation matches.
 */
struct state_table *lookup_ftp_expectation(const struct sk_buff *skb,
 const struct iphdr *iph,
 __be16 sport, __be16 dport,
 short *reverse,
 const ipfi_flow *flow) {
 struct state_table *table_entry;
 /* Probe with sport=0 (wildcard) and the server data port as dport.
 * MUST use hash_for_each_possible_rcu - it internally applies hash_min()
 * (golden-ratio multiplicative hash) to map the key to a bucket, which is
 * the same function that hash_add_rcu used when storing the entry.
 * A manual "key & mask" bitmask computes a DIFFERENT bucket! */
 __u32 ftp_key = get_state_hash(iph->saddr, iph->daddr, 0, dport, IPPROTO_TCP);

 hash_for_each_possible_rcu(state_hashtable, table_entry, h.hnode, ftp_key) {
 if (table_entry->ftp == FTP_DEFINED) {
 if (skb_matches_state_table(skb, table_entry, reverse, iph, sport, dport,
 flow) > 0)
 return table_entry;
 }
 }
 return NULL;
}

```

## helpers/icmp\_nat.h

```

#ifndef IPFI_ICMP_NAT_H
#define IPFI_ICMP_NAT_H

```

```

#include "../filter/state/state_table.h"
#include <linux/skbuff.h>

/* Handle internal rewriting of ICMP errors that reference heavily-NATed
 * connections. hooknum expected: NF_INET_PRE_ROUTING or NF_INET_POST_ROUTING.
 * Returns 1 on success, -1 otherwise.
 */
int icmp_nat_process(struct sk_buff *skb, unsigned int hooknum);

/* Matches ICMP error payload against a state entry */
int match_icmp_error_payload(const struct sk_buff *skb,
 const struct state_table *entry, short *reverse);

/* Hashes the inner IP payload of an ICMP error for state table lookups */
int get_icmp_inner_hash(const struct sk_buff *skb, __u32 *hash_key);

#endif /* IPFI_ICMP_NAT_H */

```

## helpers/icmp\_nat.c

```

/*
 * nat/icmp_nat.c: ICMP NAT Logic
 */

#include "icmp_nat.h"
#include "../filter/state/state_table.h"
#include "../ipfi_machine.h"
#include "../ipfire.h"
#include "../nat/nat.h"
#include "../nat/nat_table.h"
#include <linux/icmp.h>
#include <linux/ip.h>
#include <linux/skbuff.h>
#include <linux/tcp.h>
#include <linux/types.h>
#include <linux/udp.h>
#include <net/checksum.h>
#include <net/ip.h>

/* Helper to safely get the inner IPv4 header and the start of the inner L4
 * header */
static int get_inner_headers(const struct sk_buff *skb,
 struct iphdr **inner_iph, void **inner_l4h) {
 struct iphdr *iph = ip_hdr(skb);
 struct icmphdr *icmph;
 int ihl = iph->ihl * 4;

 /* Ensure we have the ICMP header */
 if (!pskb_may_pull((struct sk_buff *)skb, ihl + sizeof(struct icmphdr)))
 return -1;

 icmph = (struct icmphdr *)((void *)iph + ihl);

 /* Check if it's an ICMP error type */
 if (icmph->type != ICMP_DEST_UNREACH && icmph->type != ICMP_TIME_EXCEEDED &&
 icmph->type != ICMP_PARAMETERPROB)
 return -1;
}

```

```

/* The inner IP header starts after the ICMP header (8 bytes) */
if (!pskb_may_pull((struct sk_buff *)skb,
 ihl + sizeof(struct icmphdr) + sizeof(struct iphdr)))
 return -1;

/* Recompute pointers after potential pull */
iph = ip_hdr(skb);
*inner_iph = (struct iphdr *)((void *)iph + ihl + sizeof(struct icmphdr));

/* Ensure we have enough of the inner L4 header (at least 8 bytes for ports)
 */
if (!pskb_may_pull((struct sk_buff *)skb,
 ihl + sizeof(struct icmphdr) + (*inner_iph->ihl * 4 + 8)))
 return -1;

/* Recompute pointers */
iph = ip_hdr(skb);
*inner_iph = (struct iphdr *)((void *)iph + ihl + sizeof(struct icmphdr));
*inner_l4h = (void *)(*inner_iph) + (*inner_iph->ihl * 4;

return 0;
}

static void recompute_icmp_checksum(struct sk_buff *skb,
 struct icmphdr *icmph) {
 unsigned int icmp_off = (unsigned char *)icmph - skb->data;
 unsigned int icmp_len = skb->len - icmp_off;

 icmph->checksum = 0;
 icmph->checksum = csum_fold(skb_checksum(skb, icmp_off, icmp_len, 0));
}

/*
 * icmp_nat_process: Handle NAT translation for ICMP error packets.
 * This function translates the outer IP header and the embedded inner headers
 * to correctly route an ICMP error packet across the NAT.
 */
int icmp_nat_process(struct sk_buff *skb, unsigned int hooknum) {
 struct iphdr *outer_iph, *inner_iph;
 struct icmphdr *icmph;
 void *inner_l4h;
 net_quadruplet inner_quad;
 u32 key;
 unsigned int bkt;
 struct nat_table *nt = NULL;
 enum nat_type n_type;
 int ret = -1;
 u8 inner_proto;

 if (!skb || !(skb))
 return -1;

 /* We only process IPv4 ICMP packets here */
 if (ip_hdr(skb)->protocol != IPPROTO_ICMP)
 return -1;

 if (get_inner_headers(skb, &inner_iph, &inner_l4h) < 0)
 return -1;

 outer_iph = ip_hdr(skb);
 icmph = (struct icmphdr *)((void *)outer_iph + outer_iph->ihl * 4);

```

```

/* Extract the inner network quadruplet.
 * Note: The inner packet represents the ORIGINAL traffic that caused the
 * error.
 */
memset(&inner_quad, 0, sizeof(inner_quad));
inner_quad.saddr = inner_iph->saddr;
inner_quad.daddr = inner_iph->daddr;
inner_quad.valid = 1;
/* Capture the inner protocol early to prevent TOCTOU drift */
inner_proto = inner_iph->protocol;

if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inner_quad.sport = th->source;
 inner_quad.dport = th->dest;
} else if (inner_proto == IPPROTO_UDP) {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 inner_quad.sport = uh->source;
 inner_quad.dport = uh->dest;
} else {
 /* We only handle ICMP errors for TCP/UDP inner payloads right now */
 IPFI_PRINTK("ipfire: icmp_nat: inner proto %u not supported\n",
 inner_proto);
 return -1;
}

// IPFI_PRINTK("ipfire: icmp_nat: inner quad s=%pI4:%u d=%pI4:%u prot=%u\n",
// &inner_quad.saddr, ntohs(inner_quad.sport), &inner_quad.daddr,
// ntohs(inner_quad.dport), inner_proto);

/*
 * Look up the inner tuple in the NAT tables.
 * The inner packet corresponds to the POSTNAT index because the original
 * packet has ALREADY been translated before it caused the error downstream.
 */
key = get_nat_tuple_hash(inner_quad.saddr, inner_quad.sport, inner_quad.daddr,
 inner_quad.dport, inner_proto);
bkt = key & ((1 << NAT_HASH_BITS) - 1);

rcu_read_lock_bh();

/* Try DNAT ORIG index using hash_for_each_rcu since the inner_quad hash key
 * doesn't match the client IP hashed in NAT_IDX_ORIG.
 */
n_type = NAT_DNAT;
{
 unsigned int iter_bkt;
 hash_for_each_rcu(nat_hashtables[n_type][NAT_IDX_ORIG], iter_bkt, nt,
 h.hnode) {
 if (nt->protocol == inner_proto) {
 /* Define IP Terminology for DNAT:
 * A = Client (Original Source) -> nt->old_saddr
 * B = Firewall Public IP -> nt->old_daddr
 * B' = Firewall Masq/Local IP -> nt->our_ifaddr (if SNAT'd too)
 * C = DNAT Server (Internal Target) -> nt->new_addr
 *
 * Expected inner packet (the original packet that hit the error):
 * expected_saddr is A (if pure DNAT) or B' (if DNAT + Masquerade).
 */
 __be32 expected_saddr = nt->our_ifaddr ? nt->our_ifaddr : nt->old_saddr;

```

```

/* Match Case 1: Error generated by FORWARD-direction traffic.
 * "Forward" means the traffic was flowing Client A -> Server C.
 * Server C received A's packet, rejected it, and sent an ICMP error
 * back. The Inner Quad is that original packet sent TO Server C: Src =
 * expected_saddr (A or B') Dst = nt->new_addr (C)
 */
if (inner_quad.saddr == expected_saddr &&
 inner_quad.sport == nt->old_sport &&
 inner_quad.daddr == nt->new_addr &&
 inner_quad.dport == nt->new_port) {
 if (nat_hold_rcu(nt)) {
 goto match_found;
 }
}

/* Match Case 2: Error generated by REVERSE-direction traffic.
 * "Reverse" means the traffic was flowing Server C -> Client A.
 * Client A (or the firewall B) received C's response, rejected it,
 * and sent an ICMP error back to C.
 * The Inner Quad is that original packet sent FROM Server C:
 * Src = nt->new_addr (C)
 * Dst = expected_saddr (A or B')
 */
if (inner_quad.saddr == nt->new_addr &&
 inner_quad.sport == nt->new_port &&
 inner_quad.daddr == expected_saddr &&
 inner_quad.dport == nt->old_sport) {
 if (nat_hold_rcu(nt)) {
 goto match_found;
 }
}

/* Match Case 3: DNAT forward direction in POST_ROUTING.
 * The PRE_ROUTING hook already completely translated the inner packet
 * back to its original pre-NAT state (A -> B).
 * Inner Quad is now:
 * Src = nt->old_saddr (A)
 * Dst = nt->old_daddr (B)
 */
if (hooknum == NF_INET_POST_ROUTING &&
 inner_quad.saddr == nt->old_saddr &&
 inner_quad.sport == nt->old_sport &&
 inner_quad.daddr == nt->old_daddr &&
 inner_quad.dport == nt->old_dport) {
 if (nat_hold_rcu(nt)) {
 goto match_found;
 }
}
}
}
}

/* Try SNAT POSTNAT index */
n_type = NAT_SNAT;
hlist_for_each_entry_rcu(nt, &nat_hashtables[n_type][NAT_IDX_POSTNAT][bkt],
 h_indices[NAT_IDX_POSTNAT - 1]) {
 if (nt->protocol == inner_proto) {
 /* Define IP Terminology for SNAT:
 * A = Client (Original Source) -> nt->old_saddr
 * B = Firewall Public IP -> nt->old_daddr
 * B' = Firewall Masq/Local IP -> nt->our_ifaddr or nt->new_addr
 * C = Remote Server -> nt->old_daddr

```

```

*
* Match Case: SNAT forward direction (client A to server C).
* Server C generated the ICMP error back to B'.
* Inner Quad is the packet sent TO Server C (post-SNAT):
* Src = expected_saddr (B', the translated source)
* Dst = nt->old_daddr (C)
*/
__be32 expected_saddr = nt->our_ifaddr ? nt->our_ifaddr : nt->new_addr;
if (inner_quad.saddr == expected_saddr &&
 inner_quad.sport == nt->new_port &&
 inner_quad.daddr == nt->old_daddr &&
 inner_quad.dport == nt->old_dport) {
 if (nat_hold_rcu(nt)) {
 goto match_found;
 }
}
}
}

rcu_read_unlock_bh();
return -1; /* No translation matched */

match_found:
rcu_read_unlock_bh();
/*
 * It is safe to copy fields into locals after rcu_read_unlock_bh()
 * because we held a reference count via nat_hold_rcu() inside the
 * critical section. The memory won't be freed, and these 5-tuple
 * fields are immutable once initialized.
 */
__be32 nt_old_saddr = nt->old_saddr;
__be32 nt_old_daddr = nt->old_daddr;
__be32 nt_new_addr = nt->new_addr;
__be16 nt_old_sport = nt->old_sport;
__be16 nt_old_dport = nt->old_dport;
__be16 nt_new_port = nt->new_port;
enum nat_type nt_type = nt->type;
__be32 nt_our_ifaddr = nt->our_ifaddr;

/* We must ensure skb is writable before making modifications.
 * Only need it up to the end of the inner L4 header we plan to modify.
 * Note: 'needed' is an offset from skb->data, so it implicitly covers
 * all outer headers (IP and ICMP).
 */
unsigned int needed = (unsigned char *)inner_l4h - skb->data;
if (inner_proto == IPPROTO_TCP)
 needed += sizeof(struct tcphdr);
else if (inner_proto == IPPROTO_UDP)
 needed += sizeof(struct udphdr);

if (skb_ensure_writable(skb, needed)) {
 if (net_ratelimit())
 printk(KERN_DEBUG
 "ipfire: icmp_nat: skb_ensure_writable failed (needed %u)\n",
 needed);
 nat_put(nt);
 return -1;
}

/* Re-fetch pointers after skb_ensure_writable */
outer_iph = ip_hdr(skb);

```



```

icmph = (struct icmphdr *)((void *)outer_iph + outer_iph->ihl * 4);
inner_iph = (struct iphdr *)((void *)icmph + sizeof(struct icmphdr));
inner_l4h = (void *)inner_iph + inner_iph->ihl * 4;

/* Defensive TOCTOU check: ensure the protocol hasn't been modified
 * concurrently by another packet processor before the skb was linearized */
if (unlikely(inner_iph->protocol != inner_proto ||
 inner_proto != nt->protocol)) {
 if (net_ratelimit())
 printk(KERN_WARNING "ipfire: icmp_nat: inner protocol drift detected!\n");
 nat_put(nt);
 return -1;
}

/*
 * For NF_IP_PRE_ROUTING, the error is coming from the outside returning
 * inside. For NF_IP_POST_ROUTING, the error is generated inside returning
 * outside.
 */
if (hooknum == NF_INET_PRE_ROUTING) {
 if (nt_type == NAT_SNAT) {
 /*
 * SNAT PREROUTING: Error coming back from external host (reply to SNAT
 * client). Outer destination and Inner source must change from
 * `new_addr` (or `our_ifaddr` for masquerading) to `old_saddr`.
 * Inner source port must change from `new_port` to `old_sport`.
 */
 __be32 new_ip = nt_old_saddr;
 __be32 old_ip = inner_iph->saddr;
 __be16 new_port = nt_old_sport;
 __be16 old_port = inner_quad.sport;

 /* 1. Modify outer destination IP */
 csum_replace4(&outer_iph->check, outer_iph->daddr, new_ip);
 outer_iph->daddr = new_ip;

 /* 2. Modify inner source address */
 csum_replace4(&inner_iph->check, old_ip, new_ip);
 inner_iph->saddr = new_ip;

 /* Update inner L4 checksum for pseudo-header IP change */
 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace4(&th->check, skb, old_ip, new_ip, true);
 } else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace4(&uh->check, skb, old_ip, new_ip, true);
 }

 /* 3. Modify inner source port */
 if (old_port != new_port) {
 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace2(&th->check, skb, old_port, new_port, false);
 th->source = new_port;
 } else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace2(&uh->check, skb, old_port, new_port,
 false);
 }
 }
 }
}

```

```

 uh->source = new_port;
 }
}
ret = 1;

} else if (nt_type == NAT_DNAT) {
/*
 * DNAT PREROUTING: ICMP error from/to a DNATed destination.
 */
if (inner_quad.daddr == nt_new_addr) {
/*
 * DNAT Forward direction error (Client A to Server C).
 * Inner Quad is the packet sent TO Server C.
 * We must reverse the DNAT (and any SNAT/masquerade) on the OUTER Dst
 * and on all INNER headers.
 *
 * We use inner_quad.daddr (= C) as the match key, NOT
 * outer_iph->daddr (= B', the masquerade addr). The inner packet
 * is the post-NAT original that reached the server; its destination
 * is C = nt_new_addr. The outer destination is the masq addr (B'),
 * which is unrelated to the NAT entry's new_addr.
 *
 * Ideally the outer Src after translation should be B (the firewall's
 * public address), because A thinks it was talking to B. However,
 * changing the outer Src to B (a local IP) causes the kernel's
 * fib_validate_source to reject the packet as a martian source (a
 * local address arriving on the external interface). Therefore we
 * leave the outer Src as C (the server that generated the error).
 * This is the same behaviour as iptables DNAT + MASQUERADE.
 *
 * IP Terminology:
 * A = Client (Original Source) = nt_old_saddr
 * B = Firewall Public IP = nt_old_daddr
 * B' = Firewall Masq/Local IP = nt_our_ifaddr /
 * inner_iph->saddr C = DNAT Server (Internal Target) = nt_new_addr
 *
 * As seen on the wire (arriving on firewall):
 * Outer IP: Src=C (server .49) Dst=B' (masq addr .245)
 * Inner IP: Src=B' (masq .245) Dst=C (server .49)
 *
 * After translation (forwarded to original client A):
 * Outer IP: Src=C (.49, UNCHANGED - see above) Dst=A (client .25)
 * Inner IP: Src=A (client .25) Dst=B (firewall public .245)
 */
__be32 old_inner_s = inner_iph->saddr; /* B' (masq IP) or A (client) */
__be32 new_inner_s = nt_old_saddr; /* A (original client) */
__be32 old_inner_d = inner_iph->daddr; /* C (DNAT target server) */
__be32 new_inner_d = nt_old_daddr; /* B (public FW, pre-DNAT dst) */

__be16 old_inner_sp = inner_quad.sport;
__be16 new_inner_sp = nt_old_sport;
__be16 old_inner_dp = inner_quad.dport;
__be16 new_inner_dp = nt_old_dport;

/* 1. Fix Outer Dst: B' -> A
 * Outer Src (C) is NOT changed - keep server as sender. */
csum_replace4(&outer_iph->check, outer_iph->daddr, new_inner_s);
outer_iph->daddr = new_inner_s;

/* 2. Fix Inner IP Headers */
/* Inner Src: B' -> A */

```

```

csum_replace4(&inner_iph->check, old_inner_s, new_inner_s);
inner_iph->saddr = new_inner_s;
/* Inner Dst: C -> B */
csum_replace4(&inner_iph->check, old_inner_d, new_inner_d);
inner_iph->daddr = new_inner_d;

/* 3. Fix Inner L4 Header (pseudo-header + port updates) */
if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace4(&th->check, skb, old_inner_s, new_inner_s,
 true);
 inet_proto_csum_replace4(&th->check, skb, old_inner_d, new_inner_d,
 true);
 if (old_inner_sp != new_inner_sp) {
 inet_proto_csum_replace2(&th->check, skb, old_inner_sp,
 new_inner_sp, false);
 th->source = new_inner_sp;
 }
 if (old_inner_dp != new_inner_dp) {
 inet_proto_csum_replace2(&th->check, skb, old_inner_dp,
 new_inner_dp, false);
 th->dest = new_inner_dp;
 }
} else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check) {
 inet_proto_csum_replace4(&uh->check, skb, old_inner_s, new_inner_s,
 true);
 inet_proto_csum_replace4(&uh->check, skb, old_inner_d, new_inner_d,
 true);
 if (old_inner_sp != new_inner_sp)
 inet_proto_csum_replace2(&uh->check, skb, old_inner_sp,
 new_inner_sp, false);
 if (old_inner_dp != new_inner_dp)
 inet_proto_csum_replace2(&uh->check, skb, old_inner_dp,
 new_inner_dp, false);
 }
 uh->source = new_inner_sp;
 uh->dest = new_inner_dp;
}
ret = 1;
} else if (inner_quad.saddr == nt_new_addr) {
 /*
 * DNAT Reverse direction error (Server C to Client A).
 * Inner Quad is the packet sent FROM Server C.
 * The router or A generated an ICMP error back to C.
 * Only the source needs to be un-DNAT-ed: C -> B (or B').
 *
 * IP Terminology:
 * A = Client (Original Source) = nt_old_saddr
 * B = Firewall Public IP = nt_old_daddr
 * C = DNAT Server (Internal Target) = nt_new_addr
 *
 * Here, the ICMP error is going TO C, meaning outer Dst=C.
 * Inner packet was C -> A. So inner Src=C.
 * We need to change inner Src from C -> B to match the original
 * packet as it left the firewall.
 */
 __be32 new_ip = nt_old_daddr; /* B (public Fw) */
 __be32 old_ip = inner_iph->saddr; /* C (server) */

```

```

__be16 new_port = nt_old_dport;
__be16 old_port = inner_quad.sport;

csum_replace4(&outer_iph->check, outer_iph->saddr, new_ip);
outer_iph->saddr = new_ip;

csum_replace4(&inner_iph->check, old_ip, new_ip);
inner_iph->saddr = new_ip;
if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace4(&th->check, skb, old_ip, new_ip, true);
} else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace4(&uh->check, skb, old_ip, new_ip, true);
}

if (old_port != new_port) {
 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace2(&th->check, skb, old_port, new_port,
 false);
 th->source = new_port;
 } else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace2(&uh->check, skb, old_port, new_port,
 false);
 uh->source = new_port;
 }
}
ret = 1;
}
}
} else if (hooknum == NF_INET_POST_ROUTING) {
 if (nt_type == NAT_DNAT) {
 if (inner_quad.saddr == nt_old_saddr &&
 inner_quad.sport == nt_old_sport &&
 inner_quad.daddr == nt_old_daddr &&
 inner_quad.dport == nt_old_dport) {
/*
 * DNAT POST_ROUTING Forward direction (Client A to Server C).
 * This ICMP error was already partially translated in PRE_ROUTING.
 * The inner headers are already de-NATed back to original (Src=A,
 * Dst=B). Outer Dst was already changed to A.
 *
 * IP Terminology:
 * A = Client (Original Source) = nt_old_saddr
 * B = Firewall Public IP = nt_old_daddr
 * C = DNAT Server (Internal Target) = nt_new_addr
 *
 * Now we only need to change the outer Src from C (.49, the real
 * server) to B (.245, the firewall's public address), so client A
 * sees the ICMP as coming from B – the address it thinks it is
 * talking to.
 *
 * This Cannot be done in PRE_ROUTING because B is a local IP of the
 * incoming interface (martian source algorithm drops it). Here in
 * POST_ROUTING, the packet is going OUT to A, so setting Src=B is valid
 * (masquerade-like).
 */

```

```

__be32 old_src = outer_iph->saddr; /* C (server, .49) */
__be32 new_src = nt_old_daddr; /* B (firewall public, .245) */
csum_replace4(&outer_iph->check, old_src, new_src);
outer_iph->saddr = new_src;
ret = 1;
} else {
/*
 * DNAT POSTROUTING Reverse direction (Server C to Client A).
 * Error generated internally returning to external client A.
 *
 * IP Terminology:
 * A = Client (Original Source) = nt_old_saddr
 * B = Firewall Public IP = nt_old_daddr
 * B' = Firewall Masq/Local IP = nt_our_ifaddr
 * C = DNAT Server (Internal Target) = nt_new_addr
 *
 * Outer source must change from C (server IP) back to B (or B').
 * Inner destination must change from C to B (or B').
 * Inner destination port must change from new_port to old_dport.
 */
__be32 new_ip =
 (nt_our_ifaddr) ? nt_our_ifaddr : nt_old_daddr; /* B' or B */
__be32 old_ip = inner_iph->daddr; /* C */
__be16 new_port = nt_old_dport;
__be16 old_port = inner_quad.dport;

/* 1. Modify outer source IP */
csum_replace4(&outer_iph->check, outer_iph->saddr, new_ip);
outer_iph->saddr = new_ip;

/* 2. Modify inner destination address */
csum_replace4(&inner_iph->check, old_ip, new_ip);
inner_iph->daddr = new_ip;

if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace4(&th->check, skb, old_ip, new_ip, true);
} else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace4(&uh->check, skb, old_ip, new_ip, true);
}

/* 3. Modify inner destination port */
if (old_port != new_port) {
 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace2(&th->check, skb, old_port, new_port,
 false);

 th->dest = new_port;
 } else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace2(&uh->check, skb, old_port, new_port,
 false);

 uh->dest = new_port;
 }
}
ret = 1;
}
} else if (nt_type == NAT_SNAT) {

```

```

/*
 * SNAT (or Masquerade) POSTROUTING Forward direction Error.
 * Error generated internally (by the firewall itself) returning to A.
 *
 * IP Terminology:
 * A = Client (Original Source) = nt_old_saddr
 * B = Firewall Public IP = nt_old_daddr
 * B' = Firewall Masq/Local IP = nt_our_ifaddr or nt_new_addr
 * C = Remote Server = nt_old_daddr
 */
if (inner_quad.saddr == nt_old_saddr) {
 __be32 new_ip = nt_our_ifaddr ? nt_our_ifaddr : nt_new_addr; /* B' */
 __be32 old_ip = inner_iph->saddr; /* A */
 __be16 new_port = nt_new_port;
 __be16 old_port = inner_quad.sport;

 csum_replace4(&outer_iph->check, outer_iph->saddr, new_ip);
 outer_iph->saddr = new_ip;

 csum_replace4(&inner_iph->check, old_ip, new_ip);
 inner_iph->saddr = new_ip;
 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace4(&th->check, skb, old_ip, new_ip, true);
 } else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace4(&uh->check, skb, old_ip, new_ip, true);
 }

 if (old_port != new_port) {
 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inet_proto_csum_replace2(&th->check, skb, old_port, new_port,
 false);
 th->source = new_port;
 } else {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 if (uh->check)
 inet_proto_csum_replace2(&uh->check, skb, old_port, new_port,
 false);
 uh->source = new_port;
 }
 }
 ret = 1;
}
}
}

if (ret == 1) {
 /* Recompute full ICMP checksum since inner packet changed */
 recompute_icmp_checksum(skb, icmp);

 /* Update the timer for the session tracking the error */
 nt->state = state_machine(skb, nt->state, 0);
 ipfi_entry_update_timer(&nt->h, nt->protocol, nt->state);
}

nat_put(nt);
return ret;
}

```

```

/* Matches ICMP error payload against a state entry */
int match_icmp_error_payload(const struct sk_buff *skb,
 const struct state_table *entry, short *reverse) {

 struct iphdr *iph;
 struct icmphdr *icmph;
 struct iphdr *inner_iph;
 void *inner_l4h;
 net_quadruplet inner_quad;
 u8 inner_proto;

 if (!skb || !entry || !reverse)
 return -1;

 iph = ip_hdr(skb);
 if (iph->protocol != IPPROTO_ICMP)
 return -1;

 icmph = (struct icmphdr *)((void *)iph + iph->ihl * 4);
 if (icmph->type == ICMP_DEST_UNREACH || icmph->type == ICMP_TIME_EXCEEDED ||
 icmph->type == ICMP_PARAMETERPROB) {

 if (get_inner_headers(skb, &inner_iph, &inner_l4h) < 0)
 return -1;

 memset(&inner_quad, 0, sizeof(inner_quad));
 inner_quad.saddr = inner_iph->saddr;
 inner_quad.daddr = inner_iph->daddr;
 inner_proto = inner_iph->protocol;

 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 inner_quad.sport = th->source;
 inner_quad.dport = th->dest;
 } else if (inner_proto == IPPROTO_UDP) {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 inner_quad.sport = uh->source;
 inner_quad.dport = uh->dest;
 } else {
 return -1;
 }

 if (inner_proto == entry->protocol) {
 /* Direct match of the inner packet */
 if (inner_quad.saddr == entry->saddr &&
 inner_quad.daddr == entry->daddr &&
 inner_quad.sport == entry->sport &&
 inner_quad.dport == entry->dport) {
 *reverse = 0;
 return 1;
 }
 /* Reverse match of the inner packet */
 if (inner_quad.saddr == entry->daddr &&
 inner_quad.daddr == entry->saddr &&
 inner_quad.sport == entry->dport &&
 inner_quad.dport == entry->sport) {
 *reverse = 1;
 return 1;
 }
 }
 }
}

```

```

 return -1;
}

/* Hashes the inner IP payload of an ICMP error for state table lookups */
int get_icmp_inner_hash(const struct sk_buff *skb, u32 *hash_key) {
 struct iphdr *iph;
 struct icmphdr *icmph;
 struct iphdr *inner_iph;
 void *inner_l4h;
 u8 inner_proto;
 u16 sport = 0, dport = 0;

 if (!skb || !hash_key)
 return -1;

 iph = ip_hdr(skb);
 if (iph->protocol != IPPROTO_ICMP)
 return -1;

 icmph = (struct icmphdr *)((void *)iph + iph->ihl * 4);
 if (icmph->type == ICMP_DEST_UNREACH || icmph->type == ICMP_TIME_EXCEEDED ||
 icmph->type == ICMP_PARAMETERPROB) {

 if (get_inner_headers(skb, &inner_iph, &inner_l4h) < 0)
 return -1;

 inner_proto = inner_iph->protocol;

 if (inner_proto == IPPROTO_TCP) {
 struct tcphdr *th = (struct tcphdr *)inner_l4h;
 sport = th->source;
 dport = th->dest;
 } else if (inner_proto == IPPROTO_UDP) {
 struct udphdr *uh = (struct udphdr *)inner_l4h;
 sport = uh->source;
 dport = uh->dest;
 } else {
 return -1;
 }

 *hash_key = get_state_hash(inner_iph->saddr, inner_iph->daddr, sport, dport,
 inner_proto);

 return 0;
 }
 return -1;
}

```

## netlink/message\_builder.h

```

#ifndef NETLINK_PACKET_BUILDER_H
#define NETLINK_PACKET_BUILDER_H

#include "ipfire.h"
#include "ipfi_machine.h"
#include "ipfire.h"
#include "../nat/nat_table.h"

struct sk_buff *build_packet(void *buf, int numbytes);

```



```

struct sk_buff* build_command_packet(const command *cmd);

struct sk_buff* build_info_t_packet(const ipfire_info_t *info);

struct sk_buff* build_info_t_nmsg(const struct sk_buff *skb,
 const ipfi_flow* flow,
 const struct response *res,
 const struct info_flags *flags,
 int *err);

struct sk_buff* build_dnat_t_packet(const struct nat_table *dt);

struct sk_buff* build_snat_t_packet(const struct nat_table *dt);

struct sk_buff* build_dnat_info_packet(const struct dnat_info *dni);

struct sk_buff* build_snat_info_packet(const struct snat_info *sni);

struct sk_buff* build_state_packet(const struct state_table *st);

struct sk_buff* build_state_info_packet(const struct state_info *sti);

struct sk_buff* build_kstats_packet(const struct kernel_stats *kst);

struct sk_buff* build_kstats_light_packet(const struct kstats_light *kctl);

struct sk_buff* build_ktable_info_packet(const struct ktables_usage* ktu);

#endif

```

## netlink/message\_builder.c

```

#include "message_builder.h"

/*! returns a new struct sk_buff */
struct sk_buff *build_packet(void *buf, int numbytes) {
 struct sk_buff *skb_to_user;
 struct nlmsg_hdr *nlh;
 void *data;
 int len = NLMSG_SPACE(numbytes);

 skb_to_user = alloc_skb(len, GFP_ATOMIC);

 if (skb_to_user == NULL)
 return NULL;

 nlh = nlmsg_put(skb_to_user, 0, 0, 0, numbytes, 0);

 if (NLMSG_OK(nlh, NLMSG_LENGTH(numbytes))) {
 nlh->nlmsg_len = NLMSG_LENGTH(numbytes);
 data = NLMSG_DATA(nlh);
 memcpy(data, buf, numbytes);
 } else {
 printk("IPFIRE: NLMSG_OK failed: NLMSG_LENGTH(%d) is %d, "
 "NLMSG_SPACE(numbytes %d) is %d in build_packet(), "
 "ipfi_netl_packet_builder.c\n",
 len, NLMSG_LENGTH(len), numbytes, NLMSG_SPACE(numbytes));
 }
}

```

```

 return NULL;
}
return skb_to_user;

printk("IPFIRE: nlmsg failure in build_packet() - NLMSG_SPACE:%dbytes - "
 "netlink header:%lubytes - data length:%dbytes - NLMSG_LENGTH(data "
 "len): %dbytes - ipfi_netl_packet_builder.c\n",
 len, sizeof(struct nlmsgghdr), numbytes, NLMSG_LENGTH(numbytes));
return NULL;
}

struct sk_buff *build_command_packet(const command *cmd) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)cmd, sizeof(command));
 return skb_to_user;
}

struct sk_buff *build_info_t_packet(const ipfire_info_t *info) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)info, sizeof(ipfire_info_t));
 return skb_to_user;
}

struct sk_buff *build_info_t_nlmsg(const struct sk_buff *skb,
 const ipfi_flow *flow,
 const struct response *res,
 const struct info_flags *flags, int *err) {

 struct sk_buff *nl_skb;
 struct nlmsgghdr *nlh;
 ipfire_info_t *msg;
 int size = sizeof(*msg);
 struct iphdr *iph = ip_hdr(skb); // checked at the origin
 nl_skb = nlmsg_new(size, GFP_ATOMIC);
 if (!nl_skb) {
 *err = ENOMEM;
 } else {
 nlh = nlmsg_put(nl_skb, 0, 0, 0, size, 0);
 if (!nlh) {
 kfree_skb(nl_skb);
 *err = EMSGSIZE;
 return NULL;
 }
 msg = nlmsg_data(nlh);
 memset(msg, 0, sizeof(*msg));
 if (iph) {
 msg->packet.ip.protocol = iph->protocol;
 msg->packet.ip.saddr = iph->saddr;
 msg->packet.ip.daddr = iph->daddr;
 }

 switch (iph ? iph->protocol : 0) {
 case IPPROTO_TCP:
 memcpy(&msg->packet.transport_header.tcphead, (void *)iph + iph->ihl * 4,
 sizeof(struct tcphdr));
 break;
 case IPPROTO_UDP:
 memcpy(&msg->packet.transport_header.udphdr, (void *)iph + iph->ihl * 4,
 sizeof(struct udphdr));
 break;
 case IPPROTO_ICMP:
 memcpy(&msg->packet.transport_header.icmphead, (void *)iph + iph->ihl * 4,

```

```

 sizeof(struct icmp_hdr));
 break;
case IPPROTO_IGMP:
 memcpy(&msg->packet.transport_header.igmphead, (void *)iph + iph->ihl * 4,
 sizeof(struct igmp_hdr));
 break;
default:
 break;
}

msg->response = *res;
msg->flags = *flags;
msg->flags.direction =
 flow->direction; /* Ensure direction is set from flow */
if (flow->in)
 strncpy(msg->netdevs.in_devname, flow->in->name, IFNAMSIZ - 1);
if (flow->out)
 strncpy(msg->netdevs.out_devname, flow->out->name, IFNAMSIZ - 1);
}
return nl_skb;
}

struct sk_buff *build_dnat_t_packet(const struct nat_table *dt) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)dt, sizeof(*dt));
 return skb_to_user;
}

struct sk_buff *build_snat_t_packet(const struct nat_table *st) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)st, sizeof(*st));
 return skb_to_user;
}

struct sk_buff *build_snat_info_packet(const struct snat_info *sni) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)sni, sizeof(*sni));
 return skb_to_user;
}

struct sk_buff *build_dnat_info_packet(const struct dnat_info *dni) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)dni, sizeof(*dni));
 return skb_to_user;
}

struct sk_buff *build_state_packet(const struct state_table *st) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)st, sizeof(*st));
 return skb_to_user;
}

struct sk_buff *build_ktable_info_packet(const struct ktables_usage *ktu) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)ktu, sizeof(*ktu));
 return skb_to_user;
}

struct sk_buff *build_state_info_packet(const struct state_info *sti) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)sti, sizeof(*sti));
}

```

```

 return skb_to_user;
}

struct sk_buff *build_kstats_packet(const struct kernel_stats *kst) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)kst, sizeof(*kst));
 return skb_to_user;
}

struct sk_buff *build_kstats_light_packet(const struct kstats_light *kstl) {
 struct sk_buff *skb_to_user = NULL;
 skb_to_user = build_packet((void *)kstl, sizeof(*kstl));
 return skb_to_user;
}

```

## netlink/netlink\_data.c

```

/* netlink/netlink_data.c: Netlink data channel for ipfire-wall */

#include "../logging/log.h"
#include "globals.h"
#include "ipfire.h"
#include "netlink/ipfi_netl.h"
#include <linux/module.h>
#include <linux/netlink.h>
#include <linux/skbuff.h>

#include "globals.h"

int process_data_received(struct sk_buff *skb) {
 if (nlmsg_len(nlmsg_hdr(skb)) < sizeof(listener_message)) {
 IPFI_PRINTK("IPFIRE: process_data_received(): netlink message too small "
 "for listener_message (%d < %lu)\n",
 nlmsg_len(nlmsg_hdr(skb)), sizeof(listener_message));
 return -EINVAL;
 }

 listener_message *listener_mess = (listener_message *)extract_data(skb);
 if (listener_mess == NULL)
 return -1;

 if (listener_mess->message == STARTING) {
 return 0;
 } else if (listener_mess->message == EXITING) {
 return 0;
 }
 return 0;
}

int is_to_send(const struct sk_buff *skb, const struct ipfire_options *fwopts,
 const struct response *res, const ipfi_flow *flow,
 const struct info_flags *flags) {
 if (res->nolog)
 return 0;

 if (smartlog_func != NULL)
 return smartlog_func(skb, res, flow, flags);
 else {

```

```

 if (fwopts->loguser >= 6)
 return 1;
 if (res->verdict == IPFI_IMPLICIT && fwopts->loguser >= 2) {
 return 1;
 } else if (res->verdict == IPFI_ACCEPT && fwopts->loguser >= 5) {
 return 1;
 } else if (res->verdict == IPFI_DROP && fwopts->loguser >= 4) {
 return 1;
 }
}
return 0;
}

unsigned long long update_sent_counter(int direction) {
 unsigned long long sent = 0;
 switch (direction) {
 case IPFI_INPUT:
 sent = this_cpu_inc_return(ipfi_counters->in_sent_touser) - 1;
 break;
 case IPFI_OUTPUT:
 sent = this_cpu_inc_return(ipfi_counters->out_sent_touser) - 1;
 break;
 case IPFI_FWD:
 sent = this_cpu_inc_return(ipfi_counters->fwd_sent_touser) - 1;
 break;
 case IPFI_INPUT_PRE:
 sent = this_cpu_inc_return(ipfi_counters->pre_sent_touser) - 1;
 break;
 case IPFI_OUTPUT_POST:
 sent = this_cpu_inc_return(ipfi_counters->post_sent_touser) - 1;
 break;
 default:
 sent = 0;
 }
 IPFI_STAT_INC(sent_tou);
 return sent;
}

int skb_send_to_user(struct sk_buff *skb, int type_of_message) {
 struct sock *socket = NULL;
 pid_t pid = 0;
 int ret;
 if (type_of_message == CONTROL_DATA) {
 socket = sknl_ipfi_control;
 pid = userspace_control_pid;
 } else if (type_of_message == LISTENER_DATA) {
 socket = sknl_ipfi_data;
 pid = userspace_data_pid;
 } else if (type_of_message == GUI_NOTIF_DATA) {
 socket = sknl_ipfi_gui_notifier;
 pid = userspace_control_pid;
 }
 if (socket == NULL || pid == 0) {
 if (skb)
 kfree_skb(skb);
 return -1;
 }
 if (skb == NULL)
 return -1;
 ret = send_data_to_user(skb, pid, socket);
 if (ret < 0)

```

```

 return ret;
return 0;
}

```

## netlink/rule\_sync.c

```

/* netlink/rule_sync.c: Rule synchronization for ipfire-wall */

#include <linux/module.h>
#include <linux/slab.h>
#include <linux/list.h>
#include <linux/netdevice.h>
#include "ipfire.h"
#include "netlink/ipfi_netl.h"
#include "ipfi_machine.h"
#include "globals.h"

int add_rule_to_list_by_command(command *cmd_with_rule)
{
 ipfire_rule *newrule;
 newrule = (ipfire_rule *) kmalloc(sizeof(ipfire_rule), GFP_KERNEL);
 if(newrule == NULL || cmd_with_rule == NULL)
 return -1;
 memcpy(newrule, &(cmd_with_rule->content.rule), sizeof(ipfire_rule));

 /* ifindex resolution removed for namespace robustification */
 // if (newrule->nflags.indev) {
 // struct net_device *dev = dev_get_by_name(&init_net, newrule-
>devpar.in_devname);
 // if (dev) {
 // newrule->devpar.in_ifindex = dev->ifindex;
 // dev_put(dev);
 // } else {
 // newrule->devpar.in_ifindex = -1;
 // }
 // }
 // if (newrule->nflags.outdev) {
 // struct net_device *dev = dev_get_by_name(&init_net, newrule-
>devpar.out_devname);
 // if (dev) {
 // newrule->devpar.out_ifindex = dev->ifindex;
 // dev_put(dev);
 // } else {
 // newrule->devpar.out_ifindex = -1;
 // }
 // }

 INIT_LIST_HEAD(&newrule->list);
 spin_lock(&rulelist_lock);

 if (newrule->direction == IPFI_INPUT)
 {
 if (newrule->nflags.policy == ACCEPT)
 list_add_tail_rcu(&(newrule->list), &in_acc.list);
 else
 list_add_tail_rcu(&(newrule->list), &in_drop.list);
 }
 else if (newrule->direction == IPFI_OUTPUT)

```

```

{
 if (newrule->nflags.policy == ACCEPT)
 list_add_tail_rcu(&(newrule->list), &out_acc.list);
 else if (newrule->nflags.policy == TRANSLATION)
 {
 if (newrule->owner == 0)
 list_add_tail_rcu(&(newrule->list), &translation_out.list);
 }
 else
 list_add_tail_rcu(&(newrule->list), &out_drop.list);
}
else if (newrule->direction == IPFI_FWD)
{
 if (newrule->nflags.policy == ACCEPT)
 list_add_tail_rcu(&(newrule->list), &fwd_acc.list);
 else
 list_add_tail_rcu(&(newrule->list), &fwd_drop.list);
}
else if (newrule->direction == IPFI_INPUT_PRE && newrule->nat)
{
 if (newrule->owner == 0)
 list_add_tail_rcu(&(newrule->list), &translation_pre.list);
}
else if ((newrule->direction == IPFI_OUTPUT_POST) && ((newrule->nat) || (newrule->masquerade)))
{
 if (newrule->owner == 0) {
 if (newrule->nat & newrule->snat)
 list_add_tail_rcu(&(newrule->list), &translation_post.list);
 else if (newrule->masquerade)
 list_add_tail_rcu(&(newrule->list), &masquerade_post.list);
 }
}
spin_unlock(&rulelist_lock);
return 0;
}

int rule_not_loaded(const command* cmd)
{
 ipfire_rule *newrule = (ipfire_rule *) kcalloc(sizeof(ipfire_rule), GFP_KERNEL);
 if(newrule == NULL)
 return 0;
 memcpy(newrule, &(cmd->content.rule), sizeof(ipfire_rule));

 if (!find_rules(&in_acc, newrule) || !find_rules(&in_drop, newrule) ||
 !find_rules(&fwd_drop, newrule) || !find_rules(&fwd_acc, newrule) ||
 !find_rules(&out_acc, newrule) || !find_rules(&out_drop, newrule) ||
 !find_rules(&translation_pre, newrule) || !find_rules(&translation_post, newrule)
 ||
 !find_rules(&translation_out, newrule) || !find_rules(&masquerade_post, newrule))
 {
 kfree(newrule);
 return 0;
 }
 kfree(newrule);
 return 1;
}

int find_rules(const ipfire_rule *rlist, const ipfire_rule * rule)
{
 ipfire_rule *tmp;

```

```

int len = sizeof(deviceparams) + sizeof(ipparams) +
 sizeof(transparams) + sizeof(icmp_params) +
 sizeof(netflags) + sizeof(meanings) +
 sizeof(u8) + sizeof(u32) + sizeof(u16) +
 sizeof(struct packet_manip);

rcu_read_lock();
list_for_each_entry_rcu(tmp, &rlist->list, list)
{
 if (memcmp(tmp, rule, len) == 0)
 {
 rcu_read_unlock();
 return 0;
 }
}
rcu_read_unlock();
return -1;
}

int manage_rule(command * rule_from_user)
{
 short add = 0;
 uid_t rule_owner;
 if(rule_from_user == NULL)
 return -1;
 rule_owner = rule_from_user->content.rule.owner;

 if ((rule_owner != 0) && (!fwopts.user_allowed))
 rule_from_user->cmd = RULE_NOT_ADDED_NO_PERM;
 else if (rule_not_loaded(rule_from_user))
 add = 1;
 else
 rule_from_user->cmd = RULE_ALREADY_PRESENT;

 if (send_back_command(rule_from_user) < 0)
 return -1;

 if (add > 0)
 add_rule_to_list_by_command(rule_from_user);
 return 0;
}

void update_ifindex_in_rules(const char *name, int new_index)
{
 /* No-op: ifindex is no longer used for rule matching */
 return;
}

int flush_ruleset(uid_t userspace_commander, int flush_com)
{
 unsigned l = 0, m = 0, n = 0, o = 0, p = 0,
 q = 0, r = 0, s = 0, t = 0, u = 0;

 if ((flush_com == FLUSH_RULES) || (flush_com == FLUSH_DENIAL_RULES))
 {
 l = free_rules(&in_drop, userspace_commander);
 m = free_rules(&out_drop, userspace_commander);
 n = free_rules(&fwd_drop, userspace_commander);

 if (flush_com == FLUSH_DENIAL_RULES) {
 free_state_tables();
 }
 }
}

```



```

 }
}
if ((flush_com == FLUSH_RULES) || (flush_com == FLUSH_PERMISSION_RULES))
{
 o = free_rules(&in_acc, userspace_commander);
 p = free_rules(&out_acc, userspace_commander);
 q = free_rules(&fwd_acc, userspace_commander);
}
if (((flush_com == FLUSH_RULES) || (flush_com == FLUSH_TRANSLATION_RULES)) &&
(userspace_commander == 0))
{
 r = free_rules(&translation_pre, 0);
 s = free_rules(&translation_out, 0);
 t = free_rules(&translation_post, 0);
 u = free_rules(&masquerade_post, 0);
}
return l + m + n + o + q + p + r + s + t + u;
}

void free_rule_rcu_call(struct rcu_head* head)
{
 ipfire_rule* ipfirule_entry = container_of(head, ipfire_rule, rule_rcuh);
 kfree(ipfirule_entry);
}

int free_rules(ipfire_rule *rulelist, uid_t user)
{
 int rules_freed = 0;
 ipfire_rule *rule;
 spin_lock(&rulelist_lock);
 list_for_each_entry(rule, &rulelist->list, list)
 {
 if ((rule->owner == user) || (user == 0))
 {
 list_del_rcu(&rule->list);
 call_rcu(&rule->rule_rcuh, free_rule_rcu_call);
 rules_freed++;
 }
 }
 spin_unlock(&rulelist_lock);
 return rules_freed;
}

```

## netlink/netlink\_control.c

```

/* netlink/netlink_control.c: Netlink control channel for ipfire-wall */

#include "globals.h"
#include "ipfire.h"
#include "logging/log.h"
#include "message_builder.h"
#include "netlink/ipfi_netl.h"
#include <linux/delay.h>
#include <linux/module.h>
#include <linux/netlink.h>
#include <linux/skbuff.h>
#include <linux/slab.h>
#include <linux/user_namespace.h>

```

```

void fill_dnat_info(struct dninfo *dninfo, const struct nat_table *dnnt) {
 dninfo->saddr = dnnt->old_saddr;
 dninfo->daddr = dnnt->old_daddr;
 dninfo->sport = dnnt->old_sport;
 dninfo->dport = dnnt->old_dport;
 dninfo->newdport = dnnt->new_port;
 dninfo->newdaddr = dnnt->new_addr;

 dninfo->id = dnnt->rule_id;
 dninfo->timeout = (dnnt->h.timer.expires - jiffies) / HZ;
 dninfo->direction = dnnt->direction;
 dninfo->state.state = dnnt->state;
 strncpy(dninfo->in_devname, dnnt->in_devname, IFNAMSIZ);
 strncpy(dninfo->out_devname, dnnt->out_devname, IFNAMSIZ);
 dninfo->protocol = dnnt->protocol;
}

void fill_sninfo(struct sninfo *sninfo, const struct nat_table *snnt) {
 sninfo->saddr = snnt->old_saddr;
 sninfo->daddr = snnt->old_daddr;
 sninfo->sport = snnt->old_sport;
 sninfo->dport = snnt->old_dport;
 sninfo->newsport = snnt->new_port;
 sninfo->newsaddr = snnt->new_addr;

 sninfo->id = snnt->rule_id;
 sninfo->timeout = (snnt->h.timer.expires - jiffies) / HZ;
 sninfo->direction = snnt->direction;
 sninfo->state.state = snnt->state;
 strncpy(sninfo->in_devname, snnt->in_devname, IFNAMSIZ);
 strncpy(sninfo->out_devname, snnt->out_devname, IFNAMSIZ);
 sninfo->protocol = snnt->protocol;
}

pid_t get_sender_pid(const struct sk_buff *skbuff);

void *extract_data(struct sk_buff *skb) {
 struct nlmsg_hdr *nlh;
 int nlmsglen, skblen;
 void *data_pointer;

 skblen = skb->len;
 if (skblen < sizeof(*nlh)) {
 IPFI_PRINTK("IPFIRE: skblen < sizeof(*nlh) in extract_data()\n");
 return NULL;
 }
 #if LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 24)
 nlh = (struct nlmsg_hdr *)skb->data;
 #else
 nlh = nlmsg_hdr(skb);
 #endif
 nlmsglen = nlh->nlmsg_len;
 if (nlmsglen < sizeof(*nlh) || skblen < nlmsglen) {
 IPFI_PRINTK("IPFIRE: nlmsglen < sizeof(*nlh) || skblen < nlmsglen in "
 "extract_data()\n");
 return NULL;
 }

 data_pointer = NLMSG_DATA(nlh);
 return data_pointer;
}

```

```

}

int process_control_received(struct sk_buff *skb) {
 uid_t commander;
 short command_id;
 int ret;

 if (nlmsg_len(nlmsg_hdr(skb)) < sizeof(command)) {
 IPFI_PRINTK("IPFIRE: process_control_received(): netlink message too small "
 "for command (%d < %lu)\n",
 nlmsg_len(nlmsg_hdr(skb)), sizeof(command));
 return -EINVAL;
 }

 command *cmd_from_user = (command *)extract_data(skb);

 if (cmd_from_user == NULL) {
 IPFI_PRINTK("IPFIRE: process_control_received(): error extracting data "
 "from socket buffer\n");
 return -1;
 }

 memcpy(&command_id, (short *)cmd_from_user, sizeof(short));

 commander = from_kuid(&init_user_ns, NETLINK_CREDS(skb)->uid);

 if (send_acknowledgement(userspace_control_pid) < 0) {
 IPFI_PRINTK(
 "IPFIRE: failed to send acknowledgement to pid %d!! This is strange!\n"
 "IPFIRE: I will not process the control command!",
 userspace_control_pid);
 return -1;
 }

 if (command_id == HELLO) {
 IPFI_PRINTK("IPFIRE: hello from userspace \"%s\", pid %d, uid %d.\n",
 cmd_from_user->content.fwsizes.uspace_firename,
 userspace_control_pid, cmd_from_user->content.fwsizes.uid);
 return initial_handshake(cmd_from_user, userspace_uid);
 } else if (command_id == SIMPLE_GOODBYE)
 return simple_exit();
 else if ((command_id == OPTIONS) & (cmd_from_user->options))
 return set_firewall_options(cmd_from_user, commander);
 else if (command_id == PRINT_RULES)
 return send_rule_list_to_userspace();
 else if (command_id == EXITING)
 return do_userspace_exit_tasks(commander);
 else if (command_id == START_LOGUSER) {
 loguser_enabled = 1;
 IPFI_PRINTK("IPFIRE: enabling log to userspace.\n");
 return 0;
 } else if (command_id == STOP_LOGUSER) {
 loguser_enabled = 0;
 IPFI_PRINTK("IPFIRE: disabling log to userspace.\n");
 return 0;
 } else if (command_id == IS_LOGUSER_ENABLED) {
 ret = send_loguser_enabled(loguser_enabled);
 return ret;
 } else if (command_id == FLUSH_RULES) {
 ret = flush_ruleset(commander, FLUSH_RULES);
 tell_user_howmany_rules_flushed(ret);
 return ret;
 }
}

```

```

} else if (command_id == FLUSH_PERMISSION_RULES) {
 ret = flush_ruleset(commander, FLUSH_PERMISSION_RULES);
 tell_user_howmany_rules_flushed(ret);
 return ret;
} else if (command_id == FLUSH_DENIAL_RULES) {
 ret = flush_ruleset(commander, FLUSH_DENIAL_RULES);
 tell_user_howmany_rules_flushed(ret);
 return ret;
} else if (command_id == FLUSH_TRANSLATION_RULES) {
 ret = flush_ruleset(commander, FLUSH_TRANSLATION_RULES);
 tell_user_howmany_rules_flushed(ret);
 return ret;
} else if (command_id == PRINT_STATE_TABLE)
 return send_tables();

else if (command_id == PRINT_DNAT_TABLE)
 return send_dnat_tables();
else if (command_id == PRINT_SNAT_TABLE)
 return send_snat_tables();
else if (command_id == PRINT_KTABLES_USAGE)
 return send_ktables_usage();

else if (command_id == KSTATS_REQUEST)
 return send_kstats();
else if (command_id == KSTATS_LIGHT_REQUEST)
 return send_kstats_light();
else if (command_id == KSTRUCT_SIZES)
 return send_struct_sizes();
else if (command_id == SMART_SIMPLE || command_id == SMART_STATE)
 return register_log_function(command_id);
else if (command_id == SMARTLOG_TYPE)
 return send_smartlog_type();

else if (command_id == START_NOTIFIER) {
 gui_notifier_enabled = 1;
 return 0;
} else if (command_id == STOP_NOTIFIER) {
 gui_notifier_enabled = 0;
 return 0;
} else if (cmd_from_user->is_rule) {
 if (cmd_from_user->content.rule.owner != commander) {
 cmd_from_user->content.rule.owner = commander;
 }
 manage_rule(cmd_from_user);
 return IPFI_ACCEPT;
} else {
 IPFI_PRINTK("command id \"%d\" not recognized!\n", command_id);
 return -1;
}
return -1;
}

int send_acknowledgement(pid_t uspace_pid) {
 int ret = -1;
 command *acknow = (command *)kmalloc(sizeof(command), GFP_KERNEL);
 if (acknow != NULL) {
 memset(acknow, 0, sizeof(command));
 acknow->cmd = ACKNOWLEDGEMENT;
 ret = send_back_command(acknow);
 kfree(acknow);
 }
}

```

```

 return ret;
}

int send_loguser_enabled(int logu_enabled) {
 int ret = -1;
 command *info_logu = (command *)kmalloc(sizeof(command), GFP_KERNEL);
 if (info_logu != NULL) {
 memset(info_logu, 0, sizeof(command));
 info_logu->cmd = IS_LOGUSER_ENABLED;
 info_logu->anumber = logu_enabled;
 ret = send_back_command(info_logu);
 kfree(info_logu);
 }
 return ret;
}

void get_struct_sizes(struct firesizes *fsz) {
 if (fsz == NULL)
 return;
 fsz->rulesize = sizeof(ipfire_rule);
 fsz->infosize = sizeof(ipfire_info_t);
 fsz->cmdsized = sizeof(command);
 fsz->statesize = sizeof(struct state_table);
 fsz->snatsize = sizeof(struct nat_table);
 fsz->dnatsize = sizeof(struct nat_table);
 fsz->loginfosize = sizeof(struct ipfire_loginfo);
}

void fill_firesizes_with_kernel_values(command *cmd, size_t krulesize,
 size_t kinfosize, size_t kcmdsized,
 uid_t uspace_uid) {
 struct firesizes fsz = cmd->content.fwsizes;
 fsz.rulesize = krulesize;
 fsz.infosize = kinfosize;
 fsz.cmdsized = kcmdsized;
 fsz.uid = uspace_uid;
}

int initial_handshake(command *hello, uid_t userspace_uid) {
 int result;
 struct firesizes *kernel_sizes;
 struct firesizes fsz = hello->content.fwsizes;
 kernel_sizes =
 (struct firesizes *)kmalloc(sizeof(struct firesizes), GFP_KERNEL);

 if (kernel_sizes == NULL) {
 IPFI_PRINTK("IPFIRE: failed to allocate memory for struct firesizes in "
 "initial_handshake()\n");
 return -1;
 }
 get_struct_sizes(kernel_sizes);
 result = HELLO_OK;
 if (fsz.rulesize != kernel_sizes->rulesize)
 result = H_RULESIZE_MISMATCH;
 if (fsz.infosize != kernel_sizes->infosize)
 result = H_INFOSIZE_MISMATCH;
 if (fsz.cmdsized != kernel_sizes->cmdsized)
 result = H_CMDSIZE_MISMATCH;

 hello->cmd = result;
}

```

```

if (result != HELLO_OK) {
 IPFI_PRINTK("HELLO FAILED: %d.\n", result);
 fill_firesizes_with_kernel_values(hello, kernelsizes->rulesize,
 kernelsizes->infosize,
 kernelsizes->cmdsiz, userspace_uid);
}

kfree(kernelsizes);

if (send_back_command(hello) < 0) {
 IPFI_PRINTK("IPFIRE: error sending hello response to userspace!\n");
 return -1;
}
return 1;
}

int send_struct_sizes(void) {
 int ret = -1;
 command *cmdsizes = (command *)kmalloc(sizeof(command), GFP_KERNEL);

 if (cmdsizes != NULL) {
 memset(cmdsizes, 0, sizeof(command));
 get_struct_sizes(&(cmdsizes->content.fwsizes));
 ret = send_back_command(cmdsizes);
 kfree(cmdsizes);
 }
 return ret;
}

int send_smartlog_type(void) {
 int ret = -1;
 struct sk_buff *to_user;
 command *logtype = (command *)kmalloc(sizeof(command), GFP_KERNEL);
 if (logtype != NULL) {
 memset(logtype, 0, sizeof(command));
 logtype->cmd = SMARTLOG_TYPE;
 logtype->is_rule = 0;
 logtype->anumber = fwopts.loguser;
 to_user = build_command_packet(logtype);
 if (to_user != NULL)
 ret = skb_send_to_user(to_user, CONTROL_DATA);
 kfree(logtype);
 }
 return ret;
}

int set_firewall_options(command *cmd, const uid_t commander) {
 if (commander == 0) {
 check_max_timeout_values(cmd);
 fwopts.nat = cmd->nat;
 fwopts.masquerade = cmd->masquerade;
 fwopts.state = cmd->stateful;
 fwopts.all_stateful = cmd->all_stateful;
 fwopts.user_allowed = cmd->user_allowed;
 fwopts.noflush_on_exit = cmd->noflush_on_exit;
 fwopts.snatted_lifetime = cmd->snatted_lifetime;
 fwopts.dnatted_lifetime = cmd->dnatted_lifetime;
 fwopts.state_lifetime = cmd->state_lifetime;
 fwopts.max_loginfo_entries = cmd->max_loginfo_entries;
 fwopts.loginfo_lifetime = cmd->loginfo_lifetime;
 fwopts.max_nat_entries = cmd->max_nat_entries;
 }
}

```

```

fwopts.max_state_entries = cmd->max_state_entries;
state_lifetime = fwopts.state_lifetime;
setup_shutd_state_lifetime = cmd->setup_shutd_state_lifetime;
fwopts.setup_shutd_state_lifetime = cmd->setup_shutd_state_lifetime;
fwopts.loguser = cmd->loguser;
loginfo_lifetime = fwopts.loginfo_lifetime;
max_loginfo_entries = fwopts.max_loginfo_entries;
max_state_entries = fwopts.max_state_entries;
fwopts.loglevel = cmd->loglevel;

{
 char state_lim[16], nat_lim[16];
 if (fwopts.state)
 snprintf(state_lim, sizeof(state_lim), "%u", max_state_entries);
 else
 strcpy(state_lim, "disabled");

 if (fwopts.nat)
 snprintf(nat_lim, sizeof(nat_lim), "%lu",
 (unsigned long)fwopts.max_nat_entries);
 else
 strcpy(nat_lim, "disabled");

 IPFI_PRINTK("IPFIRE: default policy: %s. tables limits: state: %s, nat: "
 "%s, log info: %u\n",
 kstats.policy == IPFI_DROP ? "drop" : "accept", state_lim,
 nat_lim, max_loginfo_entries);
}
}

if (fwopts.loglevel > 5) {
 print_nat_entries_memory_usage();
 print_state_entries_memory_usage();
}
register_log_function(fwopts.loguser);

opts_to_cmd(cmd);
if (fwopts.loglevel > 2)
 print_command(cmd);
if (send_back_command(cmd) < 0)
 IPFI_PRINTK("IPFIRE: error sending ack to userspace!\n");
return IPFI_ACCEPT;
}

int register_log_function(int loglevel) {
 fwopts.loguser = loglevel;
 switch (loglevel) {
 case SMART_LOG:
 smartlog_func = smart_log;
 break;

 case SMART_LOG_WITH_STATE_CHECK:
 smartlog_func = smart_log_with_state_check;
 break;
 default:
 smartlog_func = NULL;
 break;
 }
 return 0;
}

```

```

void check_max_timeout_values(command *cmd) {
 if (cmd->snatted_lifetime > MAX_TIMEOUT)
 cmd->snatted_lifetime = MAX_TIMEOUT;
 if (cmd->dnatted_lifetime > MAX_TIMEOUT)
 cmd->dnatted_lifetime = MAX_TIMEOUT;
 if (cmd->state_lifetime > MAX_TIMEOUT)
 cmd->state_lifetime = MAX_TIMEOUT;
 if (cmd->setup_shutd_state_lifetime > MAX_TIMEOUT)
 cmd->setup_shutd_state_lifetime = MAX_TIMEOUT;

 /* Logininfo bounds (technical consensus for system safety/usability) */
 if (cmd->logininfo_lifetime < 5) {
 cmd->logininfo_lifetime = 5;
 IPFI_PRINTK("IPFIRE: logininfo_lifetime too low, adjusted to 5s\n");
 } else if (cmd->logininfo_lifetime > 600) {
 cmd->logininfo_lifetime = 600;
 IPFI_PRINTK("IPFIRE: logininfo_lifetime %lu > 600s, adjusted to 600s\n",
 cmd->logininfo_lifetime);
 }

 if (cmd->max_logininfo_entries < 64) {
 cmd->max_logininfo_entries = 64;
 IPFI_PRINTK("IPFIRE: max_logininfo_entries too low, adjusted to 64\n");
 } else if (cmd->max_logininfo_entries > 65536) {
 cmd->max_logininfo_entries = 65536;
 IPFI_PRINTK("IPFIRE: max_logininfo_entries %lu too high, adjusted to 65536\n",
 cmd->max_logininfo_entries);
 }
}

void opts_to_cmd(command *cmd) {
 cmd->nat = fwopts.nat;
 cmd->masquerade = fwopts.masquerade;
 cmd->stateful = fwopts.stateful;
 cmd->user_allowed = fwopts.user_allowed;
 cmd->snatted_lifetime = fwopts.snatted_lifetime;
 cmd->dnatted_lifetime = fwopts.dnatted_lifetime;
 cmd->state_lifetime = fwopts.state_lifetime;
 cmd->setup_shutd_state_lifetime = fwopts.setup_shutd_state_lifetime;
 cmd->noflush_on_exit = fwopts.noflush_on_exit;
 cmd->loglevel = fwopts.loglevel;
 cmd->loguser = fwopts.loguser;
 cmd->max_logininfo_entries = fwopts.max_logininfo_entries;
 cmd->logininfo_lifetime = fwopts.logininfo_lifetime;
 cmd->max_nat_entries = fwopts.max_nat_entries;
 cmd->max_state_entries = max_state_entries;
}

int print_command(const command *opt) {
 if (opt->nat)
 IPFI_PRINTK("| NAT: ENABLED | ");
 else
 IPFI_PRINTK("| NAT: DISABLED | ");
 if (opt->masquerade)
 IPFI_PRINTK("MASQUERADE: ENABLED | ");
 else
 IPFI_PRINTK("MASQUERADE: DISABLED | ");
 if (opt->stateful)
 IPFI_PRINTK("STATEFUL FIREWALL: ENABLED |\n");
 else
 IPFI_PRINTK("STATEFUL FIREWALL: DISABLED |\n");
}

```



```

 if (opt->user_allowed)
 IPFI_PRINTK("USERS ARE ALLOWED TO INSERT THEIR RULES.\n");
 else
 IPFI_PRINTK("USERS ARE NOT ALLOWED TO INSERT THEIR RULES.\n");
 if (opt->noflush_on_exit)
 IPFI_PRINTK("RULES NOT FLUSHED ON EXIT | ");
 else
 IPFI_PRINTK("RULES FLUSHED ON EXIT | ");
 IPFI_PRINTK("LOGLEVEL: %u | LOGUSER: %u |\n", opt->loglevel, opt->loguser);
 return 0;
}

void print_state_entries_memory_usage(void) {
 IPFI_PRINTK("IPFIRE: state table entries lifetime is %u seconds.\n",
 state_lifetime);
 IPFI_PRINTK("IPFIRE: max. number of entries is %u.\n", max_state_entries);
 IPFI_PRINTK("IPFIRE: size of a source nat table is %zu bytes, so"
 " total memory occupied by snat entries is %zu KB.\n",
 sizeof(struct state_table),
 (max_state_entries * sizeof(struct state_table)) / 1024);
}

void print_nat_entries_memory_usage(void) {
 IPFI_PRINTK("IPFIRE: source nat entries lifetime is %lu seconds.\n",
 fwopts.snatted_lifetime);
 IPFI_PRINTK("IPFIRE: max. number of entries is %lu.\n",
 fwopts.max_nat_entries);
 IPFI_PRINTK(
 "IPFIRE: size of a source nat table is %zu bytes.\nIPFIRE:"
 " total memory occupied by snat entries is %lu KB.\n",
 sizeof(struct nat_table),
 (unsigned long)((fwopts.max_nat_entries * sizeof(struct nat_table)) /
 1024));

 IPFI_PRINTK("IPFIRE: destination nat entries lifetime is %lu seconds.\n",
 fwopts.dnatted_lifetime);
 IPFI_PRINTK("IPFIRE: max. number of entries is %lu.\n",
 fwopts.max_nat_entries);
 IPFI_PRINTK(
 "IPFIRE: size of a dest nat table is %zu bytes.\nIPFIRE:"
 " total memory occupied by dnat entries is about %lu KB.\n",
 sizeof(struct nat_table),
 (unsigned long)((fwopts.max_nat_entries * sizeof(struct nat_table)) /
 1024));
}

void print_loginfo_memory_usage(unsigned long lifetime) {
 IPFI_PRINTK("IPFIRE: log entry lifetime is %lu seconds, ", lifetime);
 IPFI_PRINTK("max n. of entries is %u\n"
 "IPFIRE: size of an entry is %zu bytes.\nIPFIRE: maximum memory"
 " occupied by log entries is about %lu KB.\n",
 max_loginfo_entries, sizeof(struct ipfire_loginfo),
 (unsigned long)((max_loginfo_entries *
 sizeof(struct ipfire_loginfo) / 1024)));
}

int send_rule_list_to_userspace(void) {
 command *end_list_cmd = (command *)kmalloc(sizeof(command), GFP_KERNEL);
 if (end_list_cmd == NULL)
 return -1;
 send_a_list(&in_drop);
}

```

```

send_a_list(&out_drop);
send_a_list(&fwd_drop);
send_a_list(&in_acc);
send_a_list(&out_acc);
send_a_list(&fwd_acc);
send_a_list(&translation_pre);
send_a_list(&translation_out);
send_a_list(&translation_post);
send_a_list(&masquerade_post);

end_list_cmd->cmd = PRINT_FINISHED;
if (send_back_command(end_list_cmd) < 0)
 IPFI_PRINTK("IPFIRE: error sending end of rules list!\n");
kfree(end_list_cmd);
return 0;
}

int send_a_list(ipfire_rule *rlist) {
 ipfire_rule *tmp = NULL;
 command *cmd = NULL;
 unsigned i = 0;
 list_for_each_entry(tmp, &rlist->list, list) {
 i++;
 cmd = (command *)kmalloc(sizeof(command), GFP_KERNEL);
 if (cmd != NULL) {
 memset(cmd, 0, sizeof(command));
 cmd->cmd = PRINT_RULES;
 memcpy(&cmd->content.rule, tmp, sizeof(ipfire_rule));
 if (send_back_command(cmd) < 0) {
 IPFI_PRINTK("IPFIRE: error sending rule %d to userspace!\n", i);
 kfree(cmd);
 return -1;
 }
 kfree(cmd);
 }
 }
 return 0;
}

/* Helper: send a single skb to userspace with retry on failure.
 * netlink_unicast consumes the skb on both success and failure,
 * so we must rebuild on retry. Returns 0 on success, < 0 on failure.
 */
static int send_with_retry(void *data, int data_size, int retries) {
 struct sk_buff *skb;
 while (retries-- > 0) {
 skb = build_packet(data, data_size);
 if (skb == NULL)
 return -ENOMEM;
 if (skb_send_to_user(skb, CONTROL_DATA) >= 0)
 return 0;
 if (retries > 0)
 msleep(20);
 }
 return -ENOBUFS;
}

int send_tables(void) {
 struct state_table *st;
 struct state_info *entries = NULL;
 struct state_info endmess;

```

```

int count = 0, i, max_entries;

/* Phase 1: Pre-allocate array outside RCU (can sleep) */
max_entries = percpu_counter_read(&state_tables_counter);
if (max_entries > 0) {
 entries = kcalloc_array(max_entries, sizeof(struct state_info), GFP_KERNEL);
 if (entries == NULL)
 goto send_end_marker;
}

/* Phase 2: Collect entries under RCU lock */
rcu_read_lock_bh();
{
 unsigned int bkt;
 hash_for_each_rcu(state_hashtable, bkt, st, h.hnode) {
 if (count >= max_entries)
 break;
 fill_state_info(&entries[count], st);
 count++;
 }
}
rcu_read_unlock_bh();

/* Phase 3: Send entries outside RCU lock (can sleep/retry) */
for (i = 0; i < count; i++) {
 if (send_with_retry(&entries[i], sizeof(struct state_info), 5) < 0)
 break;
}

kfree(entries);

send_end_marker:
/* Phase 4: Send PRINT_FINISHED (always, with retry) */
memset(&endmess, 0, sizeof(endmess));
endmess.direction = PRINT_FINISHED;
send_with_retry(&endmess, sizeof(struct state_info), 5);
return 0;
}

int send_dnat_tables(void) {
 struct nat_table *dt;
 struct dnat_info *entries = NULL;
 struct dnat_info endmess;
 int count = 0, i, max_entries;

 /* Phase 1: Pre-allocate array outside RCU (can sleep) */
 max_entries = percpu_counter_read(&nat_counters[NAT_DNAT]);
 if (max_entries > 0) {
 entries = kcalloc_array(max_entries, sizeof(struct dnat_info), GFP_KERNEL);
 if (entries == NULL)
 goto send_end_marker;
 }

 /* Phase 2: Collect entries under RCU lock */
 rcu_read_lock();
 {
 unsigned int bkt;
 hash_for_each_rcu(nat_hashtables[NAT_DNAT][NAT_IDX_ORIG], bkt, dt,
 h.hnode) {
 if (count >= max_entries)
 break;

```

```

 fill_dnat_info(&entries[count], dt);
 count++;
 }
}
rcu_read_unlock();

/* Phase 3: Send entries outside RCU lock (can sleep/retry) */
for (i = 0; i < count; i++) {
 if (send_with_retry(&entries[i], sizeof(struct dnat_info), 5) < 0)
 break;
}

kfree(entries);

send_end_marker:
/* Phase 4: Send PRINT_FINISHED (always, with retry) */
memset(&endmess, 0, sizeof(endmess));
endmess.direction = PRINT_FINISHED;
send_with_retry(&endmess, sizeof(struct dnat_info), 5);
return 0;
}

int send_snat_tables(void) {
 struct nat_table *st;
 struct snat_info *entries = NULL;
 struct snat_info endmess;
 int count = 0, i, max_entries;

 /* Phase 1: Pre-allocate array outside RCU (can sleep) */
 max_entries = percpu_counter_read(&nat_counters[NAT_SNAT]);
 if (max_entries > 0) {
 entries = kmalloc_array(max_entries, sizeof(struct snat_info), GFP_KERNEL);
 if (entries == NULL)
 goto send_end_marker;
 }

 /* Phase 2: Collect entries under RCU lock */
 rcu_read_lock();
 {
 unsigned int bkt;
 hash_for_each_rcu(nat_hashtables[NAT_SNAT][NAT_IDX_ORIG], bkt, st,
 h.hnode) {
 if (count >= max_entries)
 break;
 fill_snat_info(&entries[count], st);
 count++;
 }
 }
 rcu_read_unlock();

 /* Phase 3: Send entries outside RCU lock (can sleep/retry) */
 for (i = 0; i < count; i++) {
 if (send_with_retry(&entries[i], sizeof(struct snat_info), 5) < 0)
 break;
 }

 kfree(entries);

send_end_marker:
/* Phase 4: Send PRINT_FINISHED (always, with retry) */
memset(&endmess, 0, sizeof(endmess));

```

```

 endmess.direction = PRINT_FINISHED;
 send_with_retry(&endmess, sizeof(struct snat_info), 5);
 return 0;
}

int send_ktables_usage(void) {
 struct ktables_usage *ktu;
 struct sk_buff *skb_to_user = NULL;
 int ret = -1;

 ktu =
 (struct ktables_usage *)kmallocc(sizeof(struct ktables_usage), GFP_KERNEL);
 if (ktu != NULL) {
 ktu->loguser = fwopts.loguser;
 ktu->state_tables = percpu_counter_read(&state_tables_counter);
 ktu->snat_tables = get_snatted_count();
 ktu->dnat_tables = get_dnatted_count();
 ktu->loginfo_tables = percpu_counter_read(&loginfo_entry_counter);
 ktu->state_lifetime = state_lifetime;
 skb_to_user = build_ktable_info_packet(ktu);
 if (skb_to_user != NULL)
 ret = skb_send_to_user(skb_to_user, CONTROL_DATA);
 kfree(ktu);
 }
 return ret;
}

int tell_user_howmany_rules_flushed(int howmany) {
 command *cmd = (command *)kmallocc(sizeof(command), GFP_KERNEL);
 if (cmd != NULL) {
 cmd->anumber = 0;
 if (howmany < 0)
 cmd->cmd = ROOT_NOFLUSHED;
 else {
 cmd->cmd = FLUSH_RULES;
 cmd->anumber = howmany;
 }
 send_back_command(cmd);
 kfree(cmd);
 }
 return 0;
}

int simple_exit(void) {
 userspace_control_pid = 0;
 userspace_data_pid = 0;
 IPFI_PRINTK("IPFIRE: received simple exit: resetting counters and"
 " waiting for new connections from userspace.\n");
 return 0;
}

int do_userspace_exit_tasks(uid_t userspace_commander) {
 ipfire_info_t *ipfi_info_exit = NULL;
 struct sk_buff *skb_touser = NULL;
 int ret = 0;

 if ((userspace_commander == 0) && (fwopts.noflush_on_exit)) {
 tell_user_howmany_rules_flushed(-1);
 IPFI_PRINTK("IPFIRE: did not flush rules as requested.\n");
 } else {
 ret = flush_ruleset(userspace_commander, FLUSH_RULES);
 }
}

```

```

 tell_user_howmany_rules_flushed(ret);
}

if (gui_notifier_enabled) {
 ipfi_info_exit =
 (ipfire_info_t *)kmalloc(sizeof(ipfire_info_t), GFP_KERNEL);
 if (ipfi_info_exit != NULL) {
 memset(ipfi_info_exit, 0, sizeof(ipfire_info_t));
 ipfi_info_exit->flags.exit = 1;
 skb_touser = build_info_t_packet(ipfi_info_exit);
 if (skb_touser != NULL)
 skb_send_to_user(skb_touser, GUI_NOTIF_DATA);
 kfree(ipfi_info_exit);
 }
}
userspace_control_pid = 0;
userspace_data_pid = 0;
return ret;
}

int send_kstats(void) {
 int ret = -1;
 struct sk_buff *skbtou;
 struct kernel_stats *ks;
 ks = (struct kernel_stats *)kmalloc(sizeof(struct kernel_stats), GFP_KERNEL);
 if (ks != NULL) {
 ipfi_get_total_stats(ks);
 skbtou = build_kstats_packet(ks);
 if (skbtou != NULL)
 ret = skb_send_to_user(skbtou, CONTROL_DATA);
 kfree(ks);
 }
 return ret;
}

int send_kstats_light(void) {
 int ret = -1;
 struct kstats_light ksl;
 struct sk_buff *skbtou;

 ipfi_get_light_stats(&ksl);
 skbtou = build_kstats_light_packet(&ksl);
 if (skbtou != NULL)
 ret = skb_send_to_user(skbtou, CONTROL_DATA);

 return ret;
}

int send_back_command(const command *cmd) {
 struct sk_buff *skbtou;
 skbtou = build_command_packet(cmd);
 if (!skbtou)
 return -ENOMEM;
 return skb_send_to_user(skbtou, CONTROL_DATA);
}

int send_back_fw_busy(pid_t pid) {
 command *com;
 struct sk_buff *skb;
 int status = -1;
 com = (command *)kmalloc(sizeof(command), GFP_KERNEL);

```

```

 if (com == NULL)
 return status;

 memset(com, 0, sizeof(command));
 com->cmd = ACKNOWLEDGEMENT;
 skb = build_command_packet(com);

 if (skb != NULL)
 status = send_data_to_user(skb, pid, sknl_ipfi_control);
 else
 status = -ENOMEM;

 if (status >= 0) {
 com->cmd = IPFIRE_BUSY;
 com->anumber = userspace_control_pid;
 skb = build_command_packet(com);
 if (skb != NULL)
 status = send_data_to_user(skb, pid, sknl_ipfi_control);
 else
 status = -ENOMEM;
 }
 kfree(com);
 return status;
}

```

## netlink/netlink.c

```

/* netlink/netlink.c: Netlink socket management for ipfire-wall */

#include "globals.h"
#include "ipfire.h"
#include "netlink/ipfi_netl.h"
#include <linux/module.h>
#include <linux/netlink.h>
#include <linux/version.h>
#include <net/sock.h>

#if LINUX_VERSION_CODE >= KERNEL_VERSION(2, 6, 24)
static void nl_receive_control(struct sk_buff *skb);
static void nl_receive_data(struct sk_buff *skb);
#else
static void nl_receive_control(struct sock *sk, int len);
static void nl_receive_data(struct sock *sk, int len);
#endif

static int create_control_socket(void) {
 if LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 14)
 sknl_ipfi_control =
 netlink_kernel_create(NETLINK_IPFI_CONTROL, nl_receive_control);

 elif LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 22)
 sknl_ipfi_control = netlink_kernel_create(NETLINK_IPFI_CONTROL, 0,
 nl_receive_control, THIS_MODULE);

 elif LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 24)
 sknl_ipfi_control = netlink_kernel_create(
 NETLINK_IPFI_CONTROL, 0, nl_receive_control, NULL, THIS_MODULE);
 elif LINUX_VERSION_CODE < KERNEL_VERSION(3, 6, 0)

```

```

 sknl_ipfi_control =
 netlink_kernel_create(&init_net, NETLINK_IPFI_CONTROL, 0,
 nl_receive_control, NULL, THIS_MODULE);
#else
 struct netlink_kernel_cfg netlink_cfg;
 netlink_cfg.groups = 0;
 netlink_cfg.flags = 0;
 netlink_cfg.input = nl_receive_control;
 netlink_cfg.bind = NULL;
 sknl_ipfi_control =
 netlink_kernel_create(&init_net, NETLINK_IPFI_CONTROL, &netlink_cfg);
#endif

 userspace_control_pid = 0;
 if (sknl_ipfi_control == NULL) {
 IPFI_PRINTK(
 "IPFIRE: create_socket(): failed to create netlink control socket\n");
 return -1;
 }
 return 0;
}

static int create_data_socket(void) {
 #if LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 14)
 sknl_ipfi_data = netlink_kernel_create(NETLINK_IPFI_DATA, nl_receive_data);
 #elif LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 22)
 sknl_ipfi_data =
 netlink_kernel_create(NETLINK_IPFI_DATA, 0, nl_receive_data, THIS_MODULE);
 #elif LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 24)

 sknl_ipfi_data = netlink_kernel_create(NETLINK_IPFI_DATA, 0, nl_receive_data,
 NULL, THIS_MODULE);
 #elif LINUX_VERSION_CODE < KERNEL_VERSION(3, 6, 0)
 sknl_ipfi_data = netlink_kernel_create(&init_net, NETLINK_IPFI_DATA, 0,
 nl_receive_data, NULL, THIS_MODULE);
 #else
 struct netlink_kernel_cfg netlink_cfg;
 netlink_cfg.groups = 0;
 netlink_cfg.flags = 0;
 netlink_cfg.input = nl_receive_data;
 netlink_cfg.bind = NULL;
 sknl_ipfi_data =
 netlink_kernel_create(&init_net, NETLINK_IPFI_DATA, &netlink_cfg);
 #endif

 userspace_data_pid = 0;
 if (sknl_ipfi_data == NULL) {
 printk("IPFIRE: create_socket(): failed to create netlink data socket\n");
 return -1;
 }
 return 0;
}

static int create_gui_notifier_socket(void) {
 #if LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 14)
 sknl_ipfi_gui_notifier =
 netlink_kernel_create(NETLINK_IPFI_GUI_NOTIFIER, NULL);
 #elif LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 22)
 sknl_ipfi_gui_notifier =

```



```

 netlink_kernel_create(NETLINK_IPFI_GUI_NOTIFIER, 0, NULL, THIS_MODULE);
#elif LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 24)

 sknl_ipfi_gui_notifier = netlink_kernel_create(NETLINK_IPFI_GUI_NOTIFIER, 0,
 NULL, NULL, THIS_MODULE);
#elif LINUX_VERSION_CODE < KERNEL_VERSION(3, 6, 0)
 sknl_ipfi_gui_notifier = netlink_kernel_create(
 &init_net, NETLINK_IPFI_GUI_NOTIFIER, 0, NULL, NULL, THIS_MODULE);
#else
 struct netlink_kernel_cfg netlink_cfg;
 netlink_cfg.groups = 0;
 netlink_cfg.flags = 0;
 netlink_cfg.input = NULL;
 netlink_cfg.bind = NULL;
 sknl_ipfi_gui_notifier =
 netlink_kernel_create(&init_net, NETLINK_IPFI_GUI_NOTIFIER, &netlink_cfg);
#endif
 if (sknl_ipfi_gui_notifier == NULL) {
 IPFI_PRINTK("IPFIRE: create_socket(): failed to create netlink gui "
 "notifier socket\n");
 return -1;
 }
 return 0;
}

int send_data_to_user(struct sk_buff *skb, pid_t destination_pid,
 struct sock *socket) {
 int ret = -1;
 if (socket != NULL && skb != NULL) {
 NETLINK_CB(skb).portid = 0; /* kernel sending */
 NETLINK_CB(skb).dst_group = 0;
 ret = netlink_unicast(socket, skb, destination_pid, MSG_DONTWAIT);
 if (ret < 0) {
 IPFI_PRINTK("IPFIRE: netlink_unicast() to pid %d failed with error %d. "
 "Errnos in asm-generic/errno-base.h\n",
 destination_pid, ret);
 }
 } else
 IPFI_PRINTK("socket or sk_buff null in send_data_to_user(): socket: 0x%p "
 "skb: 0x%p\n",
 socket, skb);
 return ret;
}

pid_t get_sender_pid(const struct sk_buff *skbuff) {
 int ret = 0;
 pid_t header_pid, credentials_pid;
 struct nlmsgghdr *nlh = nlmsg_hdr(skbuff);
 if (nlh == NULL) {
 IPFI_PRINTK("IPFIRE: get_sender_pid(): error extracting nlmsgghdr from "
 "socket buffer. Cannot determine header pid\n");
 ret = 0;
 } else {
 header_pid = nlh->nlmsg_pid;
 if (1) {
 credentials_pid = NETLINK_CB(skbuff).portid;
 if (credentials_pid != header_pid) {
 IPFI_PRINTK("IPFIRE: PID mismatch! Did you try to fool me? :)\nIPFIRE: "
 "CREDENTIALS PID: %u, HEADER PID: %u\n",
 credentials_pid, header_pid);
 ret = 0;
 }
 }
 }
}

```

```

 } else
 ret = header_pid;
 }
}
return ret;
}

static void nl_receive_control(struct sk_buff *skb) {
 pid_t pid;

 pid = get_sender_pid(skb);
 userspace_uid = from_kuid(&init_user_ns, NETLINK_CREDS(skb)->uid);
 if ((userspace_control_pid != 0) && (pid != userspace_control_pid))
 send_back_fw_busy(pid);
 else {
 userspace_control_pid = pid;
 process_control_received(skb);
 }
}

static void nl_receive_data(struct sk_buff *skb) {
 userspace_data_pid = get_sender_pid(skb);
 process_data_received(skb);
}

void init_ruleset_heads(void) {
 /* initialize lists */
 INIT_LIST_HEAD(&in_drop.list);
 INIT_LIST_HEAD(&out_drop.list);
 INIT_LIST_HEAD(&fwd_drop.list);
 INIT_LIST_HEAD(&in_acc.list);
 INIT_LIST_HEAD(&out_acc.list);
 INIT_LIST_HEAD(&fwd_acc.list);
 INIT_LIST_HEAD(&translation_pre.list);
 INIT_LIST_HEAD(&translation_out.list);
 INIT_LIST_HEAD(&translation_post.list);
 INIT_LIST_HEAD(&masquerade_post.list);
}

int init_netl(void) {
 int ctrl_so, data_so, gui_so;
 data_so = 0, ctrl_so = 0, gui_so = 0;
 ctrl_so = create_control_socket();
 data_so = create_data_socket();
 gui_so = create_gui_notifier_socket();

 memset(moderate_print, 0, sizeof(unsigned int) * MAXMODERATE_ARGS);
 memset(moderate_print_limit, 0, sizeof(unsigned int) * MAXMODERATE_ARGS);
 moderate_print_limit[PRINT_PROTO_UNSUPPORTED] = 10000;

 memset(&kslight, 0, sizeof(kslight));
 if (ctrl_so == 0 && data_so == 0 && gui_so == 0)
 return 0;
 else
 return -1;
}

void fini_netl(void) {
 if (sknl_ipfi_control != NULL) {
#ifdef LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 24)
 sock_release(sknl_ipfi_control->sk_socket);

```

```

#else
 netlink_kernel_release(skn_l_ipfi_control);
#endif
 skn_l_ipfi_control = NULL;
}

 if (skn_l_ipfi_data != NULL) {
#if LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 24)
 sock_release(skn_l_ipfi_data->sk_socket);
#else
 netlink_kernel_release(skn_l_ipfi_data);
#endif
 skn_l_ipfi_data = NULL;
 }

 if (skn_l_ipfi_gui_notifier != NULL) {
#if LINUX_VERSION_CODE < KERNEL_VERSION(2, 6, 24)
 sock_release(skn_l_ipfi_gui_notifier->sk_socket);
#else
 netlink_kernel_release(skn_l_ipfi_gui_notifier);
#endif
 skn_l_ipfi_gui_notifier = NULL;
 }

 IPFI_PRINTK("IPFIRE: Closing netlink sockets (control, data, gui).\n");
}

```

## netlink/ipfi\_netl.h

```

#ifndef IPFI_NETL_H
#define IPFI_NETL_H

/* See ipfi.c for details and
 * use of this software.
 * (C) 2005 Giacomo S.
 */

#include <linux/module.h>
#include <linux/skbuff.h>
#include <linux/init.h>
#include <linux/ip.h>
#include <linux/notifier.h>
#include <linux/netlink.h>
#include <linux/time.h>
#include <net/sock.h>
#include "ipfire.h"
#include "../logging/log.h"
#include "../nat/snat/snat.h"
#include "../nat/dnat/dnat.h"
#include "../filter/state/state_table.h"
#include <common/ipfi_structures.h>

struct ipfire_options;
/* Smart logging */
enum smartlog_type
{
 SMART_LOG = 1,
 SMART_LOG_WITH_STATE_CHECK,

```

```

};

/* every LOG_PACKETS_RECV packets
 * received for each hook, write a line to
 * syslog */
#define LOG_PACKETS_RECV 1000000ULL

/* every LOG_NETLINK_FAILURES write
 * a message to syslog */
#define LOG_NETLINK_FAILURES 25000

/* netlink related function calls */

/** @param skb socket buffer received by netlink socket callback nl_receive_control()
 * Extracts the command data structure from skb with NLMSG_DATA (extract_data()) and
 * according to the command type, invokes one of the functions in ipfi_process_control.
 * @return less than zero if an error occurred.
 */
int process_control_received(struct sk_buff *skb);
void *extract_data(struct sk_buff *skb);
int send_back_fw_busy(pid_t pid);

/** Handshake with userspace program: checks if structure sizes
 * are correct
 */
int initial_handshake(command* hello, uid_t userspace_uid);

/* Fills in the firesizes structure with the size of the
 * structures.
 */
void get_struct_sizes(struct firesizes* fsz);

/* copies kernel sizes in firesizes structure to send
 * back to userspace. */
void fill_firesizes_with_kernel_values(command* cmd, size_t krulesize, size_t kinfosize,
size_t kcmdsize, uid_t uspace_uid);

int init_netl(void);
void fini_netl(void);

void fill_dnat_info(struct dnat_info *dninfo, const struct nat_table *dnnt);
void fill_snat_info(struct snat_info *sninfo, const struct nat_table *snnt);

/* Sends in userspace the struct sizes */
int send_struct_sizes(void);

/* Sends in userspace the type of logging to userspace in use.
 * Reads fwopts.
 */
int send_smartlog_type(void);

/* depending on loguser, this function decides if
 * firewall has to send packet skb to userspace
 * firewall */
int is_to_send(const struct sk_buff *skb,
 const struct ipfire_options* ipo,
 const struct response *res,
 const ipfi_flow *flow,
 const struct info_flags *flags);

```

```

/* packets sent to userspace get the "logu_id" field incremented
 * by one each time. If this counter exceeds ULONG_MAX, it
 * has to be re-initialized to 0, to prevent overflow */
unsigned long long update_sent_counter(int direction);

int send_data_to_user(struct sk_buff *skb, pid_t destination_pid, struct sock*
which_socket);

pid_t get_sender_pid(const struct sk_buff *skbff);

void set_outgoing_skb_params(struct sk_buff *skbf);
int nl_receive_outcome(struct sock* sknl_ipfi_data_rec);

/* sends an acknowledgement to userspace program
 * before actuating a command */
inline int send_acknowledgement(pid_t uspace_pid);

/* given a direction or a constant identifying the counter
 * which has to be incremented, this function increments
 * corresponding counter. Response is unsigned long
 * because if the counter to be incremented is total_lost
 * (last case), printk logs the packet_id field of packet
 * that failed to be sent. Moreover, response field is
 * copied to last_failed field of kernel stats structure. */
void
update_kernel_stats(int counter_to_increment, short response);

/* checks if timeouts are too high and, if so, sets them to the
 * maximum value allowed.
 * In 32 bit architecture, maximum value for timeouts is
 * $2^{32}/2/\text{HZ}$, see ipfi_netl.h near the definition of
 * MAX_TIMEOUT.
 */
void check_max_timeout_values(command* cmd);

/* sets various options related to firewall behaviour, as specified by
 * command received from userspace.
 */
int set_firewall_options(command* cmd, const uid_t commander);

void opts_to_cmd(command* cmd);

int print_command(const command* cmd);

/* adds the new rule contained inside the command to the right list of
 * rules, depending on direction and policy */
int add_rule_to_list_by_command(command *cmd);

void update_ifindex_in_rules(const char *name, int new_index);
void init_ruleset_heads(void);

void print_loginfo_memory_usage(unsigned long lifetime);

void print_state_entries_memory_usage(void);

void print_nat_entries_memory_usage(void);

int send_rule_list_to_userspace(void);
int send_a_list(ipfire_rule* rlist);
int send_tables(void);

```

```

int send_dnat_tables(void);
int send_snat_tables(void);
int send_ktables_usage(void);
int do_userspace_exit_tasks(uid_t userspace_commander);
int flush_ruleset(uid_t userspace_commander, int flush_command);
int free_rules(ipfire_rule*, uid_t user);
int free_dynamic_tables(void);
/* NAT tables freed via free_nat_tables() in nat_table.h */
int tell_user_howmany_rules_flushed(int howmany);
int send_loguser_enabled(int logu_enabled);
int initial_handshake(command *hello, uid_t userspace_uid);
int register_log_function(int loglevel);
int send_back_command(const command *cmd);
int process_data_received(struct sk_buff *skb);
void fill_state_info(struct state_info *stinfo, const struct state_table *stt);

/** @param cmd a kmallocated command.
 * @return less than 0 in case of error (-ENOMEM)
 *
 * The parameter cmd _must_ be a kmallocated pointer to command data structure.
 * Never pass command* through & operator in the caller!
 * This function calls build_command_packet(cmd) obtaining a pointer to a struct
 * sk_buff.
 * If that pointer is not NULL, the command is ready to be sent to userspace via the
 * netlink CONTROL socket.
 */
int manage_rule(command* rule_from_user);

int skb_send_to_user(struct sk_buff* skb, int type_of_message);
int get_input_response(void);
int get_output_response(void);
int get_forward_response(void);

/* This function is invoked when userspace firewall
 * sends a simple exit command. It resets counters and
 * pid values for accepting new registrations for right
 * userspace firewalls */
int simple_exit(void);

void init_options(struct ipfire_options* opts);

/** @cmd the command containing the rule as content. The rule is compared with all
 * the rules in the list to establish if it is already present.
 */
int rule_not_loaded(const command *);

int find_rules(const ipfire_rule* rlist, const ipfire_rule* rule);

void init_kernel_stats(struct kernel_stats* nl_kstats);

/* sends to userspace the passed kernel_stats structure */
int send_kstats(void);

/* Sends to userspace the passed kernel_stats light structure.
 * This is for the GUI which has a SystemTray that indicates the
 * traffic being filtered.
 */

```

```
int send_kstats_light(void);

#endif
```

## proc/proc.h

```
#ifndef IPFIRE_PROC_H
#define IPFIRE_PROC_H

#include <linux/proc_fs.h>

#define PROCDIR "IPFIRE"
#define PROCENT "policy"
#define PROCENTRY_DATA_LEN 100

/* proc entry */
int init_procentry(const char* name, const char *value);

void set_procentry_values(void);

int proc_write_ipfire(struct file *file, const char *page,
 unsigned long count, void *data);

int proc_read_ipfire(char *page, char **start, off_t off,
 int count, int *eof, void *data);

void clean_proc(void);

void set_policy(const char* def_policy);

#endif
```

## proc/proc.c

```
#include "proc/proc.h"
#include <linux/module.h>
#include <linux/version.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/uaccess.h>

static struct proc_dir_entry *ipfire_procent, *procdir;
static char procentry_line[PROCENTRY_DATA_LEN];

static ssize_t ipfire_proc_read(struct file *file, char __user *buf,
 size_t count, loff_t *ppos)
{
 char *data = pde_data(file_inode(file));
 char msg[PROCENTRY_DATA_LEN + 64];
 int len;

 if (!data)
 return 0;

 len = snprintf(msg, sizeof(msg), "IPFIRE default behaviour when a packet does not
```

```

match: %s\n", data);

 return simple_read_from_buffer(buf, count, ppos, msg, len);
}

static ssize_t ipfire_proc_write(struct file *file, const char __user *buf,
 size_t count, loff_t *ppos)
{
 int len;

 if (count >= PROCENTRY_DATA_LEN)
 len = PROCENTRY_DATA_LEN - 1;
 else
 len = count;

 if (copy_from_user(procentry_line, buf, len))
 return -EFAULT;

 procentry_line[len] = '\0';
 /* strip newline if present */
 if (len > 0 && procentry_line[len-1] == '\n')
 procentry_line[len-1] = '\0';

 set_policy(procentry_line);
 return len;
}

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5,6,0)
static const struct proc_ops ipfire_proc_ops = {
 .proc_read = ipfire_proc_read,
 .proc_write = ipfire_proc_write,
 .proc_lseek = default_llseek,
};
#else
static const struct file_operations ipfire_proc_ops = {
 .owner = THIS_MODULE,
 .read = ipfire_proc_read,
 .write = ipfire_proc_write,
 .llseek = default_llseek,
};
#endif

int init_procentry(const char *name, const char *value)
{
 procdir = proc_mkdir(PROCDIR, NULL);
 if (!procdir) {
 printk("IPFIRE: error creating proc dir!\n");
 return -ENOMEM;
 }

 strncpy(procentry_line, value, PROCENTRY_DATA_LEN);

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5,6,0)
 ipfire_procent = proc_create_data(name, 0644, procdir, &ipfire_proc_ops,
 procentry_line);
#else
 ipfire_procent = proc_create_data(name, 0644, procdir, &ipfire_proc_ops,
 procentry_line);
#endif

 if (ipfire_procent == NULL)

```



```

 {
 printk("IPFIRE: error creating proc entry!\n");
 remove_proc_entry(PROCDIR, NULL);
 return -ENOMEM;
 }
 return 0;
}

void set_procentry_values(void)
{
 /* Deprecated/Removed functionality - no-op */
}

void clean_proc(void)
{
 if (ipfire_procent)
 remove_proc_entry(PROCENT, procdir);
 if (procdir)
 remove_proc_entry(PROCDIR, NULL);
}

```

## common/globals.c

```

#include "globals.h"
#include <linux/module.h>
#include <linux/spinlock.h>

/* Netlink related PIDs and sockets */
pid_t userspace_control_pid = 0;
pid_t userspace_data_pid = 0;
uid_t userspace_uid = 0;
struct sock *sknl_ipfi_control = NULL;
struct sock *sknl_ipfi_data = NULL;
struct sock *sknl_ipfi_gui_notifier = NULL;

/* Firewall options and status */
struct ipfire_options fwopts;
short default_policy = IPFIRE_DEFAULT_POLICY;
short loguser_enabled = 1;
short gui_notifier_enabled = 0;

/* Statistics */
struct kernel_stats kstats;
struct kstats_light kslight;

struct ipfi_counters __percpu *ipfi_counters = NULL;

void ipfi_get_total_stats(struct kernel_stats *total) {
 int cpu;
 memset(total, 0, sizeof(struct kernel_stats));

 /* Aggregate counters from all CPUs */
 if (ipfi_counters) {
 for_each_possible_cpu(cpu) {
 struct ipfi_counters *c = per_cpu_ptr(ipfi_counters, cpu);
 total->in_rcv += c->in_rcv;
 total->out_rcv += c->out_rcv;
 total->pre_rcv += c->pre_rcv;
 }
 }
}

```

```

total->post_rcv += c->post_rcv;
total->fwd_rcv += c->fwd_rcv;
total->sum += c->sum;
total->total_lost += c->total_lost;

/* Sum per-direction sent counters */
/* Note: structural update: these are not in original struct kernel_stats
 * BUT we might want to sum them somewhere or just provide them.
 * For now, we sum them into the total kstats if we added them there.
 */
total->sent_tou += c->sent_tou;
total->last_failed += c->last_failed;

total->in_acc += c->in_acc;
total->in_drop += c->in_drop;
total->in_drop_impl += c->in_drop_impl;
total->in_acc_impl += c->in_acc_impl;

total->out_acc += c->out_acc;
total->out_drop += c->out_drop;
total->out_drop_impl += c->out_drop_impl;
total->out_acc_impl += c->out_acc_impl;

total->fwd_acc += c->fwd_acc;
total->fwd_drop += c->fwd_drop;
total->fwd_drop_impl += c->fwd_drop_impl;
total->fwd_acc_impl += c->fwd_acc_impl;

total->not_sent += c->not_sent;
total->bad_checksum_in += c->bad_checksum_in;
total->bad_checksum_out += c->bad_checksum_out;
}
}

/* Copy global metadata */
total->kmod_load_time = kstats.kmod_load_time;
total->policy = kstats.policy;
}

void ipfi_get_light_stats(struct kstats_light *light) {
 int cpu;
 memset(light, 0, sizeof(struct kstats_light));

 if (ipfi_counters) {
 for_each_possible_cpu(cpu) {
 struct ipfi_counters *c = per_cpu_ptr(ipfi_counters, cpu);
 light->blocked += c->blocked;
 light->allowed += c->allowed;
 }
 }
}

/* Rulesets */
ipfire_rule in_drop;
ipfire_rule out_drop;
ipfire_rule fwd_drop;
ipfire_rule in_acc;
ipfire_rule out_acc;
ipfire_rule fwd_acc;
ipfire_rule translation_pre;
ipfire_rule translation_post;

```

```

ipfire_rule translation_out;
ipfire_rule masquerade_post;

/* State tables (NAT lists/locks/counters now in nat_table.c) */
struct kmem_cache *nat_cache;

/* State table storage */
struct hlist_head state_hashtable[1 << STATE_HASH_BITS];
/* NAT hash tables (Type, Index, Bucket) */
struct hlist_head nat_hashtables[2][NAT_IDX_COUNT][1 << NAT_HASH_BITS];
/* Logininfo hash table (list still used for LRU eviction) */
struct hlist_head logininfo_hashtable[1 << LOG_HASH_BITS];

/* Log info */
LIST_HEAD(active_logi_list);

/* Counters */
unsigned int table_id = 0;
struct percpu_counter state_tables_counter;
struct percpu_counter nat_counters[2];
struct percpu_counter logininfo_entry_counter;

/* Timeouts and Limits */
unsigned int state_lifetime = 432000; /* 5 days in seconds */
unsigned int setup_shutd_state_lifetime = 120;
unsigned int logininfo_lifetime = 30;
unsigned int max_logininfo_entries = 256;
int (*smartlog_func)(const struct sk_buff *skb, const struct response *res,
 const ipfi_flow *flow,
 const struct info_flags *flags) = NULL;
unsigned int max_state_entries = 65536;

/* Print limiting */
unsigned int moderate_print[MAXMODERATE_ARGS];
unsigned int moderate_print_limit[MAXMODERATE_ARGS];

// Locks. With the following macros
// The lock is statically initialized
// It is valid before initcall()
// It works correctly on SMP, PREEMPT, DEBUG_SPINLOCK, etc
// Lockdep gets the right metadata (name, class key)
// This is the blessed way to define a global or file-scope spinlock.
//
DEFINE_SPINLOCK(rulelist_lock);
DEFINE_SPINLOCK(logininfo_list_lock);
/* NAT locks (Type, Index, Bucket) */
spinlock_t nat_bucket_locks[2][NAT_IDX_COUNT][1 << NAT_HASH_BITS];
spinlock_t state_bucket_locks[1 << STATE_HASH_BITS];
struct workqueue_struct *ipfire_wq = NULL;

bool we_are_exiting = false;

/* Export symbols if needed by other modules, but here they are used within the
 * same module */

```

## common/stats.c

```

#include <linux/module.h>
#include "globals.h"

void update_kernel_stats(int counter_to_increment, short int response)
{
 switch (counter_to_increment)
 {
 case IPFI_INPUT:
 IPFI_STAT_INC(in_rcv);
 if (response == IPFI_IMPLICIT && default_policy == IPFI_DROP)
 {
 IPFI_STAT_INC(in_drop);
 IPFI_STAT_INC(blocked);
 IPFI_STAT_INC(in_drop_impl);
 }
 else if(response == IPFI_IMPLICIT && default_policy == IPFI_ACCEPT)
 {
 IPFI_STAT_INC(in_acc);
 IPFI_STAT_INC(allowed);
 IPFI_STAT_INC(in_acc_impl);
 }
 else if (response == IPFI_DROP)
 {
 IPFI_STAT_INC(blocked);
 IPFI_STAT_INC(in_drop);
 }
 else
 {
 IPFI_STAT_INC(allowed);
 IPFI_STAT_INC(in_acc);
 }
 break;

 case IPFI_OUTPUT:
 IPFI_STAT_INC(out_rcv);
 if (response == IPFI_IMPLICIT && default_policy == IPFI_DROP)
 {
 IPFI_STAT_INC(out_drop);
 IPFI_STAT_INC(blocked);
 IPFI_STAT_INC(out_drop_impl);
 }
 else if(response == IPFI_IMPLICIT && default_policy == IPFI_ACCEPT)
 {
 IPFI_STAT_INC(out_acc);
 IPFI_STAT_INC(allowed);
 IPFI_STAT_INC(out_acc_impl);
 }
 else if (response == IPFI_DROP)
 {
 IPFI_STAT_INC(blocked);
 IPFI_STAT_INC(out_drop);
 }
 else
 {
 IPFI_STAT_INC(allowed);
 IPFI_STAT_INC(out_acc);
 }
 break;

 case IPFI_FWD:

```

```

IPFI_STAT_INC(fwd_rcv);
if (response == IPFI_IMPLICIT && default_policy == IPFI_DROP)
{
 IPFI_STAT_INC(blocked);
 IPFI_STAT_INC(fwd_drop);
 IPFI_STAT_INC(fwd_drop_impl);
}
else if(response == IPFI_IMPLICIT && default_policy == IPFI_ACCEPT)
{
 IPFI_STAT_INC(allowed);
 IPFI_STAT_INC(fwd_acc);
 IPFI_STAT_INC(fwd_acc_impl);
}
else if (response == IPFI_DROP)
{
 IPFI_STAT_INC(fwd_drop);
 IPFI_STAT_INC(blocked);
}
else
{
 IPFI_STAT_INC(allowed);
 IPFI_STAT_INC(fwd_acc);
}
break;

case IPFI_INPUT_PRE:
 IPFI_STAT_INC(pre_rcv);
 break;

case IPFI_OUTPUT_POST:
 IPFI_STAT_INC(post_rcv);
 break;

case IPFI_FAILED_NETLINK_USPACE:
 /* Since total_lost is now per-CPU, a precise global check here is expensive.
 * We do a best-effort per-CPU check for logging. */
 if ((this_cpu_read(ipfi_counters->total_lost) % 25000 == 0)
 && (this_cpu_read(ipfi_counters->total_lost) != 0))
 IPFI_PRINTK("IPFIRE: failed to send %llu packets to userspace "
 "via netlink socket [loguser: %d, response: %d]!\n",
 this_cpu_read(ipfi_counters->total_lost), fwopts.loguser,
 response);
 IPFI_STAT_INC(total_lost);
 this_cpu_write(ipfi_counters->last_failed, (unsigned long long)response);
 break;
}
}

```

## globals.h

```

#ifndef IPFI_GLOBALS_H
#define IPFI_GLOBALS_H

#include "ipfi_machine.h"
#include "ipfire.h"
#include "logging/log.h"
#include "nat/dnat/dnat.h"
#include "nat/nat.h"

```

```

#include "nat/snat/snat.h"
#include <linux/percpu.h>
#include <linux/spinlock.h>
#include <linux/types.h>

#ifndef timer_container_of
#define timer_container_of(ptr, timer, member) from_timer(ptr, timer, member)
#endif

/*
 * Internal pseudo-protocol for loginfo timer refresh.
 * Uses IANA-reserved value 254 (experimentation/testing).
 * This lets log entries reuse ipfi_entry_update_timer()
 * without needing a separate flat-timeout function.
 */
#define IPPROTO_IPFI_LOG 254

/* Netlink related PIDs and sockets */
extern pid_t userspace_control_pid;
extern pid_t userspace_data_pid;
extern uid_t userspace_uid;
extern struct sock *sknl_ipfi_control;
extern struct sock *sknl_ipfi_data;
extern struct sock *sknl_ipfi_gui_notifier;

/* Firewall options and status */
extern struct ipfire_options fwopts;
extern short default_policy;
extern short loguser_enabled;
extern short gui_notifier_enabled;

/* Statistics */
extern struct kernel_stats kstats;
extern struct kstats_light kslight;

/* Per-CPU Statistics */
extern struct ipfi_counters __percpu *ipfi_counters;
#define IPFI_STAT_INC(field) this_cpu_inc(ipfi_counters->field)

void ipfi_get_total_stats(struct kernel_stats *total);
void ipfi_get_light_stats(struct kstats_light *light);

/* Rulesets */
extern ipfire_rule in_drop;
extern ipfire_rule out_drop;
extern ipfire_rule fwd_drop;
extern ipfire_rule in_acc;
extern ipfire_rule out_acc;
extern ipfire_rule fwd_acc;
extern ipfire_rule translation_pre;
extern ipfire_rule translation_post;
extern ipfire_rule translation_out;
extern ipfire_rule masquerade_post;

/* State table */
/*
 * SIZING EXPLANATION:
 * We use `1 << STATE_HASH_BITS` to force the table size to be a Power of Two.
 * Example: 1 << 10 = 1024.
 *
 * INTENT:

```

```

 * Computers are much faster at bitwise operations than division. By
 * guaranteeing a Power of Two size, the kernel can use a fast bitwise mask
 * (`key & mask`) to find the bucket index, instead of a slow modulo operation
 * (`key % size`).
 */
extern struct hlist_head state_hashtable[1 << STATE_HASH_BITS];

/* NAT tables — per type (SNAT=0, DNAT=1); hash mode uses nat_hashtables */
#ifndef NAT_HASH_BITS
#define NAT_HASH_BITS 8 /* 256 buckets; override via make NAT_HASH_BITS=N */
#endif
extern struct hlist_head nat_hashtables[2][NAT_IDX_COUNT][1 << NAT_HASH_BITS];
/* NAT locks (Type, Index, Bucket) */
extern spinlock_t nat_bucket_locks[2][NAT_IDX_COUNT][1 << NAT_HASH_BITS];
/* Counters shared between list and hash modes */
extern struct percpu_counter nat_counters[2];

/* Log info — active_logi_list kept for LRU eviction even in hash mode */
extern struct list_head active_logi_list;
#ifndef LOG_HASH_BITS
#define LOG_HASH_BITS 7 /* 128 buckets; override via make LOG_HASH_BITS=N */
#endif
extern struct hlist_head loginfo_hashtable[1 << LOG_HASH_BITS];

/* Counters */
extern unsigned int table_id;
extern struct percpu_counter state_tables_counter;
/* Inline macros to directly read for conditionals */
#define get_dnatted_count() percpu_counter_read(&nat_counters[NAT_DNAT])
#define get_snatted_count() percpu_counter_read(&nat_counters[NAT_SNAT])
extern struct percpu_counter loginfo_entry_counter;

/* Timeouts and Limits */
extern unsigned int state_lifetime;
extern unsigned int setup_shutd_state_lifetime;
extern unsigned int loginfo_lifetime;
extern unsigned int max_loginfo_entries;
extern int (*smartlog_func)(const struct sk_buff *skb,
 const struct response *res, const ipfi_flow *flow,
 const struct info_flags *flags);
extern unsigned int max_state_entries;

/* Print limiting */
extern unsigned int moderate_print[MAXMODERATE_ARGS];
extern unsigned int moderate_print_limit[MAXMODERATE_ARGS];

/* Locks */
extern spinlock_t rulelist_lock;
extern spinlock_t state_bucket_locks[1 << STATE_HASH_BITS];
extern spinlock_t loginfo_list_lock;
extern struct workqueue_struct *ipfire_wq;

/* Other */
extern bool we_are_exiting;

#endif /* IPFI_GLOBALS_H */

```

## offset.c

```
#include <linux/skbuff.h>
#include <linux/tcp.h>

int main() {
 printf("%lu\n", offsetof(struct sk_buff, _skb_refdst));
 return 0;
}
```

---